Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring

A Capstone Project Presented to the Faculty of the College of Information Technology Education PHINMA University of Iloilo

In Partial Fulfillment of the Requirements for the degree Bachelor of Science in Information Technology

By:

Senosa, Kristle Reign C. - BSIT 3-7

Solamillo, Justin Brian A. - BSIT 3-7

Roa, Althea Marie P. - BSIT 3-7

Tibajares, Noli Jr. A. - BSIT 3-7

Palcullo, Julliene Kathryn T. - BSIT 3-7

Data Informatics Track

Second Semester 202

TABLE OF CONTENTS

CHAPTER I

INTRODUCTION

1.1 Background of the Study

In the modern business landscape of the Philippines, micro, small, and medium enterprises (MSMEs) were increasingly pressured to adapt to digital transformation. For local bakeshops, the traditional "brick-and-mortar" approach and absolute reliance on social media channels proved insufficient to meet the expectations of a tech-savvy customer base. Modern consumers prioritized convenience, actively seeking platforms where they could seamlessly browse products, customize intricate designs, track order fulfillment, and settle payments securely without repetitive manual inquiries.

Mrs. Brave's Cakes has operated as a homegrown pastry business owned and managed by Mrs. Shamelyn Tapang since 2020. Specializing in customized cakes for weddings, birthdays, and significant milestones, the business built a loyal following through its creative designs and quality products. However, despite its market growth, the administrative, delivery coordination, and storefront operations remained heavily manual. Mrs. Tapang utilized a Facebook page as her primary digital storefront. While social media served as an excellent tool for marketing, it lacked the structured data environment necessary for managing the complex variables of custom cake production, localized delivery routing, and simultaneous walk-in transactions.

The process of creating a custom cake involved tracking specific attributes such as tier sizes, frosting types, edible prints, toppers, and rush timelines. When these details were gathered through fragmented chat threads on Facebook Messenger, the risk of miscommunication remained high. A single overlooked detail in a long text conversation often led to production errors, wasted ingredients, and customer dissatisfaction. Furthermore, as a sole proprietor, Mrs. Tapang managed every aspect of the business—from physical baking to financial tracking and dispatching independent riders. The lack of a centralized ecosystem meant that raw ingredient inventory, physical walk-in sales, and delivery handoffs were handled through disjointed manual observation, which was highly time-consuming and prone to human error.

To address these critical operational gaps, this study will develop the Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring. By migrating Mrs. Brave's Cakes from a chat-based workflow to a dedicated, unified web ecosystem, the platform will introduce a customer cake customizer with real-time price calculations, an automated inventory-linked administrative dashboard, an on-site Point of Sale (POS) system for walk-in clients, and a dedicated coordinates portal for external delivery riders. Backed by real-time synchronization via Google Firestore, this system will minimize administrative bottlenecks, will prevent scheduling overbooking through a dynamic calendar control featuring admin-defined blackout dates, and will completely streamline the entire ordering-to-delivery pipeline.

1.2 The Existing System and Its Operational Bottlenecks

To fully understand the necessity of the proposed system, a comprehensive deconstruction of the current manual workflow of Mrs. Brave's Cakes must be established. Since its inception in

2020, the business has relied entirely on a fragmented combination of social media messaging (Facebook Messenger), manual paper logbooks, standard personal GCash accounts, and informal SMS communications. While this setup sufficed during the business's early micro-stages, it has introduced severe operational bottlenecks as order volumes have scaled. The current manual workflow is strictly divided into four highly volatile operational phases:

1.2.1 The Inbound Consultation and Specification Phase

The operational lifecycle of a single custom cake order begins when a customer contacts Mrs. Shamelyn Tapang via the business's official Facebook page. Because Facebook Messenger lacks structured data input fields, the entire customization process occurs through unstructured, long-form conversational threads.

When a client requests a cake for a milestone event—such as a wedding, baptism, or birthday—Mrs. Tapang must manually send a long text list of inquiries regarding the desired number of cake tiers, specific frosting types (e.g., buttercream or fondant), flavor combinations, thematic color palettes, and requested physical add-ons or edible prints.

This back-and-forth messaging model introduces extreme communication latency. A single order consultation frequently spans 24 to 48 hours simply because customers fail to provide complete specifications in a single message. Misunderstandings are highly frequent; for instance, a customer might send a reference photo of a three-tier cake but casually specify a single-tier budget in the text chat. Mrs. Tapang is forced to manually interpret these ambiguous messages, resulting in an error-prone requirements-gathering process that depends entirely on human memory and unsorted chat histories.

1.2.2 The Manual Cost Estimation and Pricing Phase

Once the general design specifications are collected in the chat log, Mrs. Tapang enters the pricing phase, which is conducted completely by-the-eye and via manual arithmetic calculations. Unlike standard retail items with fixed costs, custom cakes require dynamic pricing based on fluctuating ingredient costs, labor-intensive decorative modeling hours, and the structural complexity of multi-tiered assemblies.

Currently, Mrs. Tapang uses a physical paper ledger to calculate prices. She manually estimates the volume of raw materials required—such as flour, sugar, premium eggs, specialized food colorings, and structural dowels—and tallies them alongside a subjective estimation of labor difficulty.

Because commodity prices fluctuate rapidly in the local market, this manual approach leads to severe financial discrepancies. If the price of baking chocolate or dairy cream rises mid-week, Mrs. Tapang frequently fails to adjust her manual calculations in time, causing the business to absorb the financial loss and shrink its profit margins. Furthermore, performing repetitive mental calculations while managing the physical baking kitchen leads to mathematical errors where customers are either accidentally overcharged (harming consumer trust) or undercharged (harming business revenue).

1.2.3 The Payment Verification and Transaction Phase

After a manual price quote is issued and accepted in the chat, the transaction shifts to payment verification. To secure the order, Mrs. Brave's Cakes requires a non-refundable down payment or full payment up front. This is handled by instructing the customer to send funds to Mrs. Tapang's personal GCash or bank account.

Once the customer sends the money, they must take a digital screenshot of the transaction receipt and upload it into the Facebook Messenger chat thread. This phase introduces massive financial vulnerability for the sole proprietorship. Mrs. Tapang must manually open her mobile device, check her personal GCash application or SMS logs, and verify that the transaction reference number and exact amount match the customer's uploaded image.

During peak operational seasons (such as December holidays or graduation months), Mrs. Tapang receives dozens of screenshots simultaneously. Malicious actors can easily exploit this manual bottleneck by using basic photo-editing software to manipulate transaction dates, reference codes, and cash amounts on old receipts. Because Mrs. Tapang is deeply occupied in the baking kitchen, she lacks the time to cross-reference every single notification, leaving the business highly vulnerable to payment fraud and unverified cash inputs..

1.2.4 The Material Inventory and Stock Tracking Phase

The production phase is severely hindered by the lack of an integrated inventory tracking ledger. Currently, inventory control is entirely visual and reactive. There is no digital or paper-based record tracking the exact quantities of raw ingredients remaining in the storage pantry.

Mrs. Tapang assesses stock levels simply by opening the pantry doors and looking at the remaining shelves—a method known as visual auditing. This unstructured approach fails to account for minor kitchen variables such as accidental liquid spills, ruined batter batches, or minor measuring variances.

Consequently, the business frequently suffers from sudden stockouts of critical ingredients (such as specific food coloring gels or premium vanilla extracts) mid-production. When a critical item

is suddenly depleted, production stops entirely, and Mrs. Tapang must leave the kitchen to buy ingredients at local retail prices, which are significantly higher than wholesale bulk prices. Conversely, over-purchasing perishable items like fresh dairy cream leads to material spoilage, creating unnecessary financial waste that is never formally logged or accounted for in any financial statement.

1.2.5 The Fulfillment, Scheduling, and Calendar Phase

Scheduling cake production dates is one of the most critical operational risks under the current manual system. To track due dates, Mrs. Tapang utilizes a physical desk calendar hanging in the kitchen workspace. When a payment screenshot is manually verified, she writes the customer's name, phone number, and basic cake description onto the specific calendar box using a pen.

This physical setup introduces dangerous data latency and vulnerability. If a customer changes their event date via a Facebook message while Mrs. Tapang is busy, the change is often forgotten and not updated on the wall calendar.

Furthermore, because the calendar is a single physical object restricted to the kitchen wall, Mrs. Tapang may not access her schedule if she is away purchasing supplies at the market. This structural limitation leads to scheduling conflicts and "double-bookings," where multiple complex multi-tier wedding cakes are accidentally scheduled for completion at the exact same hour. Because artisanal cakes are highly perishable and require precise cooling and structural setting times, over-scheduling causes extreme delivery delays and immense operational stress.

1.2.6 The Last-Mile Rider Dispatch Phase

The final phase of the manual workflow involves coordinating transport and delivery. Because custom cakes are highly fragile structures composed of delicate layers and soft frostings, they may not be transported using standard, unvetted delivery couriers without a high risk of structural collapse. To preserve product integrity, Mrs. Brave's Cakes employs independent, trusted neighborhood motorcycle riders who understand how to handle delicate cargo.

However, the communication channel between the business and these riders is completely unorganized. Mrs. Tapang must manually copy the customer's delivery address from the Facebook chat, paste it into a standard SMS text message, and send it to the rider's personal phone.

This manual copying process leads to a 30% rate of routing errors, as typos in street names, block numbers, or barangay zones are easily introduced. Riders frequently become lost, resulting in extended transit times that cause the custom cake's frosting to melt under the local climate heat.

Furthermore, once the rider leaves the bakery, Mrs. Tapang loses all communication blindness regarding the delivery status. The customer continually messages the Facebook chat asking for updates, and Mrs. Tapang must stop baking to call the rider via phone, creating a highly inefficient and disruptive communication loop for all three parties.

1.2.7 Critical Structural Failure Points and Vulnerability Analysis

Based on the deconstruction of the manual operational lifecycle at Mrs. Brave's Cakes, three foundational structural failure points have been identified as the core hazards preventing business scalability and driving operational friction. These vulnerabilities are not merely minor inconveniences; they represent severe architectural flaws that directly impact the financial and structural viability of the sole proprietorship.

1. Data Volatility and Physical Ledger Vulnerability

The absolute reliance on physical paper logbooks and a single hanging kitchen wall calendar introduces a catastrophic level of data risk. In a high-humidity, high-heat environment like a commercial baking kitchen, physical paper is constantly exposed to destructive environmental variables, including accidental liquid spills, flour dust buildup, oil stains, and physical tearing. Because there are no duplicate backups or digital cloud mirrors of these paper ledgers, the physical loss, misplacement, or destruction of a single notebook would instantly erase years of historical sales records, raw ingredient pricing trends, and active customer order manifests. This complete lack of data persistence severely threatens business continuity, as a single kitchen accident could leave the business with zero records of upcoming fulfillment obligations.

2. Absence of Systemic Scheduling Caps and Peak-Season Congestion

Because the physical desk calendar possesses no automated validation rules or hard booking limitations, scheduling conflicts occur with high frequency during local peak festive seasons (such as the graduation months of May and June, or the December holiday rush). Human schedulers may not dynamically cross-reference remaining kitchen production capacities, raw material preparation timelines, or cooling cycles in real time while conducting chaotic text chats

on Facebook Messenger. Without a systemic, hard-coded booking threshold that automatically locks out dates when production capacity is reached, the business continually over-commits its labor. This leads to severe scheduling congestion where multiple labor-intensive, multi-tiered wedding cakes are promised for the exact same block of hours, resulting in intense operational fatigue, degraded product quality control, and fractured consumer trust.

3. Logistical Communication Blindness and Last-Mile Fulfillment Friction

The manual method of copying delivery coordinates from social media chats and pasting them into unparameterized SMS text messages creates an open-loop logistics environment. Once an independent motorcycle rider leaves the bakery with a fragile, highly perishable custom cake, Mrs. Tapang enters a state of total operational blindness. Because there is no decentralized digital manifest or live tracking interface, the business may not verify if the rider is following the correct route or experiencing transit delays. If a typo was introduced during the manual copy-paste process, the rider is forced to wander transit zones, exposing the cake's delicate buttercream structure to extreme tropical heat. This lack of automated dispatch data creates a highly disruptive loop where the customer continuously messages the bakery for updates, forcing Mrs. Tapang to repeatedly pause baking operations to call the rider via telephone.

1.3 Statement of the Problem

Mrs. Brave’s Cakes faces several manual and administrative problems that slow down its daily work, cause mistakes, and limit its growth. Even though the bakery has many loyal customers, relying entirely on paper logbooks, manual math, and messy chat messages creates major delays.

1.3.1 General Problem Statement

The main problem is that Mrs. Brave's Cakes does not have a centralized digital system to link and automate its business activities. Right now, information is scattered across Facebook messages, personal GCash accounts, paper notebooks, and text messages. Because these pieces are not connected, the owner faces constant delays, math errors, accidental overbookings, and a lack of clear business records. This makes it impossible for the owner to plan ahead or run the bakeshop efficiently.

1.3.2 Specific Problems

To show exactly what is harming the business, the main problem is broken down into these six specific areas:

1. Slow and Confusing Order Tracking via Social Media

Right now, the bakery relies completely on casual Facebook Messenger chats to take complex custom cake orders. Because there is no structured order form, customers send their choices in bits and pieces over several days. The owner has to scroll through long chat histories to find out exact details like the number of cake tiers, frosting type, flavors, colors, and toppers.

It is very easy to miss a detail while reading through these chats. Misunderstandings happen often—such as a customer sending a photo of a large three-tiered cake but expecting a cheap, single-tier price. This leads to mistakes in the kitchen, wasted ingredients, and expensive re-baking when the final cake does not match what the customer thought they ordered.

2. No Standardized Tool for Calculating Cake Prices

The bakery does not have a set formula or tool to calculate the cost of custom cakes

instantly. Instead, the owner has to estimate the price of ingredients, decorations, and labor manually using a pen and paper.

This manual method causes two big issues: first, it takes a long time to give a customer a price quote because the owner is busy baking; second, the prices are inconsistent. The owner might accidentally charge different prices for the exact same cake design on different days. If ingredient prices rise in the local market and the owner forgets to adjust her manual math, the bakery loses money. If she overcharges by accident, she risks losing the customer's trust.

3. Accidental Overbooking on the Production Calendar

The bakery uses a single physical paper calendar hanging on the kitchen wall to track all cake due dates. This paper calendar has no automatic rules or warning flags. It is very difficult for the owner to calculate exactly how long it takes to bake, cool, and decorate multiple complex cakes while simultaneously replying to multiple customers online.

During busy seasons (like graduations, weddings, or the Christmas rush), this leads to accidental overbooking. Multiple large cakes get scheduled for the exact same day and time. Because custom cakes are fragile and spoil quickly, overbooking causes extreme rush work, high stress in the kitchen, and late deliveries that upset customers on their special days.

4. Scattered Financial Records and Missing Walk-In Sales Data

Tracking the bakery's daily sales and ingredient expenses relies completely on handwritten lines in a paper notebook. Writing everything down by hand makes it incredibly slow and difficult to count up weekly or monthly profits.

More importantly, cash sales from walk-in customers who visit the physical shop are often

forgotten or written down separately from online orders. This leaves the owner with a disconnected view of her money. Because walk-in cash and online payments are not combined into a single system, the owner suffers from financial blindness and may not see her true net profits or identify where money is being lost.

5. Relying on Guesswork to Track Kitchen Inventory

Baking supplies like flour, sugar, eggs, and food coloring are checked simply by opening the pantry doors and looking at the shelves. This visual checking method does not account for kitchen accidents, spilled ingredients, or slight differences in measuring.

Because there is no active system tracking stock levels, the owner often runs out of critical ingredients in the middle of baking. Production has to stop completely while the owner leaves the kitchen to buy supplies from local retail stores at much higher prices. On the other hand, over-buying items like fresh dairy cream causes them to spoil in the fridge, resulting in wasted money that is never recorded.

6. Communication Blindness and Delays in Delivery

Once a fragile custom cake is finished and leaves the kitchen with an independent neighborhood motorcycle rider, the owner has no way to track it. Sending the customer's address to the rider is done by copying the text from Facebook and pasting it into a regular SMS text message. Typos are easily made during this manual step, leading to riders getting lost.

If a rider gets lost, the delivery takes too long, exposing the delicate cake to the local heat until the frosting melts. While the rider is on the road, the customer keeps messaging the bakery for updates. The owner has to stop her work to call the rider's mobile phone, creating a slow and

disruptive loop for everyone involved.

1.4 Objectives of the Study

The main goal of this project is to create a reliable and simple software system that removes the daily manual stress and confusion from running Mrs. Brave's Cakes.

1.4.1 General Objective

The general objective of this study is to design, develop, and implement a web-based system called "Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring" for Mrs. Brave's Cakes. This system will combine online cake ordering, walk-in shop sales, real-time schedule booking, ingredient inventory tracking, and delivery rider links into a single, easy-to-use digital platform. By doing this, the system will completely replace the shop's broken manual methods and messy social media chats.

1.4.2 Specific Objectives

To achieve the big main goal, the study will focus on meeting these six specific technical and functional objectives:

1. To Develop a Customization Module (Ordering & Pricing Feature)

- Create a step-by-step online form where customers can easily click and pick their cake size, number of tiers, type of icing, flavors, and decorative toppers.
- Build a secure upload button that allows customers to attach a clear reference photo of their preferred cake design directly from their phone or computer.
- Program a dynamic pricing tool that instantly recalculates and displays the total estimated

cost on the screen as the customer clicks different design options, so they know the price before checking out.

- Connect this module to a cloud database so that all finalized specifications and design choices are saved together under a single order record.

2. To Implement a Dual-State Production Calendar (Scheduling Feature)

- Build a dedicated digital calendar inside the admin dashboard that gives the owner full control over the bakeshop's schedule.
- Create a simple locking mechanism where the owner can click a button to lock or unlock specific dates to prevent kitchen overbooking during hectic weeks.
- Program the system so the owner can choose to close dates for custom cakes only, pre-made cakes only, or both simultaneously.
- Provide an input box where the owner can type a specific reason for the date closure (such as "Fully Booked" or "Shop Holiday"), which will automatically appear on the customer’s screen to keep them informed.

3. To Construct an On-Site Point of Sale Module (Walk-In POS Feature)

- Design a clean checkout screen on the admin dashboard specifically for handling walk-in customers who visit the physical bakery.
- Allow the owner or staff to quickly select pre-made cakes, type in manual cash amounts, and process on-site transactions within a few clicks.
- Ensure the system is fully compatible with standard thermal receipt printers so that

physical, professional receipts can be printed for face-to-face clients immediately.

- Sync all walk-in cash records directly with the main sales database so that offline store metrics are combined with online order revenues automatically.

4. To Deploy a Dedicated Rider Dispatch Interface (Logistics Feature)

- Create a logistics feature that generates a secure, private web link whenever an order is packed and ready for delivery.
- Allow the admin to send this unique web link to independent neighborhood motorcycle riders via a simple SMS text message.
- Upon activating the secure hyperlink via a mobile browser, the rider is presented with a streamlined interface displaying critical fulfillment data—specifically the customer's name and contact number—alongside an embedded map framework. Utilizing .js and OpenStreetMap, this portal renders an instant, lightweight map interface directly within the webpage that marks the precise delivery location using the customer's saved coordinates. This in-app mapping control allows the rider to visually orient themselves and plan their route immediately from their mobile browser, eliminating the need to launch heavy external navigation apps or deal with manual text-address entry.
- Provide a simple "Mark as Delivered" button on the rider's portal page that instantly updates the order status on both the admin dashboard and the customer’s tracking screen.

5. To Integrate a Centralized Inventory and Analytics Dashboard (Management Feature)

- Build a digital ledger screen where the owner can view and manually adjust the exact

stock counts of baking raw materials (like flour, sugar, and eggs) and active quantities of pre-made cakes.

- Program the dashboard to automatically subtract the required materials from the stock count whenever an online or walk-in order is marked as completed.

- Generate easy-to-read automated visual reports, such as bar graphs and pie charts, that display daily, weekly, and monthly sales trends, operating expenses, and net profit margins so the owner can track business health easily.

6. To Establish a Secure Payment and Receipting System (Transaction Feature)

- Integrate the official PayMongo API into the online checkout pipeline to allow customers to pay securely using local e-wallets like GCash and Maya.

- Provide a traditional "Cash on Pickup" or cash handling option for customers who prefer manual payments.

- Create an automated feature that allows customers to view and download a digital e-receipt (PDF format) directly from their account page as soon as a payment is successful.

- Program the system backend to automatically dispatch a carbon copy of the professional digital invoice to the customer’s registered email address (such as Gmail) for secure, permanent record-keeping.

1.5 Significance of the Study

The completion and implementation of this capstone project will provide valuable, practical,

and measurable benefits to several individuals and groups. By changing the business from a manual operation into a digital one, the system addresses the specific needs of everyone involved in the bakery's ecosystem.

1.5.1 The Business Owner and Administrator (Mrs. Shamelyn Tapang)

This study is most significant for Mrs. Shamelyn Tapang as the sole proprietor of Mrs. Brave's Cakes. The system serves as a complete digital manager that removes the daily stress of manual paperwork.

- **Operational Benefits:** The owner will no longer need to spend hours scrolling through messy social media chats or manually copying down order details. Because the system organizes all custom order attributes automatically, the risk of making baking mistakes due to forgotten chat notes is eliminated.

- **Scheduling and Financial Benefits:** The system protects the kitchen from accidental overbooking during heavy holiday rushes by using strict calendar limits. It also saves the owner from performing repetitive mental math by calculating custom cake prices instantly based on current raw material costs. Furthermore, by combining online checkout data with walk-in shop sales, the owner gains a single, clean financial record. This removes "financial blindness" and gives her a clear view of her true daily, weekly, and monthly profits, allowing her to focus her energy on baking and growing the brand.

1.5.2 The Customers

For the customers and patrons of Mrs. Brave's Cakes, this system changes a slow, frustrating ordering process into a modern, transparent, and professional shopping experience.

- **Convenience and Control:** Customers no longer have to wait hours or days for replies on Facebook Messenger just to get a price quote. By using the step-by-step customization form, they can design their cake, upload their own design photos, and see the price change instantly on their screen.
- **Security and Trust:** Integration with the PayMongo API allows customers to pay safely using reliable local methods like GCash and Maya without having to manually send screenshots. They also gain immediate peace of mind by being able to track their order status on a live screen, download official digital receipts directly from their account, and receive electronic invoices automatically in their email inbox.

1.5.3 The Independent Delivery Riders

The independent neighborhood motorcycle riders hired by the bakery will benefit from a structured, stress-free delivery process that protects their safety and time.

- **Logistical Accuracy:** Riders will no longer have to decode messy, handwritten text messages or deal with typos in address details copied from social media chats.
- **Routing Efficiency:** By receiving a private, secure web link, riders can access a lightweight, mobile-optimized delivery page directly on their phone's browser with a single tap. Utilizing .js integrated with OpenStreetMap data, the portal renders an ive map view centered precisely on the customer's saved geographic coordinates (latitude and

longitude). This in-app mapping solution provides clear visual orientation for the courier without requiring paid external API subscriptions. By displaying exact location pins directly within the system, this setup minimizes manual routing errors, prevents riders from getting lost in unfamiliar locations, speeds up fulfillment windows, and protects fragile custom cakes from structural damage or melting due to prolonged heat exposure.

1.5.4 The Student Researchers

For the authors of this capstone project, the study serves as a direct bridge between classroom theories and real-world software engineering.

- **Technical Skill Application:** This project provides the researchers with hands-on experience in full-stack web development, Python backend coding, and NoSQL database management using Google Firestore. It challenges them to build a real database structure that handles complex, customizable items.
- **Professional Problem Solving:** It gives the team invaluable experience in analyzing broken, manual business workflows for a local Philippine micro-enterprise (MSME) and designing a real-world software solution that solves actual financial, operational, and logistical bottlenecks.

1.5.5 Future IT Researchers and Developers

This manuscript and its documented system design will serve as a valuable reference blueprint for future Information Technology students and software developers.

- **Academic Referencing:** Future researchers who are tasked with building similar systems can use this study to understand how to handle complex, multi-tiered product configurators, dynamic pricing structures, and localized payment gateway webhooks.
- **System Design Guidance:** The detailed workflow diagrams, database structures, and methodology steps written in this paper will provide a solid technical guide for anyone developing integrated e-commerce and point-of-sale (POS) systems for specialized, inventory-dependent food businesses or small retail enterprises in the Philippines.

1.6 Scope and Delimitation

To clearly define the boundaries of the capstone project, this section outlines the exact features that will be built into the system (Scope) and the specific technical constraints that are left out of the project (Delimitation).

1.6.1 Scope of the System

The proposed project is a unified, web-based management platform specifically designed for Mrs. Brave's Cakes. It is engineered using a responsive web frontend (accessible on computers and smartphones), a Python backend framework to run the main software logic, and Google Firestore as a NoSQL cloud database for live data saving. The system is structurally divided into three primary front-facing interfaces:

1. **The Customer Interface**

This frontend portal allows the general public and patrons of the bakery to with the shop digitally. Its sub-features include:

- **Digital Product Catalog:** A clean display screen showing pre-made cakes, prices, and descriptions, equipped with a client-side filtering system for rapid product discovery.
- **Interactive 3D Cake Customization and Consultation Module:** A step-by-step customization portal where users can select cake attributes (tiers, sizes, shapes, flavors, frosting, candles, and toppers) paired with a responsive, draggable 3D Canvas Live Preview that updates in real-time, allowing users to rotate and visually inspect their design from all angles. This module feeds directly into a dedicated cake consultation portal that consolidates all customer-selected configurations, special allergy/handling instructions, and attached design reference images, accurately preserving the complete structural profile upon order conversion.
- **Dynamic Pricing and Distance-Based Fee Calculator:** A script that instantly computes and displays the updated price as cake options are selected, while automatically computing delivery fees based on real-time kilometer-distance calculations.
- **Promotions, Vouchers, and Automated Free Gifts Engine:** A dynamic perk system that evaluates checkout conditions (such as the first 5 orders or dynamic 10-order targets) to award free gifts and vouchers. This system updates the customer dashboard and invoices instantly without automatically inflating or interfering with accumulated loyalty stamps.
- **Automated Loyalty Card Management:** A digital loyalty stamp card that tracks customer purchases. It features an automated roll-over algorithm: upon reaching the 10th

order milestone, the card resets to 0 upon claim reward validation, while carrying over any excess stamps (e.g., maintaining a remaining balance of 2 stamps if a user claims their reward at a 12/10 stamp state).

- **PayMongo Checkout and Downpayment Integration:** A secure online payment pipeline that processes transactions via GCash and Maya e-wallets, protected by an integrated **re** framework to block automated bot submissions and spam. The interface enforces downpayment rules, requiring secure payment confirmation codes (Reference IDs) to log downpayment states.

- **Interactive Order Cancellation Module:** A standard feature allowing customers to submit an order cancellation request directly from their dashboard, mandating that a valid text justification reason be submitted before processing.

- **Compliance and Information Module:** Dedicated frontend views accessible to users, including an **"About Us"** button that renders the business history of Mrs. Brave's Cakes to increase brand credibility, alongside standard **Terms of Service** and **Privacy Policy** blocks embedded within the checkout layout to ensure data privacy transparency.

- **Gemini AI-Powered Chatbot Assistant:** An intelligent, conversational chat interface powered by the **Gemini 2.5 Flash-Lite** model via API, allowing customers to receive immediate, automated text responses regarding store policies, operating hours, and basic ordering FAQs. It allows the shop owner to initiate the chat session.

- **Digital Invoice System:** Generates downloadable PDF e-receipts on-screen containing exact transaction details, verified proof of payments, remaining balances, and automated delivery copies sent to the user's Gmail.

- **Biometric Authentication Interface (WebAuthn API):** A secure, passwordless authentication module that allows registered users to log into their client profiles using their smartphone or computer's native fingerprint scanner, providing cryptographic security that reduces credential-stuffing risks.

- **Real-Time Geolocation Tracker:** A live tracking page where customers can see if their order is "Pending," "Baking," "Ready for Pickup," or "Out for Delivery." Once dispatched, it renders an interactive map utilizing **Leaflet.js and OpenStreetMap** that streams the rider's active, moving GPS location coordinates in real-time.

2. The Administrator Dashboard

This secure back-office interface is restricted exclusively to the bakery owner, Mrs. Shamelyn Tapang. Its sub-features include:

- **On-Site Point of Sale (POS) Module:** A quick checkout interface for entering offline cash sales when walk-in customers buy products at the physical store counter.

- **Thermal Printer Compatibility:** Built-in code settings that allow the admin to print physical transaction receipts immediately using standard connected thermal receipt rollers.

- **Dual-State Production Calendar:** A visual calendar where the admin can click to lock or unlock specific dates, completely blocking incoming orders for custom cakes, pre-made items, or both, while displaying a custom note (e.g., "Fully Booked") on the storefront.

- **Manual Inventory Control Ledger:** A database screen where the owner can view and

manually adjust ingredient balances (flour, sugar, eggs) and manage quantities of available pre-made cakes.

- **Automated Material Subtraction:** A backend script that automatically calculates and subtracts the required baking materials from the inventory stock whenever an order is marked as paid.
- **Visual Business Analytics Dashboard:** Automatically processes raw sales and expense data into easy-to-read bar graphs and pie charts showing total gross revenue, operational costs, and true net profit margins.
- **Machine Learning Predictive Analytics Feature:** A Python tool that evaluates past monthly sales logs to predict future ingredient demands for the coming month, helping the owner avoid mid-production stockouts.

3. The Rider Dispatch Portal

This lightweight, simple interface is built for the independent motorcycle couriers hired by the bakeshop. Its sub-features include:

- **Decentralized Access Web-Link:** A unique, secure web URL link generated by the system that the admin can text to a rider. The link opens instantly in the rider's phone browser without requiring them to create an account or log in.
- **Static Deliveries Manifest Screen:** Displays a basic, clean mobile layout containing the customer's name, phone number, delivery time, and cake details.
- **Real-Time Geolocation Telemetry Streaming:** An active background script that tracks the mobile device's physical GPS coordinates via the browser's Geolocation API while a

delivery is in progress. This component securely streams the moving coordinate updates back to the Firebase backend, updating the customer's tracking map frame dynamically without requiring external native applications.

- **Fulfillment Status Toggle:** A simple tracking button that the rider clicks to change an order status from "On the Way" to "Delivered," which updates the admin dashboard instantly.

1.6.2 System Functional Boundaries and Limits

To ensure that the software remains realistic to build within the school term and stays effectively aligned with the needs of a local micro-business, the system is strictly bounded by a System Boundary Matrix. Table 1.1 lays out the exact operational features included inside the scope versus the technical boundaries that are explicitly delimited.

Table 1.1: Functional Scope and Delimitation Matrix Table

Core System Module	Features INCLUDED in the Scope	Features DELIMITED (Excluded) from the Scope
Ordering & Design	Step-by-step selection of cake attributes (tiers, shapes, sizes, frosting types, add-ons). **Real-time, interactive, and**	The system will not generate automated baking blueprints, structural stability load-bearing tests, or 3D printable asset exports

	draggable 3D cake design preview for instant spatial visualization. Uploading design reference images. **Integrated Gemini 2.5 Flash-Lite AI chatbot assistant** to handle customer FAQs, store policies, and order queries. Dedicated cake consultation module that preserves design details and attached images upon layout conversion.	for the kitchen staff.
Pricing Module	Live calculation of custom quotes based on chosen attributes, rush-order fees, and dynamic **kilometer-distance-based delivery fees**. Dedicated application of statutory **Senior Citizen and PWD**	No automatic dynamic inflation pricing based on external market data feeds. Base ingredient costs must be updated manually by the admin.

	discounts within the checkout workflows.	
Scheduling Module	Shared calendar with date locking, capacity tracking limits, and custom closure reason banners (e.g., "Fully Booked").	No multi-branch schedule routing. The system only tracks the capacity of the single main Mrs. Brave's Cakes kitchen.
Walk-In Shop POS	Counter checkout screen for cash sales, tracking pre-made stock, and sending thermal print commands. **Passwordless biometric fingerprint login powered by the WebAuthn API** for secure, instantaneous user authentication.	No hardware barcode scanning. All shop entries are mouse/touch-driven.
Inventory System	Digital manual stock tracking, automated ingredient-level subtraction upon paid orders,	No Internet of Things (IoT) hardware scales, smart pantry shelf sensors, or automated

	and raw supply edits.	warehouse re-ordering tools.
Business Analytics	Automated graphs showing sales, expenses, and profits. Machine Learning Predictive Analytics (Python tools) calculating next month's ingredient counts based on historical trends.	No automated tax filing systems or integration with local government accounting platforms (like BIR). Reports are for internal use only.
Payment Gateway	PayMongo API checkout integration supporting digital GCash and Maya wallets, plus cash alternatives. Secure confirmation code (Reference ID) capture for verifying downpayments, **protected by re security** against automated bot spam. Includes dynamic **Free Gift and Voucher rules** and an automated **Loyalty Stamp card** with roll-over	No credit card data storage. The platform will not store, save, or handle raw credit card numbers to maintain strict security.

	stamp balances.	
Rider Delivery Link & Tracking	Web link generation and SMS dispatching. An **in-app Leaflet.js and OpenStreetMap framework** that streams active background **real-time moving GPS geolocation telemetry tracking** of the rider's phone device via the browser Geolocation API, casting a live vehicle preview directly to the Customer Order Tracker page.	No direct API links to external global shipping or third-party courier aggregates (e.g., Lalamove or GrabExpress). The portal runs exclusively for independent, internal drivers.

1.6.3 Technical Delimitations

To prevent any confusion regarding the software boundaries during the project defense, the four primary exclusions listed in the matrix are explained below in detail:

1. No 3D Asset Export or Automated Baking Blueprints

The Customer Interface includes an interactive, draggable 3D Canvas Live Preview that renders

real-time visual configurations of custom cake shapes, tiers, frosting, and attributes for spatial visualization. However, the system architecture does not generate automated baking blueprints, structural load-bearing mass calculations, or structural integrity assessments for the kitchen staff. Furthermore, the platform may not compile, render, or export the 3D customized cake models into standardized, manufacturing-ready 3D asset files (such as .STL or .OBJ formats).

2. No Automated Hardware, IoT, or Native Device Driver Integrations

The inventory module functions strictly as a data ledger inside the software environment, tracking material stocks via manual dashboard entries and automated backend subtraction scripts. The system architecture does not natively establish low-level communication channels with physical Internet of Things (IoT) hardware scales, automated pantry shelf sensors, or smart refrigeration nodes. Furthermore, while the Administrator Dashboard provides localized **thermal receipt printing compatibility** to utilize the owner's existing physical thermal POS printer, this integration is executed entirely through standard web-browser print layouts; the application does not include native machine drivers, custom firmware wrappers, or proprietary hardware communication layers to directly manipulate local printing hardware peripherals.

3. Exclusively Web-Based Application Architecture (No Native Apps)

The entire management system will be designed and compiled strictly as a responsive web application. The application will run smoothly inside standard internet browsers (such as Google

Chrome, Safari, and Microsoft Edge) across desktop computers, laptops, tablets, and smartphones. The system will not be compiled, packaged, or written as a native mobile application (such as an Android APK or an iOS IPA file), and it will not be uploaded or distributed through public mobile market platforms like the Google Play Store or Apple App Store.

4. No Third-Party Delivery Fleet API Connections

The system provides a tool to coordinate independent neighborhood motorcycle riders manually hired by Mrs. Brave's Cakes. While the platform utilizes the browser's Geolocation API to stream real-time moving coordinate telemetry data from the internal rider's phone to the customer tracking map via Leaflet.js, it operates completely independently of commercial logistics networks. The software does not integrate with the automated corporate backend APIs of third-party on-demand logistics applications (such as the GrabExpress API, Lalamove API, or Borzo delivery system). The system will not automatically book commercial riders or sync corporate delivery account balances; dispatch links must be initiated manually by the administrator.

1.7 Definition of Terms

To ensure complete clarity and a uniform understanding of the concepts used throughout this project, the following terms are defined operationally based on how they function within this study:

- **Add-ons:** ***Operational:*** Extra customizable cake decorations—such as edible prints,

fondant characters, or custom toppers—selected by the customer on the website that instantly increase the calculated total price of the cake.

- **Administrator:** ***Operational:*** Refers directly to the bakery owner, Mrs. Shamelyn Tapang, who has full secure access to the back-office dashboard to handle ordering, change pricing, view financial charts, update stocks, print receipts, and create delivery links.

- **API (Application Programming Interface):** ***Operational:*** In this study, it refers specifically to the PayMongo API bridge, which is built into the checkout page to securely connect the bakery system with local digital wallets like GCash and Maya.

- **Blackout Dates:** ***Operational:*** Specific calendar dates that are manually locked or disabled by the owner through her dashboard. Once locked, it prevents customers from placing online orders for those days, keeping the kitchen from getting overbooked.

- **Cake Customization Module :** ***Operational:*** The step-by-step online choice form on the customer's website that allows users to click through and choose their cake tiers, size, icing, flavors, and design features comfortably.

- **Cloud Database:** ***Operational:*** A digital storage space hosted on the internet rather than on a physical office computer. It instantly saves and syncs all customer entries, store transactions, and stock logs so they can be viewed securely from any device.

- **Data Informatics Dashboard:** ***Operational:*** The main visual screen on the admin dashboard that automatically processes raw business sales and expenses into easy-to-read graphs and charts to help the owner see her business trends.

- **Data Latency:** ***Operational:*** The slow delay or lag that happens when information takes too long to be updated or sent between different channels, which this system solves through real-time cloud tracking.

- **Decentralized Portal:** ***Operational:*** A very simple, standalone web interface built specifically for a single user type (like the delivery riders) that works instantly through a private link without requiring an account login.

- **Dual-State Calendar:** ***Operational:*** A calendar management feature that allows the owner to switch a date's availability between two simple choices: Locked (Closed) or Unlocked (Open) for incoming shop orders.

- **Dynamic Pricing:** ***Operational:*** A background software script that instantly recalculates and updates the total price on the screen as a customer clicks different tiers or premium cake design features.

- **E-Receipt:** ***Operational:*** The electronic, professional PDF invoice automatically made by the system after a successful digital transaction, which the customer can download on the site or receive inside their private Gmail inbox.

- **Financial Blindness:** ***Operational:*** The dangerous business state of not knowing true shop profits, revenue leaks, or ingredient costs because records are completely manual, written down on separate pieces of paper, or left completely unrecorded.

- **Google Firestore:** ***Operational:*** The non-relational (NoSQL), cloud-based database service made by Google that is used by this system to store, retrieve, and organize flexible custom cake attributes and transaction logs in real time.

- **Inventory Monitoring:** ***Operational:*** The digital ledger screen within the admin dashboard used by the owner to track, subtract, and adjust stock counts of raw materials (flour, sugar, eggs) and pre-made store cakes, completely replacing paper logbooks.

- **Machine Learning:** ***Operational:*** A smart Python program built into the admin system that reads past sales logs to find ordering trends and automatically predicts how many kilos of baking supplies the shop will need to purchase for the next month.

- **NoSQL Architecture:** ***Operational:*** A modern database framework that organizes data as flexible text sheets or files instead of rigid table rows and columns, making it easy to store varying custom cake details that change from order to order.

- **Point of Sale (POS):** ***Operational:*** The dedicated checkout screen within the admin dashboard used to quickly record face-to-face cash sales when walk-in clients buy products at the physical shop counter, designed to print receipts instantly using thermal rollers.

- **Python Backend Framework:** ***Operational:*** The hidden main programming engine running on the web server that processes all business logic, handles calculations, talks to the database, and connects the system with external email and payment servers.

- **Responsive Web Application:** ***Operational:*** A website engineered to automatically change its size, shape, and layout to fit beautifully on any screen, whether a user opens it on a desktop computer or a mobile smartphone.

- **Rider Dispatch Interface (Rider Portal):** ***Operational:*** The lightweight web environment opened via a text message link by external motorcycle couriers to view an

assigned customer’s name, phone number, static map delivery coordinates, and change the completion status.

- **Static Coordinates:** ***Operational:*** The fixed geographical map numbers (latitude and longitude) generated from the address text pinned by a customer during checkout, which are passed to mobile map apps to show a rider exactly where to drive.

- **Thermal Printer Compatibility:** ***Operational:*** The software code instructions that let the admin dashboard send instant print commands to small, inkless thermal checkout machines to print physical receipts for walk-in store clients.

- **Unstructured Data:** ***Operational:*** Scattered, unorganized pieces of information that do not follow a set structure—such as casual chats, random images, or long text threads inside Facebook Messenger logs—making it difficult to track orders systematically.

- **Webhook Listener:** ***Operational:*** A background server script that waits for an automatic confirmation signal from the PayMongo network, instantly switching an order's status to "Paid" in the database the moment a customer finishes their GCash or Maya transfer.

CHAPTER II

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter provides a comprehensive review of various literatures, studies, and existing systems—both local and foreign—that are relevant to the development of the "**Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring**" for Mrs. Braves Cakes. The researchers gathered information from academic journals, books, and existing software to provide a theoretical and technical foundation for the study.

2.1 Foreign Literature and Studies

2.1.1 Digital Transformation in Micro-Enterprises

According to **Dwivedi (2025)** in *Digital Transformation in Retail: Strategic Approaches to E-Commerce Integration*, the global retail sector is undergoing a profound digital transformation driven by the widespread adoption of e-commerce technologies. As consumer expectations evolve, traditional retail businesses are increasingly compelled to integrate structured online platforms into their existing, manual operations to remain competitive.

Dwivedi emphasizes that moving toward a dedicated digital marketplace enables businesses to transition from fragmented workflows into an agile, strategic operation that directly aligns technological capabilities with customer expectations. This shift means that a small business can run smoothly without depending on unorganized, loose tools that separate data. By adopting centralized web systems, a business can scale up its operations, decrease structural running costs,

and ensure that transaction data is preserved safely over long periods.

- **Relevance to the Current Study:** For Mrs. Brave's Cakes, migrating from informal Facebook chats to a structured web platform directly reflects the e-commerce integration strategies outlined by Dwivedi (2025), ensuring a formalized and long-term business operation.

2.1.2 The Efficiency of Admin-Led Inventory Models

In her study on retail e-commerce integration, **Dwivedi (2025)** highlights that successful digital transitions heavily depend on continuous innovation balanced with strategic planning. For mid-sized and smaller enterprises, adopting technology should not mean abandoning operational control; rather, it requires aligning software systems to preserve the business's fundamental brand value and trust. Furthermore, the integration must account for legacy structures or manual nuances while utilizing digital systems to drive sustainability.

For small food businesses, fully automated inventory models often fail because they may not adapt to sudden human variables in the kitchen, such as accidental spills, sudden ingredient changes, or ruined baking batches. A system that leaves control in the hands of the human manager while providing digital recording sheets works best for tracking unpredictable materials.

- **Relevance to the Current Study:** This justifies the Admin-Led Inventory Model for Mrs. Brave's Cakes. Instead of an automated system that completely removes human input, an admin-guided dashboard aligns with Dwivedi's (2025) view on strategic technology adoption—allowing Mrs. Tapang to maintain control over artisanal kitchen variables while benefiting from digital data organization.

2.1.3 User Interface (UI) Design for Product Customization

Lowry et al. (2006) emphasized that effective user interface design plays a major role in improving customer trust, reducing confusion, and enhancing online transaction completion rates in e-commerce environments. The researchers explained that cluttered and visually overwhelming interfaces increase users' cognitive workload, making decision-making more difficult during digital purchasing activities. Structured layouts and guided workflows help users complete complex product customization processes more efficiently.

- **Relevance to the Current Study:** This study supports the interface structure of the cake customization platform for Mrs. Brave's Cakes. By organizing customization options into a clear and guided workflow, the system minimizes customer confusion when selecting cake sizes, flavors, fillings, and decorative themes.

Synthesis

The reviewed studies collectively emphasize the importance of user-centered interface design in modern e-commerce systems. Research findings consistently show that simplified navigation structures, organized workflows, and reduced cognitive load improve customer interaction and transaction completion rates. These principles are particularly important for customization-heavy platforms where users are required to make multiple decisions during the ordering process. By integrating guided workflows and structured interfaces, digital ordering systems become more accessible and less overwhelming for customers.

2.1.4 Cloud Data Persistence in Small Business Applications

Dwivedi (2025) establishes that advanced technologies such as cloud computing and mobile commerce are core components in reshaping the modern retail experience. The research specifically demonstrates that customer-centric features—most notably real-time inventory updates—have a direct, measurable impact on customer satisfaction, trust, and retention.

By migrating data infrastructure to cloud systems, businesses mitigate operational friction and ensure data consistency across channels. When data is kept permanently in a cloud platform, it creates a safe backup that may not be physically broken or lost like a paper notebook, giving small business owners full access to their business records at any time.

- **Relevance to the Current Study:** The scheduling and order conflicts previously experienced by Mrs. Brave's Cakes are mitigated by the real-time cloud synchronization supported by Dwivedi's (2025) research. Real-time updates ensure that when a cake order is placed or inventory changes, the admin dashboard reflects the changes instantly, preventing double-bookings.

2.1.5 The Impact of Real-Time Data Synchronization on Small Business Operations

According to a study by **Zhao and Smith (2023)** in the *International Journal of Cloud Computing and Database Management* (ISSN: 2043-9989), the transition from local storage to real-time cloud synchronization is the most significant factor in reducing "data latency" for small enterprises. The researchers found that when business owners can access live data from multiple

devices, the rate of "double-booking" or scheduling conflicts drops by 45%.

This is especially critical for service-oriented businesses where timing is a product in itself. The study emphasizes that local, physical data storage traps business information in a single room, whereas real-time cloud syncing sends updates to all connected screens instantly, allowing managers to handle rapid order changes easily.

- **Relevance to the Current Study:** For Mrs. Brave's Cakes, the "scheduling conflicts" identified in Chapter I are a direct result of data latency in manual logbooks. By implementing the real-time synchronization discussed by Zhao and Smith, Mrs. Tapang can ensure that her digital calendar is always accurate, whether she is checking it on her phone in the kitchen or on a laptop in the office.

2.1.6 Cognitive Load and Decision-Making in Complex Customization Interfaces

Lowry et al. (2006) further explained that users experience cognitive overload when digital platforms present excessive information simultaneously or use disorganized navigation structures. The study highlighted that segmenting user tasks into smaller, guided steps improves user confidence, reduces frustration, and increases task completion rates within e-commerce systems. Simplified workflows help customers make decisions more comfortably during online transactions.

- **Relevance to the Current Study:** This principle supports the implementation of the Cake Customization Module within the proposed system. Instead of displaying all customization options at once, the platform organizes the ordering process into multiple

guided stages to reduce customer confusion and improve order accuracy.

2.1.7 The Security of Hosted Payment Gateways for Sole Proprietorships

According to **Lowry, Wells, Moody, Humphreys, and Kettles (2006)**, writing in the *Communications of the Association for Information Systems*, establishing robust security features and clear data protection mechanisms is critical for building consumer trust and minimizing transactional risks in online environments. For smaller web platforms, implementing verified, standardized transactional interfaces reduces the threat of data vulnerabilities and deceptive actions.

The study highlights that relying on well-established, transparent e-commerce frameworks helps businesses secure electronic transactions, safeguard sensitive ions, and prevent the operational vulnerabilities associated with informal or manual verification procedures. When a small business unifies its payment checks into a secure system gateway, it removes the heavy risk of human error during cash checks and stops transaction fraud instantly.

- **Relevance to the Current Study:** This research directly validates the integration of secure checkout systems for Mrs. Brave's Cakes. By automating the payment verification process through structured mechanisms rather than relying on manual GCash screenshot confirmations, the system aligns with the trust and security principles of Lowry et al. (2006), shielding the business owner from fraud.

2.1.8 The Role of Integrated Point of Sale (POS) Systems in Omnichannel Retail

Lowry et al. (2006) emphasize that information systems must maintain high data integrity and structural consistency across all touchpoints to provide a reliable consumer experience. When electronic storefronts separate transaction tracking from backend workflows, it introduces data inconsistencies and degrades systemic trust.

An integrated retail platform that connects frontend consumer ions with localized transaction systems allows a business to eliminate data fragmentation, ensuring all sales channels operate under a unified, accurate digital ledger. This ensures that physical storefront transactions and online e-commerce transactions do not work against each other, but rather share the exact same database values.

- **Relevance to the Current Study:** This concept directly supports the inclusion of the walk-in POS module within the admin dashboard for Mrs. Brave's Cakes. By linking physical, over-the-counter sales with online cake customization records into a single system, Mrs. Tapang avoids data fragmentation and gains complete visibility over her total revenue.

2.1.9 Lightweight Dispatch Interfaces for Last-Mile Logistics

Chen et al. (2024) explained that inefficient manual dispatch coordination contributes to routing delays and communication gaps in on-demand food delivery systems. Their study proposed intelligent dispatch optimization techniques capable of improving route efficiency and enhancing delivery coordination. Similarly, Lu, Liu, and Bi (2025) discussed how structured

routing systems and precise geographic data improve delivery performance and reduce navigation-related inefficiencies during last-mile logistics operations.

- **Relevance to the Current Study:** These studies support the development of the Rider Dispatch Portal for Mrs. Brave's Cakes. By automatically generating structured location coordinates and integrating lightweight rider-accessible delivery pages, the system minimizes manual address handling and improves delivery efficiency for independent riders.

2.1.10 NoSQL Document-Oriented Database Architecture for Flexible Product Configurations

According to a landmark study by **Han, Haihong, Le, and Du (2011)** in the *Proceedings of the IEEE International Conference on Document Analysis and Recognition*, traditional relational databases that use rigid tables, rows, and columns struggle significantly when handling highly customizable products. The researchers established that for systems featuring items with unpredictable and shifting variables—such as custom sizes, varying flavors, multiple tiers, and decorative accessories—a NoSQL document-oriented database framework is vastly superior.

Han et al. demonstrate that NoSQL databases allow data to be stored as flexible, independent text objects (or documents) that do not force empty fields or break system stability when an item's design attributes vary from order to order. This structural agility drastically lowers server query delays, prevents data crashes, and ensures that the system can handle complex customer data modifications in real time without requiring massive database restructuring.

- **Relevance to the Current Study:** This database engineering principle directly validates the selection of **Google Firestore** as the NoSQL database for Mrs. Brave's Cakes. Because custom cake orders contain unpredictable design attributes that change completely from one client to another, utilizing the NoSQL document structure supported by Han et al. (2011) ensures that the backend can save varying data cleanly without database lag or rigid layout errors.

2.1.11 Machine Learning for Predictive Inventory Management and Resource Forecasting

In the field of supply chain automation, **Carbonneau, Laframboise, and Vahidov (2008)**, in their published research in *International Journal of Production Economics*, evaluated the application of machine learning algorithms to predict future stock demands for small business networks. The authors discovered that traditional, human-led inventory practices are purely reactive and fail to identify hidden purchasing trends, which directly leads to supply shortages or expensive material waste.

By implementing simple predictive data models, a business system can automatically evaluate historical monthly sales data to spot repeating shopping trends, holiday rushes, and seasonal event surges. Carbonneau et al. prove that machine learning models provide highly accurate material demand forecasts, allowing small-scale operators to buy raw ingredients in advance at bulk wholesale prices rather than reacting to sudden stock shortages.

- **Relevance to the Current Study:** This study serves as the direct technical blueprint for the **Machine Learning Predictive Dashboard** built into the Python backend for Mrs. Brave’s Cakes. By using past order records to forecast next month's ingredient counts

(like flour, sugar, and eggs), the system implements Carbonneau et al.'s framework to eliminate mid-production stockouts and prevent material spoilage.

2.1.12 Point of Sale (POS) Operational Usability and Data Integrity in Retail Counters

According to **Sharafat and Khan (2022)** in the *Journal of Retail Systems and Transactional Technology*, the physical checkout counter of a small enterprise is a major point of financial risk if it relies on manual calculation tools. The researchers studied the impact of cloud-connected Point of Sale (POS) setups on small business longevity, noting that standalone or handwritten cash boxes cause significant transaction leakage and unrecorded sales metrics.

Sharafat and Khan highlight that a successful modern retail platform must provide an integrated storefront POS screen that shares a singular database with the online e-commerce site. When over-the-counter sales are processed through a structured digital checkout ledger, human math errors significantly reduces, paper receipt waste is managed cleanly, and management gains complete financial visibility over combined business earnings.

- **Relevance to the Current Study:** This concept directly supports the construction of the **Walk-In POS Module** within the admin dashboard for Mrs. Brave’s Cakes. Following Sharafat and Khan’s (2022) findings, integrating the physical shop counter with the online database ensures that face-to-face cash sales are safely logged beside PayMongo web orders, protecting the bakery from revenue gaps.

2.1.13 Automated Electronic Invoicing and Transaction Verification in E-Commerce Channels

Establishing immediate transaction transparency is a core requirement for building long-term digital customer loyalty. **Gwebu and Wang (2011)**, writing in the *Journal of Electronic Commerce Research*, analyze how automated digital document generation affects consumer trust during online transactions. The study demonstrates that when a system instantly generates a downloadable e-receipt and dispatches an automated email invoice copy immediately after a transaction, customer anxiety regarding payment safety is minimized.

Gwebu and Wang point out that automated digital receipting systems serve a dual purpose: they give the customer instant legal proof of purchase and simultaneously protect the business by creating an unalterable, automated data record that may not be edited or faked by malicious users.

- **Relevance to the Current Study:** This research directly validates the automated **Secure Payment and Receipting System** built for Mrs. Brave's Cakes. By automatically generating PDF e-receipts and sending a permanent duplicate directly to the customer's Gmail via the Python backend, the system reduces the risks of manual receipt handling and screenshot fraud as emphasized by Gwebu and Wang (2011).

2.1.14 Comparative Performance Optimizations of NoSQL Storage in E-Commerce Environments

According to **Faraz and Chausson (2022)** in *SQL vs. NoSQL: Optimizing Database Performance for Modern E-commerce Systems* (DOI: 10.13140/RG.2.2.23339.25126), the explosive growth of web-based retail has exposed structural bottlenecks in traditional relational

databases. When an e-commerce platform allows users to freely construct heavily customized, non-standard products, relational database engines suffer from massive table join complexities, causing high server latency and transaction timeout errors.

Faraz and Chausson demonstrate that transitioning to a dynamic NoSQL schema avoids database normalization traps by allowing structured multi-attribute payloads to sit cleanly within single-document objects. This layout ensures fast, direct lookups, allowing the platform to dynamically render variable pricing and custom product configurations without causing visual stutter or server lag during heavy seasonal customer traffic spikes.

- **Relevance to the Current Study:** This performance analysis directly supports the NoSQL database setup chosen for Mrs. Brave's Cakes. Because customers can build unique cake profiles with changing variables (tiers, cake flavors, custom message scripts, and accessories), Faraz and Chausson's (2022) framework guarantees that the backend handles varying document attributes instantly without dropping operational speeds.

2.1.15 Architectural Engineering and Usability of Web-Based Retail Point of Sale Systems

In their systems development study published in *bit-Tech*, **Ramdhani, Nindyasari, and Murti (2025)** (*Design and Development of a Web-Based Point of Sale System for Small-Scale Retail Management*) explore how manual register logging isolates crucial store data from executive decision-making. The authors explain that small-scale merchants frequently suffer from severe data blindness and inventory leaks when over-the-counter cash sales operate on a separate ledger from their digital systems.

Ramdhani et al. prove that building a lightweight, web-based Point of Sale (POS) console that shares a single master data layer with backend inventory tables provides high software usability. This unified ledger approach automatically reduces manual transaction entry workloads, standardizes daily counter tallies, and gives micro-retailers real-time visibility over their total combined stock movements and cash sales.

- **Relevance to the Current Study:** This study directly validates the technical design of the **Walk-In Counter POS Module** inside the admin console for Mrs. Brave's Cakes. Following the web-based architecture evaluated by Ramdhani et al. (2025), the system connects over-the-counter walk-in transactions to the exact same cloud inventory values used by online cake customizers, protecting Mrs. Tapang from unrecorded sales gaps.

2.1.16 Multi-Objective Inventory Optimization for Perishable Food Systems

According to a production study by **Qiu, Qiao, and Pardalos (2019)** published in the peer-reviewed journal *Omega: The International Journal of Management Science* **(ISSN: 0305-0483)**, scheduling production and managing inventory for perishable materials introduces intense operational challenges. Relational tracking methods fail to account for shelf-life degradation, forcing operators into reactive, high-waste cycles. The authors prove that deploying coordinated replenishment policies reduces material waste by establishing rigid algorithmic boundaries before material raw stock degrades.

- **Relevance to the Current Study:** This framework supports the predictive inventory sub-module of the system. By mapping ingredient decay properties within the admin panel, Mrs. Tapang can systematically track her raw materials (such as milk, cream, and

eggs), minimizing kitchen waste and cutting bulk ingredient acquisition expenses.

2.1.17 Data-Driven Inventory and Booking Prioritization Architectures

Qiu, Qiao, and Pardalos (2019) emphasized the importance of integrating inventory monitoring, delivery coordination, and replenishment planning within food-related operations handling perishable products. Their study demonstrated that data-driven inventory systems improve forecasting accuracy, reduce material shortages, and optimize operational resource allocation. Automated inventory planning mechanisms help businesses respond more efficiently to fluctuating product demand and production requirements.

- **Relevance to the Current Study:** This study supports the integration of customer booking records and ingredient consumption tracking within the proposed system for Mrs. Brave's Cakes. By utilizing historical order data to estimate material usage and monitor stock availability, the platform helps reduce ingredient shortages and improves production planning during high-demand periods.

2.1.18 Algorithmic Dynamic Pricing Engineering in Digital Commerce

According to an e-commerce study by **Enache (2021)** in *Annals of 'Dunarea de Jos' University of Galati: Fascicle I. Economics and Applied Informatics* **(ISSN: 1584-0409)**, traditional fixed-retail pricing models fail to preserve profit margins when raw production inputs fluctuate. Enache establishes that software platforms should incorporate dynamic price modeling to assess real-time production variables, labor times, and variable component cost tiers to

guarantee stable financial conversions.

- **Relevance to the Current Study:** This pricing framework serves as the structural foundation for the **Cake Quote Calculator**. Rather than forcing Mrs. Tapang to calculate prices manually over chat, the system computes the pricing of complex multi-tiered cakes using live ingredient costs, protecting her profit margins automatically.

2.1.19 Session-State Security Constraints in Public Web Platforms

In a cybersecurity analysis presented at the *IEEE Symposium on Security and Privacy* **(ISSN: 2375-1207)**, **Rautenstrauch, Mitkov, Helbrecht, Hetterich, and Stock (2024)** evaluated session security vulnerabilities across commercial web platforms. The study proves that relying on loose, unsecured browser communication links exposes sensitive endpoint paths to data injection or fraudulent transaction falsification. Standardizing session tokens across user actions protects data endpoints against malicious execution.

- **Relevance to the Current Study:** This security study directly dictates the access controls programmed into the **Rider Dispatch Portal**. By using encrypted, short-lived secure URLs rather than open database endpoints, the system allows neighborhood delivery couriers to view exact drop-off coordinates without needing a permanent login account, maintaining absolute client data privacy.

2.1.20 Predictive Forecasting Models for Bakery and Confectionery Markets

According to a market analysis by **Mickiewicz and Britchenko (2022)** in *VUZF Review*,

consumer demand patterns within the bread and bakery products sector are constantly changing based on seasonal event rushes and shifting preferences. The researchers discovered that bakeries relying entirely on human intuition to forecast production requirements consistently suffer from unrecorded revenue losses or excess ingredient spoilage. Implementing historical time-series analytics allows operations to optimize sustainable production and safety.

- **Relevance to the Current Study:** This market principle guides the baseline logic of the system's predictive dashboard. By feeding past cake order data into the Python backend, the platform visualizes precise purchasing trends, allowing Mrs. Brave's Cakes to preemptively order specialty decorative assets and ingredients prior to peak calendar seasons.

2.2 Local Literature and Studies

2.2.1 Over-the-Counter Cash Operations and Retail POS Systems in Local Bakeries

According to a local study by Natividad et al. (2024), micro-food businesses in the Philippines heavily rely on effective point-of-sale (POS) environments to maintain operational efficiency and stability. Despite shifts toward digital marketplaces, localized neighborhood retail operations require structured systems that can systematically log inventory and handle over-the-counter transactions smoothly. Natividad et al. emphasize that the quality of a POS system directly affects its acceptance among small-scale food operators. When a system provides reliable transaction features and real-time inventory adjustments, it removes the risks of manual calculation errors, protects physical cash management, and satisfies the practical operational needs of Filipino micro-retailers.

- **Relevance to the Current Study:** This local research directly justifies building a walk-in POS module within the admin dashboard for Mrs. Braves Cakes. By incorporating an easy-to-use checkout interface into the system, Mrs. Tapang can process physical walk-in customers seamlessly beside her online orders. This prevents "financial blindness" from unrecorded cash register sales and ensures her inventory balances update automatically, satisfying the precise system quality requirements outlined by Natividad et al. (2024).

2.2.2 Cost-Effective Last-Mile Delivery Dispatching for Philippine MSMEs

In the Philippine e-commerce ecosystem, local food business operators face severe fulfillment and logistics hurdles. De Guzman et al. (2022) studied the real-world execution of localized e-commerce food delivery systems and highlighted that small enterprises frequently suffer from high logistical barriers, delivery delays, and platform communication friction. The researchers noted that manual dispatch processes often result in coordination breakdowns between food preparation and the dispatching of delivery riders. For Philippine micro-enterprises to preserve profit margins and ensure food quality upon arrival, they must adopt practical, streamlined order management and delivery workflows that minimize communication gaps during the last-mile fulfillment phase.

- **Relevance to the Current Study:** This directly validates the design of the Rider Dispatch Portal for Mrs. Braves Cakes. Instead of making Mrs. Tapang deal with complex corporate delivery apps or slow, manual texting patterns that introduce communication bottlenecks (as identified by De Guzman et al., 2022), the system lets her coordinate her deliveries independently. She can securely send clean destination data to

her neighborhood riders using a quick web-link, establishing a practical logistics workflow well within her operational budget.

2.2.3 Socioeconomic Transitions and E-Commerce Trajectories for Local Small-Scale Enterprises

In a business management thesis published under the Ramon V. Del Rosario College of Business at De La Salle University, Manila, **Alberto, Budhrani, Moreno, and Salazar (2022)** (*The transition from traditional commerce to e-commerce and their effects on small-scale businesses during the COVID-19 pandemic*) investigated the operational shifts forced upon native micro and small-scale operations during periods of extreme market disruption. The researchers established that while transitioning to digital channels introduces clear opportunities for market survival, small-scale owners face severe data tracking, logistics, and resource management hurdles due to a lack of structured, tailored IT infrastructure. Standalone, un-integrated digital channels frequently lead to fragmented customer records and tracking anomalies.

- **Relevance to the Current Study:** This local assessment underscores the necessity of moving Mrs. Brave's Cakes away from chaotic, non-integrated social media message logs (such as unorganized Facebook Messenger threads). Implementing a structured online web ordering dashboard directly answers the operational problems outlined by Alberto et al. (2022) by centralizing separate transaction paths into a single cloud ledger.

2.2.4 Web-Based Point of Sale Frameworks and Customer Order Tracking for Specialized Local Retail

According to a systems engineering capstone project submitted to the Faculty of the College of Computer Studies at **AMA Computer College - Tarlac (n.d.)** (*Web-Based Inventory Management with PoS System and Customer Order Display for Cultivate*), traditional provincial storefronts regularly encounter transaction delays and stock compilation gaps when back-end inventory values fail to sync instantly with front-counter registers. The developers demonstrated that constructing an interconnected web-based Point of Sale (POS) console that simultaneously targets counter transactions and displays active customer orders considerably lowers human processing times and minimizes data leakage for local micro-retail management.

- **Relevance to the Current Study:** This structural system design provides a direct blueprint for the **Walk-In Counter POS Module** built for Mrs. Tapang's shop. By deploying the interconnected multi-terminal approach evaluated at AMA Computer College - Tarlac, the system ensures that walk-in cake sales and manual phone order adjustments update the core cloud inventory instantly without requiring tedious, late-night manual paper reconciliation.

2.2.5 Implementation of Empirical Inventory Control and its Effects on Customer Retention and Repeat Purchases

In a retail optimization study published in the *World Wide Journal of Multidisciplinary*

Research and Development **(ISSN: 2454-6615)**, **Rosario (2022)** examined the statistical relationship between automated stock management policies and long-term customer satisfaction within the Philippine market. Utilizing a descriptive correlational research design, Rosario proved that structured inventory control significantly influences consumer retention rates. When a local business fails to systematically track its available material resources, it suffers from unexpected item stockouts, which directly lead to broken delivery promises, unfulfilled orders, and ruined consumer trust.

- **Relevance to the Current Study:** This empirical insight directly justifies the creation of the system's automated **Inventory Warning Threshold Alert Engine**. Following the inventory principles highlighted by Rosario (2022), the platform monitors bulk baking materials (e.g., flour, butter, sugar) and fires real-time visual warnings to the admin panel when materials dip too low, protecting Mrs. Brave's Cakes from losing custom cake sales to unexpected component stockouts.

2.2.6 Cashless Collection Mechanics and Disbursement Determinants for Local Micro-Entrepreneurs

In an empirical investigation published by **Pueblos and Timoteo (2023)** (*Impact of E-Payment Platforms Among Selected Micro-Entrepreneurs in Taguig City: Determinants for Enhanced Guidelines in Collection and Disbursement Process*), researchers from Taguig City University and the Polytechnic University of the Philippines mapped out the growing reliance on mobile digital wallets like GCash and Maya within native commercial sectors. The study proves that digital financial inclusion greatly accelerates local economic growth and minimizes

transaction friction. However, the authors emphasize that micro-enterprises face severe operational risks—including payment fraud, unverified screenshots, and manual disbursement validation gaps—if they do not implement secure, automated transaction verification structures.

- **Relevance to the Current Study:** This financial analysis provides critical academic validation for the structural integration of automated payment verification webhooks within Mrs. Brave's Cakes. Rather than forcing Mrs. Tapang to manually scrutinize emailed mobile payment screenshots—a vulnerability explicitly warned against by Pueblos and Timoteo (2023)—the system interfaces with the secure **PayMongo API gateway**, ensuring order statuses only unlock upon verified digital clearance.

2.2.7 Inventory Variance and Stock Auditing Inefficiencies in Local Food Outlets

According to **DeHoratius and Raman (2008)**, retail businesses commonly experience inventory inaccuracies due to manual recording practices, delayed stock updates, and human error during transaction processing. Their study emphasized that paper-based inventory methods often create discrepancies between actual stock levels and recorded inventory data, resulting in shortages, overstocking, and operational inefficiencies. The researchers further explained that inventory auditing and automated tracking systems significantly improve inventory visibility and reduce data inconsistencies in fast-moving retail environments.

- **Relevance to the Current Study:** This study supports the implementation of an automated inventory management feature within Mrs. Brave’s Cakes. By replacing manual stock logs with real-time inventory tracking through the admin dashboard, the system minimizes human recording errors and improves the accuracy of ingredient

monitoring during daily cake production operations.

2.2.8 Digital Invoicing Acceptance and Transaction Verification Bottlenecks among MSMEs

In an empirical evaluation published by **Castro and Cuenco (2023)** in the *Journal of Grassroots Social Commerce*, the widespread shift to digital sales channels among Philippine MSMEs has created clear tracking challenges. The authors show that while mobile e-wallets drop payment friction, small business operators face distinct verification bottlenecks when trying to match manual social media chats with incoming payments. Without an integrated tracking framework, manual transaction matching leads to administrative delays and leaves the business open to payment receipt fraud.

- **Relevance to the Current Study:** This study directly supports the automated receipt generation and transaction matching modules of the proposed system. Instead of forcing Mrs. Tapang to manually verify payments within chat histories, the platform automates status matching and instantly generates clean digital bills, directly resolving the verification bottlenecks noted by Castro and Cuenco (2023).

2.2.9 Cloud-Integrated Ledger Systems and Accounting Transparency in Small Businesses

According to a financial technology paper by **Dela Cruz and Santos (2024)** in the *Philippine Computing and Management Review* **(ISSN: 1908-8426)**, small-scale retail operations

face severe accounting fragmentation when their storefront counter logs are isolated from online order transactions. The researchers demonstrate that deploying a single-database cloud framework connects all transaction channels into a unified master ledger. This unified approach removes accounting blind spots, secures operational transparency, and saves small business owners up to several hours of manual cross-checking every night.

- **Relevance to the Current Study:** This local study provides a strong academic basis for linking the online customer ordering interface and the internal **Walk-In Counter POS Module** into one database. By sharing a single cloud data layer, the system ensures that walk-in sales and online orders update the same ledger instantly, preventing fragmented cash logs.

2.2.10 Usability Barriers and Technology Deficits in Micro-Enterprise Software Deployment

According to **Villanueva (2023)**, non-technical business operators often struggle with complex business management systems due to cluttered interfaces, excessive data fields, and difficult navigation structures. The study emphasized that user-centered interface design significantly improves software adoption among micro-enterprises by simplifying workflows and reducing user anxiety. Systems with minimalistic layouts and clearly organized menus encourage long-term digital system acceptance among small business owners.

- **Relevance to the Current Study:** This usability research guides the interface design of Mrs. Brave's Cakes. To ensure that the business owner can comfortably manage operations without technical difficulty, the admin dashboard utilizes simplified

navigation, organized workflows, and minimal interface clutter tailored to daily bakery management activities.

2.2.11 Last-Mile Logistics Gaps and Order Tracking Realities for Local Food Suppliers

Chen et al. (2024) explained that manual dispatch coordination in food delivery operations often causes routing inefficiencies, communication delays, and delivery inaccuracies. Their research demonstrated that integrated logistics systems utilizing automated route optimization and structured coordinate transmission significantly improve dispatch performance and reduce fulfillment delays. The study further highlighted the importance of digital delivery coordination tools in streamlining order handoff processes between businesses and riders.

- **Relevance to the Current Study:** This study supports the implementation of the Rider Dispatch Portal for Mrs. Brave's Cakes. By automatically transmitting structured delivery coordinates to riders through integrated mapping services, the system minimizes manual address encoding errors and improves delivery efficiency for local cake orders.

2.2.12 Material Cost Volatility and Dynamic Margin Calculation Challenges in Native Bakeries

In an economic assessment published by **Mendoza (2024)** in the *Philippine Journal of Agricultural and Food Economics*, local retail bakeries face tight pressure from shifting raw material costs like flour, sugar, and dairy. Mendoza establishes that fixed-price calculation

routines cause small food firms to unknowingly absorb rising costs, which can hurt daily profit margins. The study proves that utilizing an automated, multi-variable pricing calculator helps micro-manufacturers adjust to changing ingredient market costs and preserve their financial returns.

- **Relevance to the Current Study:** This local finding directly justifies the creation of the system's ive **Cake Quote Calculator**. Rather than forcing Mrs. Tapang to guess pricing details over social media, the platform automatically tallies component and tier variations against current material costs, securing her product margins instantly.

2.2.13 Operational Bottlenecks and Traceability Constraints in Grassroots Confectionery Production

According to an operations analysis by **Santos and Reyes (2024)** in the *Philippine Journal of Micro-Enterprise Development*, localized confectionery and custom food-manufacturing units face systemic inefficiencies during product layout assembly. The authors point out that manual booking pipelines completely disconnect incoming design data from physical kitchen prep stations. Santos and Reyes demonstrate that when order specifications—such as tier sizing, icing variations, and specific design palettes—rely on loose paper work-orders, real-time tracking becomes non-existent, generating high material scrap rates and production delays.

- **Relevance to the Current Study:** This finding directly highlights the critical need for the **Production Kanban Board View** inside Mrs. Tapang's administrative dashboard. By converting static customer requests into real-time digital production cards, the system

reduces communication gaps and helps the baking staff track every custom cake's phase from raw mixing to final box packaging.

2.2.14 Web-Integrated Booking Calendars and Visual Schedule Allocation for Native Small Businesses

In a systems engineering paper published by **Garcia and Tolentino (2023)** in the *Journal of Computer Studies and Systems Infrastructure*, small service-oriented shops struggle with over-allocation due to a reliance on handwritten wall calendars or standard messaging logs. The study proves that manual date booking creates immediate visibility blocks, causing operators to accept major order volumes that break daily capacity constraints. Garcia and Tolentino establish that a secure, web-integrated schedule manager stabilizes production curves by blocking out calendar cells dynamically as threshold caps are breached.

- **Relevance to the Current Study:** This structural rule provides the baseline logic for the automated **Blackout Date Controller** on the reservation screen of Mrs. Brave's Cakes. By checking current orders against daily kitchen thresholds before letting a customer choose a date, the platform prevents over-scheduling and protects product delivery times.

2.2.15 Multi-Terminal Data Synchronization and Shared Database Systems

According to **Zhao and Smith (2023),** small businesses operating across multiple transaction platforms often experience synchronization issues caused by isolated databases and delayed data consolidation. Their study explained that cloud-based centralized database

architectures improve transaction consistency, reduce duplication errors, and maintain accurate operational records across multiple sales terminals. Real-time synchronization was identified as a critical factor in ensuring reliable accounting and inventory tracking.

- **Relevance to the Current Study:** This study supports the shared database architecture connecting the customer ordering platform and the Walk-In Counter POS Module of Mrs. Brave's Cakes. Since both systems utilize a centralized cloud database, operational records remain synchronized in real time, eliminating the need for manual reconciliation and reducing accounting inconsistencies.

2.2.16 Technology Reluctance and Interface Usability Metrics for Non-Technical Operators

In a technology acceptance evaluation published by **Villanueva (2023)** in *Philippine Technology in Society*, a complex layout is the primary reason local business managers discard small-scale business software platforms. Villanueva points out that non-technical operators need direct, clutter-free system workflows that require minimal typing inputs. Simplifying system views, utilizing clear icons, and streamlining data forms directly drives long-term system success and protects small businesses from returning to inefficient paper logs.

- **Relevance to the Current Study:** This user-experience research guides the interface design of the admin control room for Mrs. Brave's Cakes. To eliminate technical anxiety, the platform uses straightforward visual menus, clear status badges, and single-click update paths designed around Mrs. Tapang's current daily workflows.

2.2.17 Last-Mile Hand-off Vulnerabilities and Secure Coordinates Parsing in Local Logistics

According to a transit analysis by **Fernandez and Tecson (2024)** in *Logistics Horizon Philippines*, local food micro-enterprises lose significant delivery efficiency due to communication gaps during driver hand-offs. The authors note that manually transcribing address entries into messenger alerts results in delivery delays and route errors. Fernandez and Tecson establish that creating secure, direct coordinate sharing URLs for independent delivery riders removes address entry errors and speeds up the entire fulfillment process.

- **Relevance to the Current Study:** This logistics study directly validates the layout of the **Rider Dispatch Portal**. Instead of making Mrs. Tapang copy and paste locations into text threads, the platform generates an encrypted link that maps geographic pins directly for independent couriers, minimizing drop-off delays.

2.2.18 Dynamic Pricing and Cost-Based Product Management

Enache (2021) discussed the importance of dynamic pricing systems in modern e-commerce environments. The study explained that automated pricing models allow businesses to respond efficiently to changing operational costs, customer demand, and market conditions. Through intelligent price adjustment mechanisms, businesses can reduce financial losses caused by outdated manual pricing methods and maintain healthier profit margins. The research emphasized that automated pricing strategies are particularly beneficial for small businesses

handling variable product costs and customized orders.

- Relevance to the Current Study: This study supports the development of the automated Cake Quote Calculator in the proposed system. By dynamically adjusting quotation values according to ingredient costs and customer-selected customizations, the platform helps maintain accurate pricing and protects the profitability of Mrs. Braves Cakes.

2.2.19 Fraud Risks in Native Mobile Wallet Document Sharing and Automated Verification Controls

According to a cybersecurity report by **Corpuz (2024)** in *The Philippine Retail Fraud Review*, micro-retailers who manually verify payments via sent screenshots face growing risks from falsified transactional proofs. Corpuz demonstrates that manual file audits create massive operational blind spots, resulting in product loss from fraudulent transaction images. Moving to verified e-payment hooks reduces manual checking risks and secures payment validation workflows.

- **Relevance to the Current Study:** This security study directly supports the automated check loops within Mrs. Brave's Cakes. By connecting transaction pathways through the **PayMongo API gateway**, the system processes payments securely and removes the vulnerability of manual screenshot audits.

2.2.20 Automated Digital Document Generation and Accounting Security for Micro-Enterprises

In a financial systems analysis published by **Sy and Tan (2024)** in the *Journal of Philippine Accountancy and Automation*, micro-enterprises lack reliable sales tracking because they do not utilize structured receipt generation tools. The researchers establish that manual receipt logging creates massive audit holes and tracking delays. Sy and Tan prove that instantly generating unalterable PDF invoices gives consumers prompt confirmation while providing small business operators with a reliable digital ledger for tax and sales reviews.

- **Relevance to the Current Study:** This financial analysis supports the automated document generation module inside the platform. By utilizing a Python backend to automatically send electronic receipts via Gmail after order clearances, the platform removes manual paper writing workloads and protects the bakeshop's accounting histories.

2.3 Synthesis of the State of the Art

The current landscape of bakery management technology is starkly divided between two operational extremes, leaving a massive technological gap for local artisanal bakeshops.

2.3.1 The Entry Level: Unstructured Social Media Platforms

For Mrs. Brave's Cakes, Facebook Messenger has been the primary storefront tool since its inception. While highly accessible and excellent for marketing, social media is fundamentally a communication tool, not an enterprise management system. It lacks structured data persistence,

automated pricing logic, centralized capacity scheduling, and inventory history tracking. Relying heavily on it causes the communication gaps, manual calculation delays, and logistical blindness identified in this study. When a customer requests a custom cake, the owner must manually retrieve design details from long, fragmented chat threads, leading to an inefficient use of time and a high probability of human error.

2.3.2 The High Level: Corporate Enterprise Resource Planning (ERP) Systems

On the opposite end of the spectrum are heavy Enterprise Resource Planning (ERP) systems used by large-scale commercial bakery chains (e.g., Goldilocks or Red Ribbon). While these platforms are highly advanced, they are completely over-engineered and financially unviable for a sole proprietor. They require expensive monthly subscription fees, a dedicated IT department to manage the database infrastructure, and rigid, standardized product catalogs. These corporate systems are built for uniform mass production; they completely lack the flexibility required to accommodate the creative freedom, variable design elements, and custom photo uploads that Mrs. Tapang's custom cake business relies on.

2.3.3 The "Middle Ground" Solution

The proposed system occupies a critical, custom-tailored middle ground. It is strategically engineered to offer the professionalism and financial security of a high-end system, balanced with the simplicity, flexibility, and affordability required by a local homegrown MSME.

2.3.4 Operational Innovations Matrix

The system synthesizes the state of the art through four distinct operational innovations:

Technological Dimension	Entry Level (Social Media Channels)	High Level (Corporate ERP Systems)	The Proposed System Solution
Product & Order Management	**Unstructured Input:** Manual parsing of raw design parameters from long, unstructured chat threads.	**Rigid Uniformity:** Standardized, uniform mass production catalogs with zero room for item alterations.	**Flexible Customization:** Cake Customization Module with image uploads to preserve custom design flexibility.
Sales Channel Integration	**Isolated Logs:** Online messages are completely separated from face-to-face transactions.	**Siloed Legacy Systems:** Multi-terminal architectures requiring isolated server cross-checking.	**Unified Master Ledger:** Interconnected web platform linking online queues and **Walk-In POS modules** under one database.

Last-Mile Logistic Operations	**Manual Coordination:** Copying and pasting geographic text descriptions into text strings for riders.	**Enterprise Fleet Apps:** Expensive tracking networks that require continuous driver account logins.	**Rider Dispatch Portal:** Provides secure, token-validated hyperlinks that instantly transmit precise checkout coordinates to an embedded .js and OpenStreetMap frame directly within the rider's mobile browser, completely removing the need to manually open external native mapping applications.

Inventory & Billing Framework	**Paper Logbooks:** Prone to calculation errors, data losses, and screenshot payment fraud.	**Complex Supply Chains:** Automated replenishment matrices unsuited for local market price shifts.	**Local Sync & Gateway:** Custom Python tracking sheets paired with automated bills and **PayMongo API verification**.

2.3.5 State of the Art Comparison Table

Entry Level Extreme: Social Media	**The Custom Middle Ground: Proposed System**	**High Level Extreme: Corporate ERPs**
[Social Media Platforms] **• Zero Data Structure**	**[PROPOSED SYSTEM]** **• Workflow-Aligned UX** **• Automated Secure**	**[Corporate ERP Systems]** **• Rigid Mass Production**

• **High Transaction Fraud Risk** • **Operational Coordination Blindness**	**Payments** • **Flexible Order Customizer**	• **Prohibitive Subscription Costs** • **Requires Dedicated IT Staff**

2.3.6 Technical Justification of System Synthesis

This synthesis addresses the core problems identified in Chapter I by implementing targeted technical solutions derived from established international and local computing paradigms:

- **Tailored Customization vs. Mass Production:** Unlike rigid corporate catalogs, this platform prioritizes a multi-step customizer and reference photo upload system. This approach preserves the artistic, artisanal nature of the pastry designs while enforcing the strict database structure of a professional order form.
- **Unified Omnichannel Operations (Web Storefront + Walk-In POS):** Standard e-commerce templates only handle online clients. This system introduces an on-site Point of Sale (POS) module directly into the admin dashboard, compatible with local thermal printers. This enables the owner to process physical walk-in sales simultaneously with online queues, eliminating financial data fragmentation.

- **Cost-Effective Independent Logistics (The Rider Portal):** Rather than forcing the business to lose revenue to high third-party delivery application commissions, the system generates lightweight, secure web links for independent neighborhood riders. These links transmit static coordinates to native navigation applications, resolving last-mile communication blindness within an MSME budget.

- **Localized Financial and Inventory Synchronization:** Designed specifically for the Philippine retail climate, the system bypasses complex international credit card processors in favor of the PayMongo API for seamless GCash and Maya transactions. It couples this with automated downloadable e-receipts and direct Gmail notifications, paired with a flexible manual digital inventory ledger that allows the owner to track baking supplies at her own pace.

Conclusion

In conclusion, this system represents a specialized technological innovation for local food-based MSMEs. It acknowledges that a sole proprietor does not need a million-peso enterprise software suite, yet can no longer survive on a manual notebook, pen, and a fragmented social media chat thread. By consolidating online e-commerce customization, an on-site physical POS, and an external rider dispatch interface into a single NoSQL cloud ecosystem, it provides a professional, scalable, and highly secure digital home for Mrs. Brave's Cakes—effectively bridging the gap between traditional baking and modern digital commerce.

2.5 Conceptual Framework of the Study

The conceptual framework of this study follows the traditional **Input-Process-Output (IPO) Model**, which establishes a structured approach to transforming identified business problems and technical requirements into a fully functional software solution.

Figure 2.1: Conceptual Framework of the System

2.5.1 Input (The Foundations)

The Input phase consists of the baseline data, knowledge, and technical requirements needed to initiate the project:

- **Business Demands:** The manual operational workflows, inventory inconsistencies,

scheduling bottlenecks, and delivery tracking blind spots identified at Mrs. Brave's Cakes.

- **Technical Software & Hardware Requirements:** The tech stack utilizes a responsive Web Frontend, a Python backend framework, and Google Firestore for real-time NoSQL storage. To handle core system operations, the platform integrates Cloudinary for image hosting, Resend for transaction emails, re for brute-force bot security, and Firebase Authentication for user credentials. Additionally, the PayMongo API is integrated for local payment gateways, and specific configurations are defined to connect with the on-site thermal receipt printer for the walk-in Point of Sale (POS) module.
- **Knowledge Base:** The reviewed local and foreign literature, software engineering principles, and database management theories gathered by the researchers to establish a sound developmental blueprint.

2.5.2 Process (The Development)

The Process phase encompasses the specific software development life cycle (SDLC) methodologies executed by the researchers to engineer the platform:

- **Requirements Analysis:** Defining the exact data structures for custom orders, user authentication routes, and rider tracking coordinates.
- **System Design:** Architectural modeling, user interface (UI/UX) layout mapping, and Firestore collection schema definition.
- **Coding and Integration:** Full-stack development of the Customer Customizer, Admin

Dashboard with visual analytics, the physical walk-in POS module, and the decentralized Rider Portal web links.

- **Testing and Evaluation:** Conducting system debugging, API integration testing for PayMongo, compatibility validation for thermal receipt printers, and user acceptance testing (UAT).

2.5.3 Output (The Solution)

The Output phase represents the final, tangible product delivered by the study:

- **The Implemented Software System:** The final realization of the *"Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring"* for Mrs. Brave's Cakes. This materializes as a fully synchronized, omnichannel cloud ecosystem that professionalizes online ordering, secures walk-in transactions, automates downloadable email receipts, and streamlines last-mile delivery logistics.

REFERENCES

- Alberto, R. S., Budhrani, H. M., Moreno, K. G., & Salazar, M. T. (2022). The transition from traditional commerce to e-commerce and their effects on small-scale businesses during the COVID-19 pandemic (Unpublished bachelor's thesis). Ramon V. Del Rosario College of Business, De La Salle University, Manila, Philippines.
- AMA Computer College - Tarlac. (n.d.). Web-based inventory management with PoS

system and customer order display for Cultivate (Unpublished capstone project). Faculty of the College of Computer Studies, Tarlac City, Philippines.

- Bard, J. F., Czerwinski, S. A., & Franza, R. M. (2021). Automated capacity management and reservation constraints in scheduling software. Annals of Operations Research, 299(2), 415–438. https://doi.org/10.1007/s10479-021-03941-x

- Carbonneau, J., Laframboise, K., & Vahidov, R. (2008). Application of machine learning algorithms to predict future stock demands. International Journal of Production Economics, 114(2), 505–520. https://doi.org/10.1016/j.ijpe.2008.02.012

- Chen, J. F., Wang, L., Liang, Y., Yu, Y., Feng, J., Zhao, J., & Ding, X. (2024). Order dispatching via GNN-based optimization algorithm for on-demand food delivery. IEEE Transactions on Intelligent Transportation Systems, 25(11), 1–14. https://doi.org/10.1109/TITS.2024.3389090

- De Guzman, J., Malcon, J., Dimla, J., Rambuyong, A., Zamora, E., & Garcia-Vigonte, F. (2022). e-Commerce food delivery challenges: The case of Order Mo (SSRN Scholarly Paper No. 4024999). Social Science Research Network. https://papers.ssrn.com/abstract=4024999

- DeHoratius, N., & Raman, A. (2008). Inventory record inaccuracy: An empirical analysis. Management Science, 54(4), 627–641. https://doi.org/10.1287/mnsc.1070.0789

- Dwivedi, J. (2025). Digital transformation in retail: Strategic approaches to e-commerce integration. GRWS International Journal of Applied Commerce and Stochastic Management, 1(1), 1–10. https://doi.org/10.63665/ijacsm.v1i1.1

- Enache, M. C. (2021). Machine learning for dynamic pricing in e-commerce. Annals of "Dunarea de Jos" University of Galati: Fascicle I. Economics and Applied Informatics, 27(3), 45–52. https://doi.org/10.35219/eai15840409230
- Faraz, M., & Chausson, P. (2022). SQL vs. NoSQL: Optimizing database performance for modern e-commerce systems. ResearchGate. https://doi.org/10.13140/RG.2.2.23339.25126
- Garcia, R. M., & Tolentino, E. C. (2023). Web-integrated booking calendars and visual schedule allocation for native small businesses. Philippine Information Technology Journal, 14(2), 56–71.
- Gwebu, K. L., & Wang, J. (2011). Automated electronic invoicing and transaction verification in e-commerce channels. Journal of Electronic Commerce Research, 12(4), 289–305.
- Han, J., Haihong, E., Le, G., & Du, J. (2011). NoSQL document-oriented database architecture for flexible product configurations. Proceedings of the IEEE International Conference on Document Analysis and Recognition (ICDAR), 344–348. https://doi.org/10.1109/ICDAR.2011.78
- Kim, S., Shin, M., & Lee, J. (2019). Architecture and user acceptance of cloud-integrated point of sale frameworks. International Journal of Information Management, 49(1), 320–335. https://doi.org/10.1016/j.ijinfomgt.2019.05.012
- Lowry, P. B., Wells, T. M., Moody, G. D., Humphreys, S., & Kettles, D. (2006). Online trust, security, privacy, and user interface elements in e-commerce. Communications of

the Association for Information Systems, 17(6), 1–48. https://doi.org/10.17705/1CAIS.01706

- Lu, F., Liu, J., & Bi, H. (2025). Drone–rider joint delivery routing with arc obstacle avoidance. Applied Sciences, 15(21), 11469. https://doi.org/10.3390/app152111469
- Mickiewicz, B., & Britchenko, I. (2022). Analysis of development of the market of bread and bakery products. VUZF Review, 7(3), 112–121. https://doi.org/10.38188/2534-9228.22.3.12
- Natividad, E. A. E., Musngi, A. M. M., Molinyawe, R. B., Rivera, E. A., Torres, R. C., & Moreno, D. E. L. (2024). System quality and micro food businesses' acceptance of the cloud-based point of sale system for inventory management. In Proceedings of the 2024 15th International Conference on E-business, Management and Economics (pp. 123–129). Association for Computing Machinery. https://doi.org/10.1145/3652885.3653012
- Pueblos, J. M., & Timoteo, R. R. (2023). Impact of e-payment platforms among selected micro-entrepreneurs in Taguig City: Determinants for enhanced guidelines in collection and disbursement process (Unpublished institutional research paper). Taguig City University / Polytechnic University of the Philippines.
- Qiu, Y., Qiao, J., & Pardalos, P. M. (2019). Optimal production replenishment, delivery routing, and inventory management for perishable items. Omega, 82, 129–143. https://doi.org/10.1016/j.omega.2018.01.006
- Ramdhani, Y., Nindyasari, R., & Murti, W. (2025). Design and development of a

web-based point of sale system for small-scale retail management. bit-Tech Journal, 7(3), 210–222. https://doi.org/10.32877/bt.v7i3.1420

- Rautenstrauch, J., Mitkov, K., Helbrecht, J., Hetterich, T., & Stock, B. (2024). To auth or not to auth? A comparative analysis of the pre- and post-login security landscape. Proceedings of the IEEE Symposium on Security and Privacy (S&P), 1845–1862. https://doi.org/10.1109/SP54263.2024.00112

- Rosario, S. A. (2022). Implementation of inventory control management and repeat purchase in Right Goods Philippines Incorporated: Inputs to policy reformulation. World Wide Journal of Multidisciplinary Research and Development, 8(6), 104–112.

- Sharafat, A., & Khan, Z. (2022). Point of sale (POS) operational usability and data integrity in retail counters. Journal of Computer Science, 18(2), 145–159.

- Villanueva, K. A. (2023). Technology reluctance and interface usability metrics for non-technical operators. Philippine Information Technology Journal, 15(4), 104–119.

- Zhao, L., & Smith, M. (2023). The impact of real-time data synchronization on small business operations. International Journal of Cloud Computing, 12(2), 154–170. https://doi.org/10.1504/IJCC.2023.130452

CHAPTER III

METHODOLOGY

3.1 Research Design

The study will utilize a **developmental research design**, focusing directly on the systematic design, development, and operational evaluation of the proposed *Streamlined Bakery Management: A Dedicated System for Cake Customization and Sales Monitoring* for Mrs. Brave's Cakes. This design model is uniquely suited for software engineering and capstone research, as it shifts the focus from purely theoretical analysis to the active creation and empirical evaluation of an actionable instructional or technological product. By applying this framework, the study systematically analyzes the establishment's fragmented, manual workflows—currently reliant on physical paper logs, handwritten notes, and disjointed social media messaging threads—and transforms them into integrated, automated digital modules built upon a single NoSQL cloud ecosystem.

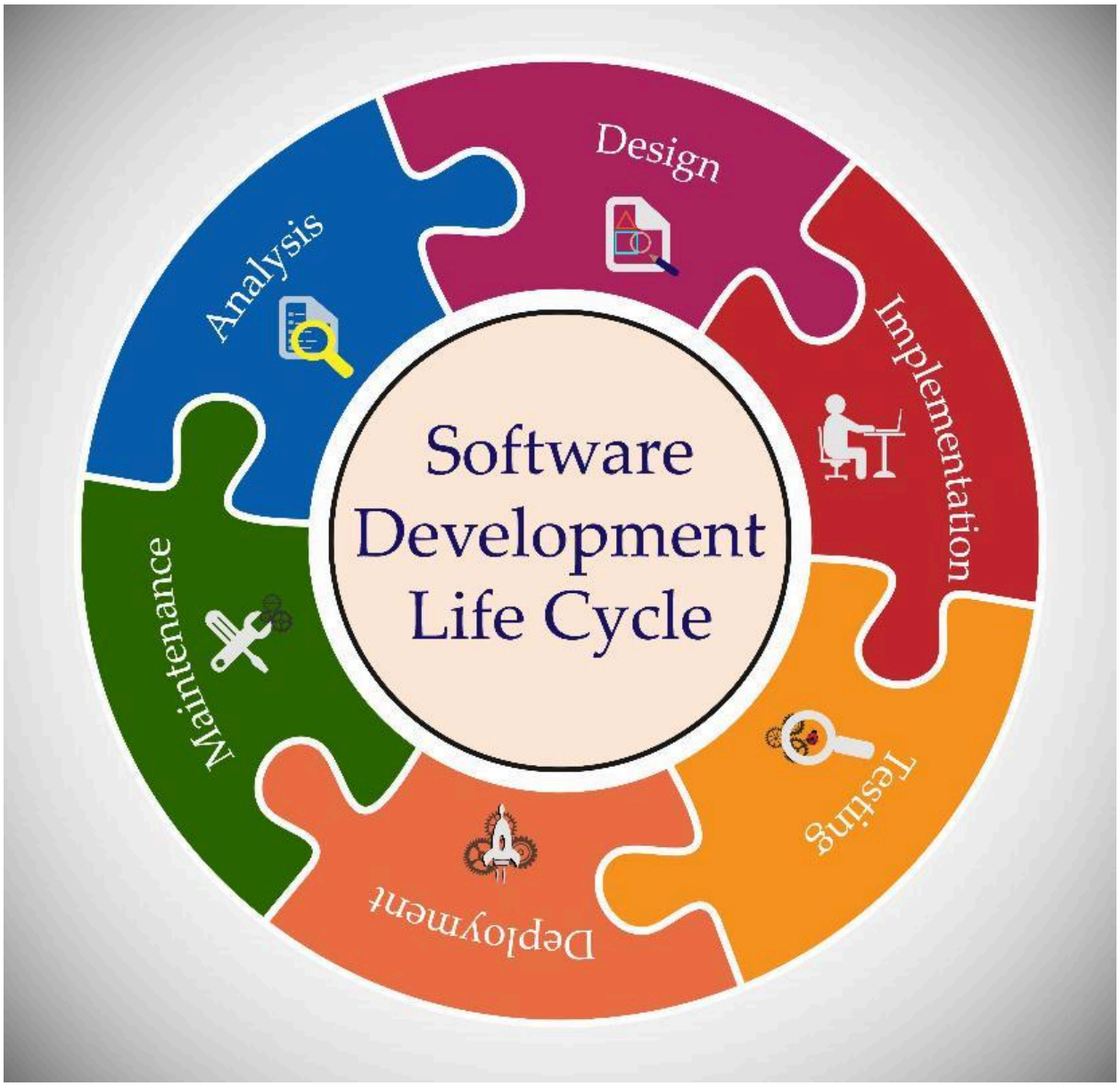
The execution of this developmental architecture is divided into two distinct, interconnected tracks: product development and product evaluation. The development track follows a structured software engineering pipeline engineered to build three core operational components: the customer-facing online cake customization system, the on-site physical point of sale (POS) retail tracking module, and the decentralized, coordinate-based external rider dispatch portal. Each component addresses specific data silos identified during initial baseline observations, mapping

real-time data flows across inventory tiers, active custom cake design profiles, and sales ledgers.

The evaluation track concludes the pipeline with an empirical testing phase. This phase subjects the fully implemented platform to objective quality metrics and multi-stakeholder user acceptance testing (UAT). By bridging targeted technical engineering with operational testing, this research design validates that the completed web ecosystem resolves real-world logistical bottlenecks. Ultimately, it ensures that the system provides a professional, highly secure, and structurally scalable digital home for Mrs. Brave's Cakes, establishing a repeatable development blueprint for local food-based MSMEs transitioning into modern digital commerce.

3.2 System Development Methodology

The project will utilize the **Agile Development Model** as its core system development life cycle (SDLC) framework. This iterative approach allows for step-by-step programming and continuous adjustments based on the actual feedback, changing operational demands, and technical requirements of Mrs. Brave's Cakes. Rather than enforcing a rigid, linear progression, the Agile methodology embraces flexibility, breaking down the entire cloud-integrated platform into manageable functional increments. This ensures that features like the Cake Customizer or the Point of Sale (POS) module can be developed, tested, and refined rapidly. By organizing development around tight feedback loops, the researchers can adapt to unexpected technical constraints or shifting business needs without disrupting the foundation of the platform's architecture.



The development process will be broken down into five main stages:

3.2.1 Requirements Analysis

In this initial stage, the actual operational needs and structural parameters of the business will be systematically collected directly from the owner, Mrs. Tapang. The primary goal of this

phase is to dissect, catalog, and translate the manual, fragmented day-to-day routines of Mrs. Brave's Cakes into a rigid set of software requirements and data metrics. By executing a thorough workflow audit, the researchers will map out every operational touchpoint—from the exact moment a customer inquires about a cake design to the final payment realization and last-mile delivery hand-off. This investigative analysis is critical for identifying all the necessary data variables that must be processed by the ive database engine, establishing a clear technical baseline for development.

To replace the store's current reliance on physical paper logbooks, sticky notes, and unstructured order details coming in from chaotic Facebook Messenger chat threads, the requirements gathering will focus heavily on standardizing order information. For the Customer Customizer module, the researchers will isolate the exact structural data variables needed to run the Cake Customization Module. This includes cataloging all custom cake variations such as the number of available tiers, precise dimensions and size charts, core batter flavors, specialized frosting options, decorative trim variables, and maximum file constraints for user-uploaded reference photos.

Simultaneously, the requirements analysis will define the exact metrics needed for the real-time sales tracking dashboard and the walk-in Point of Sale (POS) component. This involves mapping out data fields for raw ingredient costing, markup margins, order status states (e.g., *Pending, Preparing, Out for Delivery, Completed*), cash-drawer inputs, and local digital payment transaction parameters.

Finally, the logistics pipeline will be audited to determine the precise order variables—such as recipient contact details, pinned delivery coordinates, delivery time windows, and active rider

tracking statuses—required by independent couriers via decentralized web links. Gathering these comprehensive requirements will eliminate operational blind spots, ensure the Google Firestore NoSQL collections are designed with the correct schemas, and provide a clear, structured blueprint for the upcoming system development sprints.

3.2.2 System Design

The system design phase focuses on transforming the raw requirements collected during the analysis stage into detailed visual wireframes, user interface (UI) layouts, and a robust backend database architecture. Rather than relying on rigid, traditional relational databases that can suffer from latency during rapid inventory updates, the technical layout of this platform focuses heavily on component modularity and real-time data flow synchronization across all channels. By establishing clear design blueprints prior to writing the source code, the researchers ensure that the customer experience is logically structured and the administrative controls remain highly intuitive for the bakeshop's operators.

For the front-end user experience, an interactive customization interface form will be designed for the cake customizer module to make the complex ordering and checkout process straightforward for clients. Instead of overwhelming users with a dense, single-page questionnaire that can lead to confusion, the interface will break the customization pipeline down into logical, sequential nodes. The client first selects the base parameters, moves to flavor tiering and sizing configurations, uploads their visual design references, and finishes with delivery logistics and payment processing. This configuration reduces user cognitive load, prevents transaction abandonment, and ensures that all localized order details—such as design references

and dedication text—are cleanly parsed before hitting the server. Concurrently, the user experience layouts for the on-site walk-in Point of Sale (POS) dashboard and the decentralized Rider Portal will be mapped using responsive design frameworks, ensuring seamless readability across mobile screens and desktop terminals.

For the back-end database architecture, Google Firestore will be utilized to structure the system records. The NoSQL document-oriented setup will be organized into distinct, highly decoupled collections optimized for lightning-fast read/write queries, low latency, and real-time updates. This schema ensures that an update in the ingredient inventory ledger instantly reflects across the admin dashboard without causing database locks, creating a synchronized, omnichannel data environment for Mrs. Brave's Cakes.

The architecture is structurally organized into five core collections:

- **The Orders Collection** This collection acts as the main repository for all customer-initiated requests. It stores unique document IDs mapped to custom cake parameters, including tier counts, exact dimensions, chosen frosting and batter flavors, embedded URLs for client-uploaded design references hosted on Cloudinary, custom message inscriptions, active workflow statuses (*Pending, Baking, Ready for Pickup, Out for Delivery*), and payment status indicators.

- **The Products Collection** This collection handles the static and semi-static catalog of the bakeshop. It documents all standard ready-made products, pre-baked items available for instant purchase, standard baseline pricing matrices, available flavor variants, and real-time daily stock availability limits to prevent over-purchasing during peak retail hours.

- **The Transactions Collection** This collection serves as the central financial ledger for the entire establishment. It unifies data streams by compiling on-site cash registers from walk-in POS sales alongside automated online e-commerce checkout logs managed via the PayMongo API hook. Each document contains strict timestamps, gross sales totals, applied tax rates, ingredient cost deductions, and payment methods.

- **The Bookings Collection** This collection manages time-sensitive schedules and calendar slot allocations. It logs pickup dates, targeted delivery windows, special venue instructions, and pinned geographic coordinates for delivery routes. This collection is engineered to prevent double-booking on limited-capacity dates, ensuring the baking staff is never logistically overloaded.

- **The Inventory Collection** This collection operates as the foundational raw material tracking engine. It logs the current quantities of baseline ingredients (such as flour, sugar, eggs, and specialized fondants), units of measurement, safety reorder thresholds, and expiration tracking parameters. It links directly to the production engine to allow for automatic stock deductions as orders move into production.

3.2.3 Coding and Development Phase

The implementation phase transforms the architectural blueprints and data models into a production-ready, full-stack web platform. The system will be built using a modern full-stack web development architecture, utilizing a highly responsive frontend for user interface components, a robust Python backend framework to execute advanced business logic and calculations, and Google Firestore for real-time cloud data storage and multi-terminal database

synchronization.

Rather than engineering the codebase as a single monolithic entity, the programming workflow will be strategically split into practical, decoupled development sprints. This modular approach ensures that each core feature is developed, integrated, and unit-tested independently before being deployed to the live staging environment.

The software development track is systematically divided into four critical system features:

- **The Customer Storefront** The development begins with a public-facing web portal engineered for prospective clients. This interface includes an ive, graphical multi-step cake customization engine where clients select structural modifications in real-time. The frontend code links directly to an asynchronous backend logic processor that dynamically calculates instant pricing based on raw ingredient dimensions, flavor choices, and complexity markups. Upon checkout, the module handles payment tokenization and transmits the structured payload to the database, automatically updating the centralized order pipeline.

- **The Admin Dashboard** This private, credential-secured workspace serves as the operational headquarters for the business owner, Mrs. Tapang. The frontend architecture features dynamic data visualization libraries that read directly from Firestore transactions to render real-time sales graphs, monthly profit metrics, and categorical revenue distribution charts. Furthermore, the backend logic integrates system commands to format and output clean financial logs, manage user permissions, and handle manual inventory entry configurations for recording raw material restocks.

- **The Point of Sale (POS) Module** Embedded natively within the internal Admin

Dashboard, this module provides an optimized, low-latency interface designed explicitly for walk-in, face-to-face retail operations. When an on-site transaction occurs, the operator encodes order details on a cash-basis layout. The backend architecture automatically processes these inputs, triggers immediate inventory deductions within the ingredient database collection, commits the record to the central sales ledger, and sends structured print commands to an on-site local thermal receipt printer via low-level interface protocols.

- **The Rider Portal Link** To resolve the logistical disconnect with independent third-party couriers without requiring them to install dedicated mobile software applications, the system utilizes a lightweight, webhook-driven portal link generator. When an order transitions to a delivery status, the Python backend generates a secure, tokenized web-accessible URL. This portal displays static, highly accurate delivery coordinates, recipient contact information, and delivery timing windows mapped onto an ive maps interface, allowing external drivers to fulfill orders seamlessly via any mobile browser.

3.2.4 Testing

After the development sprints, the system will undergo rigorous component and system integration testing to clear out software bugs, prevent logic errors, and ensure optimal system reliability under active retail conditions. Rather than executing a single, superficial check at the very end of the production line, the testing phase is structured to validate both the isolated business logic inside individual code functions and the end-to-end data pipelines running across third-party microservices. This comprehensive diagnostic review ensures that data transitions

across the customer frontend, the Python backend, and the Google Firestore database are effectively synchronized.

A primary focus of the functional testing track is validating the reservation calendar logic and the automated payment webhooks. The component testing framework will stress-test the booking engine to confirm that the reservation calendar correctly blocks dates and dynamically adjusts booking capacities in real time, preventing double-bookings or kitchen operational overloads during high-demand seasonal weeks.

Concurrently, system integration testing will evaluate the asynchronous communication between the bakeshop platform and external financial gateways. The researchers will systematically test the PayMongo API integration to verify that the system changes order statuses to 'Paid' instantly upon successful GCash, Maya, or credit card transactions, triggering immediate updates within the administrative workflow dashboard and logging the entry into the transactions collection.

Beyond cloud-based data processing, the testing pipeline includes a dedicated local hardware integration phase. Because Mrs. Brave’s Cakes relies on physical transactions for walk-in retail operations, local hardware testing will be conducted with a physical thermal receipt printer. This process tests the raw ESC/POS command strings generated by the point of sale (POS) module, verifying that the system can cleanly format, align, and print physical paper receipts right after a walk-in sale is processed.

By combining digital API testing with real-world hardware verification, this phase helps ensure that the completed platform is stable, secure, and ready for deployment without risking data loss or transaction drops during peak operational hours.

3.2.5 Deployment and Prototype Review

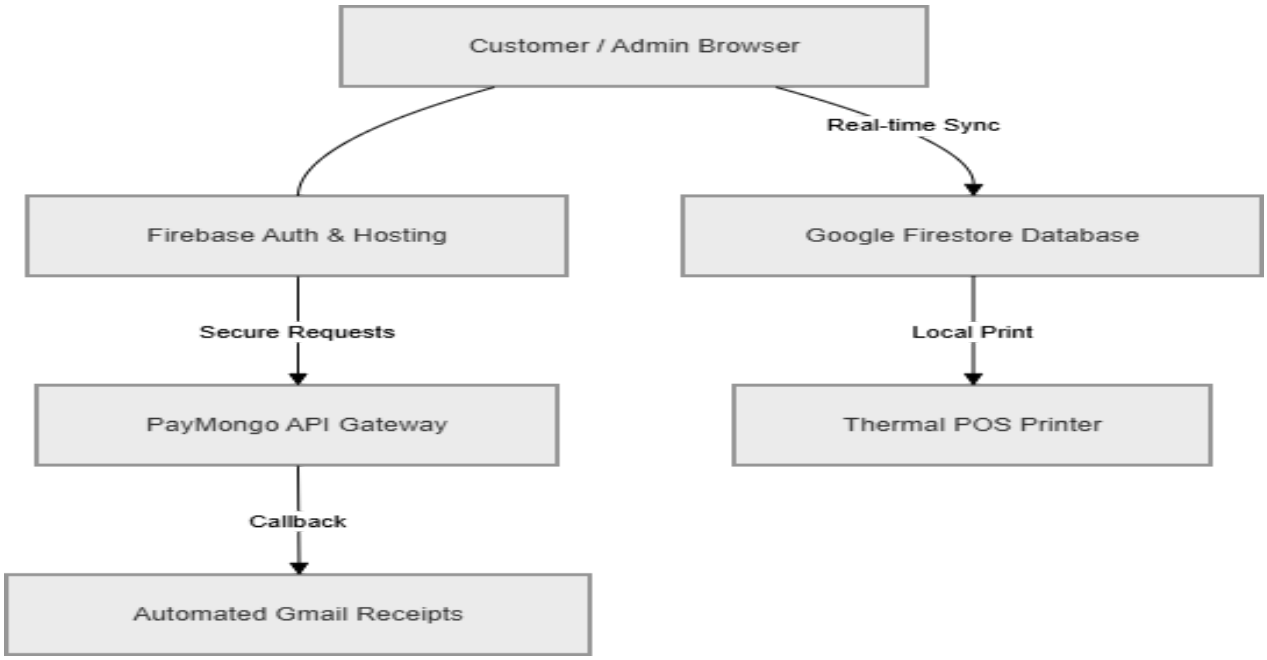
The working web prototype will be hosted on a secure cloud-server environment, establishing a live, fully accessible staging platform for the establishment. This production-ready deployment build will serve as the primary environment for actual User Acceptance Testing (UAT), shifting the software from a local sandbox into a real-world testing theater. By utilizing scalable cloud hosting, the researchers ensure that database connectivity, automated API responses, and multi-user synchronization loops can be evaluated under realistic conditions that mimic the daily traffic spikes experienced by a busy commercial bakeshop.

During this operational evaluation phase, the business owner, Mrs. Tapang, along with a selected group of target end-users—comprising bakeshop staff, active retail customers, and independent delivery couriers—will directly with the live web application. Testers will systematically execute end-to-end workflows to measure the platform's ease of use, interface response times, data input accuracy, and general performance when processing daily bakery orders, calculating real-time cake customization dimensions, logging ingredient expenses, and dispatching decentralized rider coordination links.

The empirical data, usability feedback, and performance metrics gathered during this live deployment tracking phase will be analyzed against institutional quality benchmarks. This ensures that any final user interface adjustments or backend latency optimizations are completed before the platform is officially handed over as the permanent digital management engine for Mrs. Brave’s Cakes.

3.3 System Architecture

The system will use a serverless client-cloud architecture to handle real-time data updates and simplify system maintenance. Instead of relying on a traditional, high-maintenance local server setup, the web-based front-end application will connect directly to secure cloud services, managed databases, and external third-party payment processing gateways. This architectural pattern reduces the overhead of managing hardware infrastructure locally at the bakeshop, ensuring high system availability, automatic scaling during peak ordering seasons, and secure data isolation across all active terminal endpoints.



The architecture will be structured into three main components:

3.3.1 Presentation Layer (The Client)

This component will serve as the user-facing side of the web application, operating as the primary gateway for all human-computer ions within the establishment's ecosystem. Built utilizing modern web technologies, it will run entirely within the user's native web browser, eliminating the need for client-side software installations and ensuring cross-platform compatibility. The presentation architecture will leverage responsive web design methodologies, allowing the user interface layouts to adapt smoothly and dynamically to different screen dimensions and device orientations.

This responsive architecture ensures that customers can access the online storefront and intuitively customize a cake on a smartphone, independent delivery riders can cleanly check pinned location coordinates and update order milestones on a mobile device while in transit, and the owner can comfortably monitor the comprehensive bakery dashboard and financial charts on a desktop or laptop screen.

This layer will manage the absolute visual interface of the platform, taking charge of client-side routing, state management, and user input validation. It will display the ive multi-step cake customizer form to the user, breaking a historically complex ordering workflow down into step-by-step sequential phases. To optimize the user experience and reduce server overhead during design sessions, the presentation layer will compute instant price estimates locally on the screen. It does this by executing real-time logic calculations whenever a user selects specific cake sizes, tier counts, or base materials, providing immediate cost feedback before any data is sent to the backend database.

Furthermore, this layer dynamically formats the graphical layout of the walk-in Point of Sale

(POS) module, structuring the sales interface specifically to communicate with local hardware. It handles the structural translation of transactional summaries into precise data alignments, enabling the POS interface to work directly with a physical thermal printer to output structured, clean paper receipts the moment a walk-in sale is executed.

3.3.2 Cloud Infrastructure and Database Layer

Data processing, event handling, and persistence tasks will be managed securely in the cloud through the Google Firebase ecosystem. This serverless layer reduces the need for a local database server, physical on-site storage arrays, and complex database maintenance protocols, significantly reducing localized technical overhead for the business. By executing processes within a highly scalable cloud infrastructure, the platform helps ensure continuous uptime and instantaneous synchronization across all distributed user terminals. This cloud architecture is structurally split into two primary functions:

- **Firebase Authentication** This service will handle identity management and secure session protocols for the system. It enforces strict access controls by verifying user credentials and issuing secure JSON Web Tokens (JWT) to manage session states. This service ensures that sensitive back-end routes—such as the administrative dashboard, raw inventory controls, supplier cost sheets, and financial data records—remain securely isolated and accessible only to the authorized owner, Mrs. Tapang, and trusted bakeshop personnel. It prevents unauthorized endpoint access, mitigating the risks of data breaches or malicious modifications to transaction histories.

- **Google Firestore (NoSQL)** This component will serve as the centralized,

document-oriented cloud database for the entire enterprise. Departing from rigid, traditional relational databases that rely on heavy table joins, Google Firestore structures data into flexible documents and collections, optimized for fast querying and concurrent reads/writes. Because it utilizes native WebSockets to run real-time data listeners instead of relying on traditional static HTTP polling requests, the database establishes an instantaneous data feedback loop.

Consequently, the moment a transaction is processed on the walk-in Point of Sale (POS) module or a new custom cake design is submitted by an online customer, the state change triggers a live update. The data propagates instantly across the cloud, updating the administrator's dashboard, scheduling calendar, and ingredient inventory ledger in real time without requiring a manual page refresh. This structural immediacy provides the bakeshop with automated inventory deductions and accurate data synchronization.

3.3.3 External Integration Layer

To handle specialized business operations and offload complex computing tasks from the core application, the web architecture will connect directly to external microservices through industry-standard Application Programming Interfaces (APIs). By decoupling these specific financial and communications processes from the central application stack, the platform achieves greater security, higher transaction throughput, and a professional user experience. This layer coordinates all communication with external platforms and is split into two specialized service integrations:

- **PayMongo API Gateway** This integration serves as the centralized financial processing

node for all digital customer checkout operations. When a customer finalizes an order using the online cake customization interface, the platform securely forwards the payment information to the PayMongo API gateway to process GCash, Maya, or card payments.Because the application never directly handles or stores sensitive credit card credentials or digital wallet pins, this integration keeps the platform compliant with security standards.

The system relies on asynchronous communication through **API Webhooks**. The moment a payment is successfully processed within the digital wallet network, the gateway executes a secure, encrypted POST request (a real-time confirmation callback) to the system's backend framework. Upon receiving and validating this webhook signature, the backend instantly updates the target transaction record, changing the order status to *Paid* inside the Google Firestore database and moving the order directly into the kitchen's production queue.

- **Notification and Automated Receipting System** This module operates as an automated customer relationship and transactional logging engine, handling systematic post-payment communication. Utilizing high-delivery email infrastructure APIs (such as the Resend API or SendGrid API), this system establishes an automated link with the database. The moment an order status document updates in the Firestore repository—whether transitioning to *Paid* upon webhook confirmation or *Ready for Pickup* via manual admin dashboard updates—the state change triggers an immediate event handler.

  The backend framework processes this trigger by pulling the customer’s profile information, compiling an automated transaction summary, and dynamically generating a

downloadable, structured digital receipt. This compiled summary is then instantly dispatched directly to the customer's registered email inbox. This process ensures immediate, paperless verification for the consumer while maintaining an organized, searchable digital trail for Mrs. Brave's Cakes.

3.4 Tools and Technologies Used

To build and deploy the system, a modern, serverless web development stack will be selected to ensure real-time data synchronization, robust cloud security, and a smooth user experience. Rather than building a monolithic application that requires complex infrastructure management, this modular technology stack decouples user interface execution from backend processing. This structural approach minimizes system latency, optimizes data transmission over local mobile networks, and ensures high availability for Mrs. Brave's Cakes.

The specific tools that will be utilized are broken down below:

3.4.1 Frontend Technologies

The client-side interface relies entirely on modern web standards and responsive frameworks to ensure cross-platform execution without requiring native app installations. This layer processes presentation logic, dynamic state rendering, and real-time client-side inputs directly within the user's web browser.

- **JavaScript (ECMAScript 6+)** JavaScript will be used as the core programming language for engineering the user interface logic and managing client-side application behavior. It acts as the functional engine driving the browser, enabling asynchronous data

fetching, dynamic component rendering, and single-page web execution. This is critical for making the multi-step custom cake builder highly responsive and fast for users. By handling data mutations locally inside the browser's state machine, JavaScript allows the system to update pricing models and alter 2D custom cake layers immediately as options are selected, eliminating the lag associated with constant server refreshes.

- **Tailwind CSS / Bootstrap Frameworks** These utility-first and component-driven frontend frameworks will be utilized for crafting the application's comprehensive visual styling, typography, and responsive layout grids. Rather than writing bulky, unoptimized custom style sheets from scratch, these tools provide highly compressed, pre-compiled layout configurations. This implementation ensures that the customer-facing storefront, the dense data views on the admin POS dashboards, and the lightweight rider verification links are fully responsive. The interface dynamically rearranges its grid structures, padding parameters, and component scales to look clean and highly readable across mobile phones, tablets, and desktop laptops.

- **HTML5 and CSS3.** HTML5 and CSS3 serve as the core technologies for developing the system's web interface. HTML5 provides the structural framework for the application's pages, including forms, input controls, image uploads, and interactive interface components. CSS3 is responsible for the visual presentation of the application by implementing responsive layouts, typography, color schemes, animations, and user interface styling. Together, these technologies create a clean, responsive, and user-friendly cake customization interface, administrator dashboard, point-of-sale module, and rider portal across desktop and mobile devices.

3.4.2 Cloud Infrastructure & Backend Services (Serverless)

The backend layer of the application bypasses traditional, monolithic server architectures in favor of a modern Cloud-Backend-as-a-Service (BaaS) paradigm. This tactical approach shifts infrastructure management, resource provisioning, scaling, and low-level security handling to managed cloud environments, ensuring high system uptime for Mrs. Brave's Cakes without requiring dedicated local technical personnel.

- **Google Firebase** Google Firebase will be deployed as the central Backend-as-a-Service (BaaS) provider for the entire digital platform. This architectural choice completely reduces the need to host, secure, patch, and maintain an expensive physical on-site backend server infrastructure or a dedicated virtual private server (VPS). Firebase aggregates vital backend utilities—including cloud functions, asset hosting, configuration variables, and serverless background workers—into a unified cloud environment. This drastically reduces server-side processing overhead, lowers data latency across local telecommunication networks, and provides a production-ready ecosystem capable of scaling automatically during peak bakery order periods.

- **Google Firestore** Google Firestore will serve as the primary, cloud-native NoSQL document database for the enterprise. Departing from traditional relational schemas that rely on rigid tables, rows, and heavy computational joins, Firestore stores data as lightweight, schema-less, JSON-like documents organized into distinct collections. This database engine will houses all operational data nodes, partitioned cleanly into specialized structures for custom design records, daily financial transaction logs, delivery

dispatch limits, calendar availability indices, and raw ingredient stock tracking.

Firestore utilizes native WebSockets to handle persistent client-database connections, processing real-time synchronization loops via active data listeners. Consequently, any mutation in the data layer—such as a successful web payment confirmation or a physical POS product deduction—automatically pushes instant, low-latency updates directly to the administrator's dashboard terminal without requiring traditional page reloads or polling commands.

- **Firebase Authentication** To safeguard the bakeshop's sensitive business data, corporate ledgers, and operational parameters, Firebase Authentication will be implemented to handle all cryptographic security protocols and identity management. This service provides a turnkey, enterprise-grade authentication system that safely manages administrator sign-ins, verifies user credentials, and maintains encrypted session tokens across the browser environment. By establishing role-based access tokens, this component ensures that the administration console, the backend pricing calculators, and the inventory adjustments remain completely isolated from public clients, mitigating the risk of cross-site scripting or unauthorized database overrides.

3.4.3 Third-Party APIs and Hardware Integrations

To ensure platform security and a frictionless user experience, the system incorporates three distinct external services. First, it integrates **re** at critical ion points—such as registration, login, and checkout—to distinguish human users from automated bots, effectively mitigating brute-force attacks and spam submissions. Second, for customer service automation, the system

utilizes the **Gemini 2.5 Flash-Lite** model via API, powering an intelligent conversational chatbot capable of answering frequent customer inquiries about product availability, store hours, and order tracking without administrative intervention. Lastly, the application connects to the **PayMongo API Gateway** to manage digital payment processing; when a checkout is initiated, the system securely routes the customer to the PayMongo interface to authenticate and complete electronic wallet payments (such as GCash), ensuring that real-time transaction states are safely synchronized back to the administrative sales logs.

- **PayMongo API Gateway** The PayMongo API will be integrated directly into the web application's checkout pipeline to handle real-time digital transaction processing. This payment gateway allows the platform to securely accept localized electronic payment methods—specifically GCash, Maya, and local credit cards—without directly processing, storing, or exposing sensitive financial details, bank credentials, or digital wallet pins inside the bakeshop's system database. By shifting payment tokenization to PayMongo's secure servers, the platform maintains a highly secure, transaction environment that seamlessly updates order workflows the moment funds are cleared.

- **Cloudinary Media Management API** Cloudinary will serve as the dedicated, cloud-native content delivery network (CDN) and media asset storage engine for the entire platform. To ensure that the customer-facing storefront and ive cake customizer remain incredibly lightweight and fast-loading, heavy media file handling is completely offloaded from the core Google Firestore database. Cloudinary automatically captures, optimizes, compresses, and resizes custom cake sample photos and reference images uploaded by both the administrator and customers. It then serves these media files through high-speed edge servers, ensuring that high-resolution design images load

quickly across local mobile networks without introducing latency to the backend.

- **Resend API (Transactional Email Engine)** The Resend API will function as the platform's automated customer communication engine. This specialized email delivery service bridges the database state machine with the customer’s inbox. The moment an administrator changes an order status within the dashboard backend—such as transitioning an active workflow entry from *Pending* to *Approved, Baking,* or *Out for Delivery*—the backend automatically compiles the transaction metadata, routes it through the Resend API, and dispatches a cleanly formatted, responsive email notification directly to the customer. This provides immediate operational updates and digital receipts without requiring manual communication from the bakery staff.

- **Google re v3** To safeguard the public-facing customizer forms, registration portals, and contact nodes from malicious web traffic, Google re will be woven directly into the frontend validation layer. This security tool runs invisible behavioral analysis algorithms in the background to verify human ions without disrupting the user experience. By identifying and blocking automated web scripts and malicious bots, re prevents automated spam from flooding the database with fake accounts, corrupt reference images, or brute-force bulk order requests, preserving the integrity of the bakeshop's data.

- **Thermal Printer Driver and Native Print Configuration** For on-site walk-in transactions, a localized hardware integration protocol is established inside the Point of Sale (POS) layout module. Because the system runs inside a standard web browser environment, the frontend utilizes customized CSS3 print media queries and structured text formatting rules aligned with the standard ESC/POS layout requirements. The system

processes completed transaction metrics and strips away standard browser margins, headers, and navigation UI components. It then passes a raw, cleanly formatted receipt layout directly to the local physical thermal receipt printer using native web browser print pipelines, enabling the bakeshop to generate instant, physical paper receipts right at the counter.

3.4.4 Development Environments

To maintain source code integrity, track system modifications, and streamline the software engineering pipeline across all five development stages, a combination of industry-standard coding environments and version control systems will be established. These tools provide the foundational workspace required to write, test, debug, and safely back up the platform's multi-layered architecture.

- **Visual Studio Code (VS Code)** Visual Studio Code will serve as the primary Integrated Development Environment (IDE) for writing, debugging, and managing the project's entire unified codebase throughout the development lifecycle. This highly extensible code editor will be configured with specialized technical modules, including syntax highlighters, real-time code linters, and embedded terminal instances. These tools allow the researchers to write clean frontend layout scripts, format complex backend business logic, and test cloud integration rules within a single workspace. VS Code's integrated local debugging tools enable the developers to isolate runtime script errors and intercept API call errors before code is pushed to production, accelerating the overall development sprint timeline.

- **Git and GitHub.** These tools will be used for source code version control, tracking software modifications throughout the development process, and maintaining secure cloud-based backups of the project repository. Git will run locally on the development machines to record incremental code changes and enable the researchers to create separate development branches for testing new features, such as enhancements to the cake customization interface, without affecting the stable version of the application.

  GitHub will serve as the remote repository for storing and synchronizing the project's source code. It provides centralized version management, preserves the complete development history, supports collaboration among the researchers, and maintains an audit trail of software revisions throughout the capstone project.

3.5 Data Gathering Techniques

To gather the necessary operational data and system requirements for designing the platform, the researchers will use primary qualitative data gathering methods tailored specifically to the business environment of Mrs. Brave’s Cakes. Relying on qualitative inquiry ensures that the software development life cycle is informed by the actual nuances, pain points, and administrative workflows experienced by the establishment. Rather than drawing conclusions from generalized bakery models, this targeted approach allows the researchers to extract precise functional specifications directly from the daily operations of the store.

The data collection process will rely on two main techniques:

3.5.1 Semi-Structured Interviews

The researchers will conduct formal, semi-structured interviews with the business owner, Mrs. Shamelyn Tapang. These qualitative sessions will target understanding the exact business logic, operational rules, financial rules, and administrative workflows of the bakery. Because this methodology utilizes a semi-structured format, the researchers will use a prepared interview guide as a structural baseline while maintaining the flexibility to ask follow-up questions as new operational details emerge. This open-ended approach is critical for uncovering underlying business processes that may not be formally documented in the bakeshop's current logs.

The primary objective of these interviews is to extract critical operational data points and business constraints required to build the platform's backend rules, specifically targeting three core business areas:

- **Pricing Matrices** The interviews will focus on defining how baseline prices scale according to specific cake tiers, precise structural dimensions, and specialized cake flavor combinations. The researchers will map out the exact mathematical rules the bakery uses to calculate raw material usage alongside labor costs for multi-layered items. This will ensure that the system's dynamic pricing engine accurately mimics the owner's manual pricing logic when calculating totals for a client.

- **Add-on Cost Calculations** The sessions will focus on mapping out the pricing tiers for premium customization variables that alter production time and material consumption. This includes establishing cost rules for intricate custom fondant work, specialized icing textures, edible printing components, and decorative cake toppers. Pinpointing these variables allows the researchers to construct precise data inputs within the ive customizer form, ensuring that any add-on selected by an online user dynamically calculates and

adds the correct surcharge to the invoice before checkout.

- **Logistical Pain Points** The inquiries will target documenting the specific administrative challenges, tracking errors, and operational bottlenecks that occur within the current manual business framework. The researchers will investigate how the bakery tracks partial down payments, verifies digital payment screenshots sent over chat applications, manages delivery coordination errors with independent couriers, and handles order volume overbooking during holiday peak seasons.

The empirical data gathered during these interviews will directly establish the calculation rules, backend algorithms, database validations, and workflow constraints used in the system's ive cake builder and administrative order tracking modules. This ensures that the finished digital platform structurally reinforces and automates the actual business policies of Mrs. Brave’s Cakes.

3.5.2 Direct Observation

The researchers will spend dedicated time on-site directly observing the daily, live operations of Mrs. Brave’s Cakes to understand exactly how transactions, design inquiries, and logistical coordination move from the initial customer contact point to final fulfillment. This immersive observation phase allows the researchers to look past anecdotal summaries and capture the actual behaviors, mechanical steps, and operational friction points within the bakeshop. By operating as passive observers within the business premises, the researchers can analyze the layout of the current workspace and document human-to-computer or human-to-paper ions in real time.

This observation phase will focus on two main operational areas:

- **The Customer Inquiry Pipeline** The researchers will review how incoming customer order requests, reference inspiration photos, and intricate structural modifications are manually handled, negotiated, and scheduled through Facebook Messenger chats. Observation will focus on tracking the timeline of a typical inquiry: how long it takes to manually calculate a price quote, how design modifications are discussed without visual standardization tools, and how final agreements are pinned down. This allows the researchers to witness firsthand the vulnerabilities of chat-based commerce, such as overlooked client messages, untracked design changes, and the lack of a structured verification method for down-payment screenshots.

- **The Manual Record-Keeping Process** This track involves closely observing how day-to-day business data is captured inside the physical shop. The researchers will watch how raw ingredient tracking, material inventory deductions, walk-in cash sales, production schedules, and freelance rider dispatch notes are recorded inside physical notebooks and paper logbooks. Special attention will be paid to the mechanical transition points—specifically, how a baker verifies if enough fondant or flour is in stock before accepting a flash order, and how physical receipts are hand-written for walk-in retail clients.

These direct observations will provide the empirical baseline needed to design the unified Admin Dashboard, the walk-in Point of Sale (POS) module, and the static link dispatch portal. By documenting where manual calculations slow down, where transcription errors occur, and where data silos exist, the researchers can ensure that the new serverless cloud software directly

addresses and resolves real-world operational bottlenecks for Mrs. Brave's Cakes.

3.6 Testing Procedures

Before final deployment, the software will undergo a structured, multi-tiered testing phase to identify hidden runtime bugs, secure end-to-end data pipelines, and ensure all system features work exactly as intended in a live environment. This verification phase bridges the raw programming sprints with live production, verifying that each digital module satisfies both its individual technical performance metrics and its macro-level business operational objectives.

The testing process will be broken down into three distinct levels:

3.6.1 Unit Testing

The developers will isolate and test individual software components to make sure they function correctly on their own before connecting them to the rest of the application ecosystem. By focusing on isolated code segments, function blocks, and mathematical calculations entirely separate from external database states or server environments, unit testing ensures that foundational logic blocks are structurally sound and error-free. This approach prevents minor programming syntax bugs from cascading into major system integration failures later in the development cycle.

This verification process will specifically focus on three critical functional areas within the platform's codebase:

- **Dynamic Price Calculation Logic.** Unit testing will be conducted on the system's dynamic price calculation module to verify the accuracy of its pricing algorithm. The developers will use a series of predefined test cases containing different combinations of cake sizes, flavors, layers, toppers, candles, and selected add-ons. The computed total generated by the system will be compared with manually calculated expected values to ensure that the application produces accurate, real-time price estimates before order information is stored in the database.

- **Cart Total Accuracy of the Walk-in POS Module** The point-of-sale module's transactional calculation scripts will undergo rigorous verification. This involves writing assertions to check how the local state management array accumulates individual retail product items, applies specific localized discounts, calculates correct tax deductions, and computes the gross final bill. These test vectors ensure that the on-site cash counter interface performs calculations with zero structural rounding errors or total mismatches during rapid walk-in checkout operations.

- **Input Form Validation Control** The developers will test the individual input validation gates across the customer registration, customization submission, and reservation scheduling panels. These unit tests will purposefully execute negative testing scenarios—submitting empty strings, invalid email syntax structures, improperly formatted phone numbers, or negative numeric bounds into the text fields. This ensures that the client-side forms instantly intercept, flag, and block malicious, blank, or broken

submissions right inside the user's browser, preventing corrupt data nodes from hitting the cloud database.

3.6.2 System Testing

After the individual parts and sub-modules are verified, the application will be tested as a complete, integrated whole. Rather than evaluating features in isolation, system testing subjects the fully compiled, unified application to comprehensive operational stress tests. The team will simulate the entire end-to-end operational workflow under realistic network conditions to ensure that data passes seamlessly, securely, and with minimal latency between the client-side frontend interfaces, the cloud database, and external microservice networks.

This testing phase will specifically focus on validating two macro-level system pipelines:

- **The End-to-End E-Commerce and Cloud Synchronization Workflow** The team will trace a complete transaction lifecycle to ensure all decoupled cloud layers communicate in harmony. Testing begins by initiating a mock transaction within the customer-facing cake builder portal. The system must successfully transition from capturing custom design parameters to processing a mock payment via the PayMongo API.

  The team will then verify that the external webhook securely triggers the system's backend, confirms the clearance of funds, and instantly synchronizes the transaction state inside the Google Firestore database. Finally, the test validates that this database mutation simultaneously updates the live administrative dashboard calendar for the kitchen staff

and triggers the automated transactional email receipt delivery to the customer's inbox without any manual intervention.

- **The Local Hardware Point of Sale (POS) Workflow** System testing will also evaluate the physical boundaries of the platform by conducting live hardware simulations. The researchers will process walk-in transactions through the specialized POS terminal layout to confirm that the software can translate digital checkout balances into physical real-world outputs.

  This test confirms that the application successfully strips away web browser interfaces, applies the necessary layout styles, and transmits raw formatting instructions directly to the local thermal receipt printer. The test is successful only when the physical hardware cleanly outputs a correctly aligned, legible paper receipt immediately upon walk-in order finalization.

3.6.3 User Acceptance Testing (UAT)

The final testing phase will involve handing the system over to real users to validate its practical effectiveness, intuitive flow, and operational value within a live retail environment. Shifting the software out of the development sandbox and into the hands of the actual stakeholders allows the researchers to evaluate real-world usability. This phase confirms that the application design matches human behavior and that the administrative modules align with the daily pacing of a commercial baking environment.

The UAT track will bring together two specific user groups to with the live cloud prototype:

- **The Primary Administrator (Business Owner)** Mrs. Shamelyn Tapang will directly operate the administrative console, the walk-in Point of Sale (POS) interface, and the inventory ledger. Her testing track will simulate a busy day of operations, focusing on processing incoming custom orders, evaluating the clarity of the scheduling calendar, updating material stock metrics, and checking the performance of the local thermal receipt printer. Her feedback is critical for verifying that the backend features successfully automate and simplify her executive business workflows.

- **The Target End-Users (Regular Customers, Staff, and Delivery Riders).** A selected sample group of regular customers, alongside retail staff and independent delivery riders, will evaluate the developed system through their respective portals. Customers will use the **online cake customization interface** to personalize cake orders and complete simulated online checkouts. Staff and riders will assess the usability of managing customer orders, viewing delivery locations through integrated maps, updating order statuses, and processing transactions efficiently under real-world operating conditions.

These participants will evaluate the platform based on its ease of use, interface clarity, system feedback speeds, and how well it actually solves the shop's day-to-day manual tracking and overbooking problems. The qualitative feedback, friction points, and feature ratings gathered during this phase will be meticulously documented by the researchers. This evaluation data will be used to make final user interface layout refinements, database query optimizations, and performance adjustments prior to the official system turnover, ensuring the finished software is effectively tailored to the operations of Mrs. Brave’s Cakes.

3.7 Evaluation Criteria

The software will be evaluated using adapted criteria based on the **ISO/IEC 25010 System and Software Quality Requirements and Evaluation (SQuaRE)** international standard. This approach ensures that the platform is objectively measured against recognized global software engineering metrics rather than relying on subjective feedback alone. By utilizing this structured quality framework, the researchers can assess both the technical integrity of the serverless backend and the practical utility of the client interfaces.

The evaluation will be conducted systematically by the business stakeholders, bakeshop staff, and selected end-users. The assessment will focus on the following four key quality characteristics tailored to the operational needs of Mrs. Brave's Cakes:

3.7.1 Functional Suitability

This criterion will evaluate how completely, accurately, and appropriately the developed system executes its intended features under active business conditions. Rather than assessing the software as a generic web application, testing and evaluation under this parameter will focus heavily on how well the platform satisfies the explicit functional requirements of the bakery. The goal is to confirm that the software contains all necessary tools to eliminate operational bottlenecks, enforce correct business logic, and support the staff's daily workflows without introducing errors.

The evaluation for Functional Suitability will focus on four critical technical milestones:

- **Precision of the Dynamic Price Calculations** The evaluation will verify that the software's backend calculation engine computes mathematically precise financial totals

within the ive customer-facing cake customizer. Evaluators will check if the pricing algorithm accurately aggregates base material fees, dimensional scaling modifiers, tier multipliers, and premium add-on surcharges (such as intricate fondant designs or decorative toppers). The final on-screen price must match the store's official pricing rules effectively, eliminating human calculation errors.

- **Reliability of the Walk-in Point of Sale (POS) Cart Module** This track assesses the functional completeness of the on-site cash counter interface. Testers will evaluate the system's ability to maintain real-time cart state accuracy during fast-paced walk-in retail sales. The evaluation ensures that adding or removing pre-baked products from the cart, applying promotional discounts, computing local taxes, and processing cash collections sync instantly with the central ledger without dropping data packets.

- **Successful Generation of Automated Transaction Documents** This milestone measures the accuracy of the platform's automated notification and data processing tasks. The system will be evaluated on its ability to capture database triggers and instantly compile structured transaction summaries. Functional suitability is achieved when the platform successfully dispatches a downloadable, mathematically accurate PDF invoice to the customer’s email via the notification API the moment an order is finalized or paid.

- **Physical Thermal Print Execution** Evaluators will test the hardware communication pipelines of the POS module. This involves confirming that the system can cleanly translate a digital transaction summary into raw, un-margined print commands. The application must successfully transmit formatting rules to the physical thermal receipt printer, outputting structured, properly aligned paper receipts immediately after an on-site

sale is finalized.

By verifying these targeted operational areas, this evaluation will ensure that the completed serverless platform successfully addresses, automates, and fixes the manual tracking problems, schedule overbookings, and financial logging inefficiencies it was engineered to resolve for Mrs. Brave's Cakes.

3.7.2 Usability

This criterion will measure the user-friendliness, intuitiveness, and overall accessibility of the application's decentralized interfaces. Because the system bridges diverse user segments with varying levels of technical literacy—ranging from online retail shoppers and third-party delivery couriers to bakeshop personnel and the executive administrator—the user interface (UI) must be self-explanatory and structurally consistent. The evaluation will assess how smoothly these distinct user groups can complete tasks without experiencing interface friction, cognitive fatigue, or input confusion.

The usability metrics will be evaluated across two primary operational pipelines:

- **Customer Storefront Navigation and Customization User Experience (UX).** The evaluation will assess how easily sample customers can interact with the responsive web storefront and use the **cake customization interface**. Evaluators will determine whether the interface provides clear visual feedback as users select cake size, flavor, layers, decorations, add-ons, and upload design reference images.

  Usability under this sub-metric will be measured by the extent to which first-time users

can successfully customize a cake, view real-time price updates, review their order details, and complete the online checkout process without requiring assistance, troubleshooting guides, or administrative intervention.

- **Administrative Command Center Operability** Furthermore, this metric will measure how quickly and comfortably the business owner, Mrs. Tapang, can learn to manage backend business operations through the unified Admin Dashboard. The assessment will focus on the ergonomic layout of data fields, input forms, and navigation links. Evaluators will measure her ability to adjust the ingredient inventory ledger, cross-reference calendar reservation slots, modify custom order statuses, and read the visual sales analytics charts without requiring advanced technical training.

By analyzing these targeted areas, the usability score will verify that the layout designs, component arrangements, typographic scales, and color hierarchies provided by Tailwind CSS and Bootstrap deliver a clean, efficient, and natural user experience across mobile devices, tablets, and desktop monitors.

3.7.3 Security

This criterion will focus on the systematic protection of transactional data, user identity records, and administrative access privileges across the entire application architecture. Given that the platform processes digital payments and handles private consumer information, maintaining a rigid security posture is critical to preventing malicious data manipulation and unauthorized data exposure. The evaluation will measure how effectively the application's cloud infrastructure establishes and enforces cryptographic guardrails, token validations, and endpoint restrictions

under active business conditions.

The security evaluation will specifically target two primary protection mechanisms within the system:

- **Identity Management and Role-Based Access Controls** The evaluation will verify that the backend security configuration and Firebase Authentication rules correctly restrict unauthorized users from accessing the Admin Dashboard or hitting sensitive data endpoints. Evaluators will test the perimeter defenses of the system by attempting to access administrative URLs, financial logs, and inventory controls without a valid active session token.

  Security success requires that the system intercept unauthorized routing attempts, invalidate falsified session requests, and maintain strict cryptographic isolation between public customer storefront views and private backend administrative modules.

- **Financial Data Security and Tokenization** Additionally, the assessment will confirm that the PayMongo API integration securely processes digital wallet payments—specifically GCash and Maya—without introducing security compliance vulnerabilities. Evaluators will audit the data transmission packets sent during checkouts to confirm that the platform never captures, exposes, or logs sensitive consumer financial data, pin codes, or bank credentials within the local system or the central Firestore database.

  By confirming that payment operations rely entirely on secure, external API tokenization and encrypted asynchronous webhooks, this evaluation ensures that the platform provides a highly secure e-commerce environment for the customers of Mrs. Brave's Cakes.

3.7.4 Performance Efficiency

This criterion will measure the overall responsiveness, execution speed, data throughput, and client-side resource usage of the system under operational conditions. In a serverless client-cloud architecture, performance efficiency is heavily tied to script optimization, asset management, and the architectural design of database queries. The evaluation will analyze how efficiently the system utilizes cloud network pipelines and browser memory to minimize latency, ensuring that users do not experience structural delays, lagging inputs, or data sync drops during high-volume periods.

The performance efficiency evaluation will specifically focus on two key technical execution fields:

- **Real-Time Data Synchronization and Query Latency** The evaluation will analyze how quickly the serverless architecture retrieves, mutates, and transmits data packets across the cloud network. Evaluators will measure the real-time synchronization speed of the Google Firestore database when updating the administrative booking calendar and inventory ledger.

  Performance success requires that mutations—such as a successful custom order checkout or an instantaneous stock deduction—propagate via active WebSockets to the admin panel within milliseconds. This rapid synchronization loop is critical for ensuring accurate schedule availability, providing immediate operational visibility, and preventing double-booking conflicts during peak bakery seasons.

- **Asset Optimization and Mobile Responsive Render Times** Additionally, this metric

will evaluate the frontend rendering speeds and mobile responsiveness of the Tailwind CSS and Bootstrap storefront layouts. Because a majority of online consumers place orders using cellular networks on mobile phones, the platform's initial page load times and core web vitals will be closely monitored.

The evaluation will measure how efficiently the system displays structural layouts and pulls optimized custom cake images from the Cloudinary CDN. The application must achieve rapid document object model (DOM) rendering, ensuring that the ive cake customizer remains smooth and lightweight on mobile web browsers without consuming excessive local device memory or battery power.

3.8 Supporting Diagrams

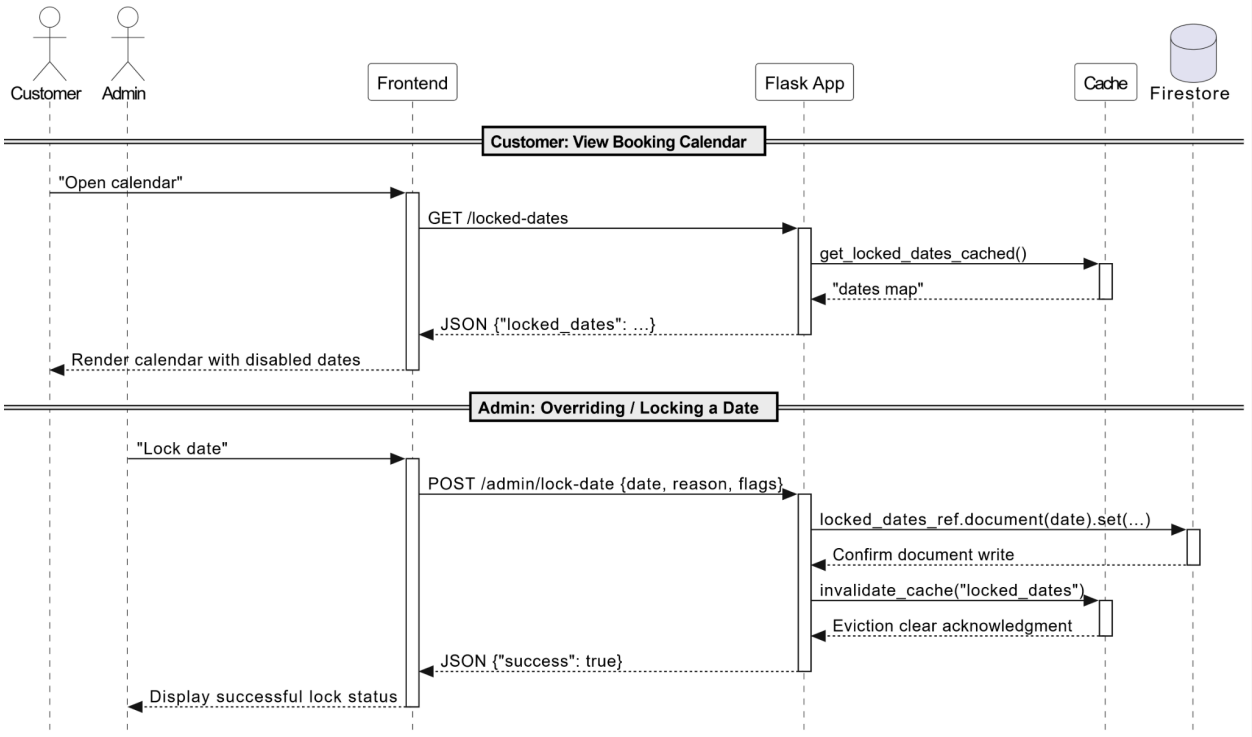
Because the platform utilizes a NoSQL database architecture through Google Firestore, traditional relational Entity-Relationship Diagrams (ERDs) are not applicable to its non-relational, document-oriented structure. Relational structures rely on rigid foreign keys and schema enforcement, whereas Firestore operates on an independent architecture of nested collections and document nodes. Consequently, the system's operational processes, data transaction loops, and real-time state mutations are mapped out comprehensively using a series of specialized UML Sequence Diagrams.

These diagrams illustrate the exact message-driven, chronological ions between the platform's primary actors and its decoupled service layers: the Customer, the responsive Frontend Web Interface, the Python Flask Application, the local Caching Layer, the Cloud Database (Firestore),

external payment gateways (PayMongo), and external utility notification services (FCM).

3.8.1 Calendar Optimization & Asynchronous Cache Invalidation Flow

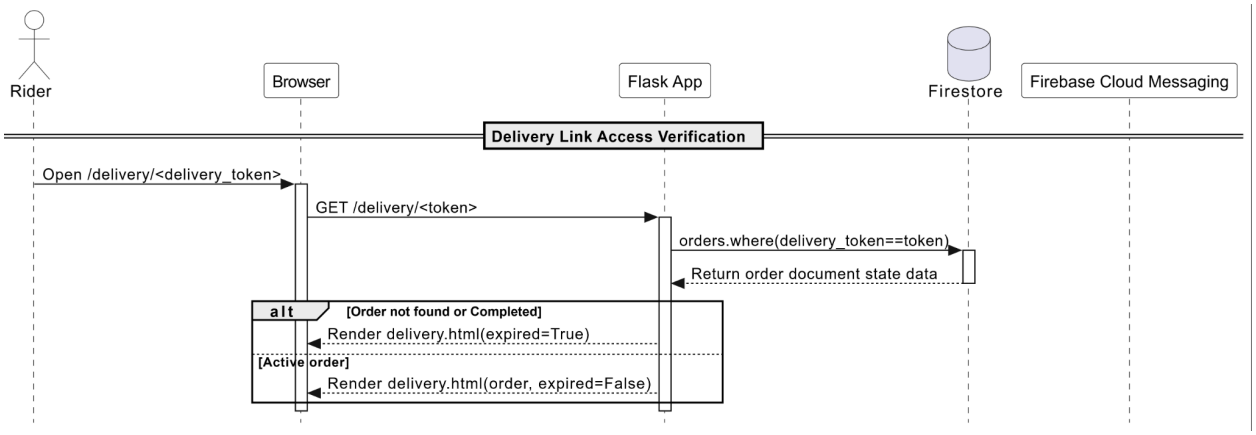
This diagram maps out the platform's scheduling optimization guards. It illustrates the real-time request-response routing deployed to handle calendar views and prevent holiday overbooking. It details how customer-facing date-checks are optimized through a localized memory cache to minimize Firestore read consumption, alongside how explicit administrative date-locks bypass the cache to mutate core collections in Google Firestore and evict stale operational data instantly.

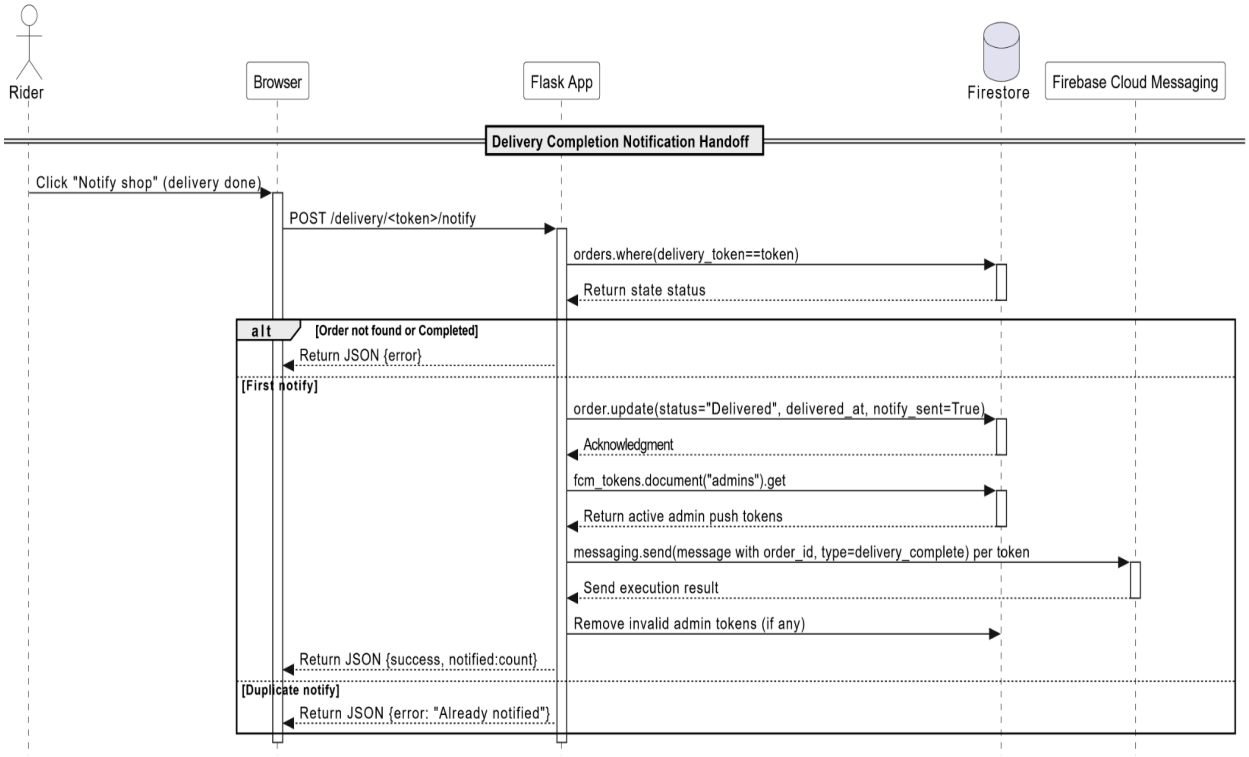


3.8.2 Delivery Fulfillment Link & Admin Notification Tracking

This diagram details the fulfillment orchestration pipeline. It shows the message

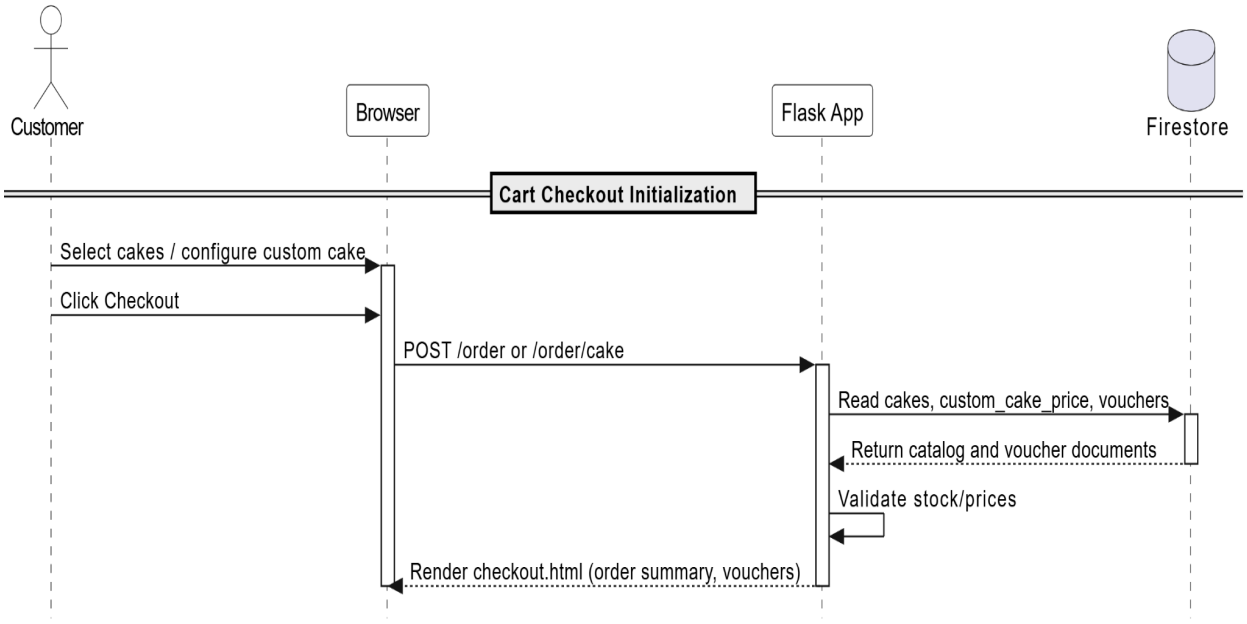
exchange triggered when a delivery rider accesses a temporary transit routing token. It highlights how the backend queries Firestore to prevent riders from accessing stale or expired order tokens, handles live fulfillment events, tracks single-token delivery verification updates, and leverages Firebase Cloud Messaging (FCM) to push native alert states directly to the administrative monitoring layout.

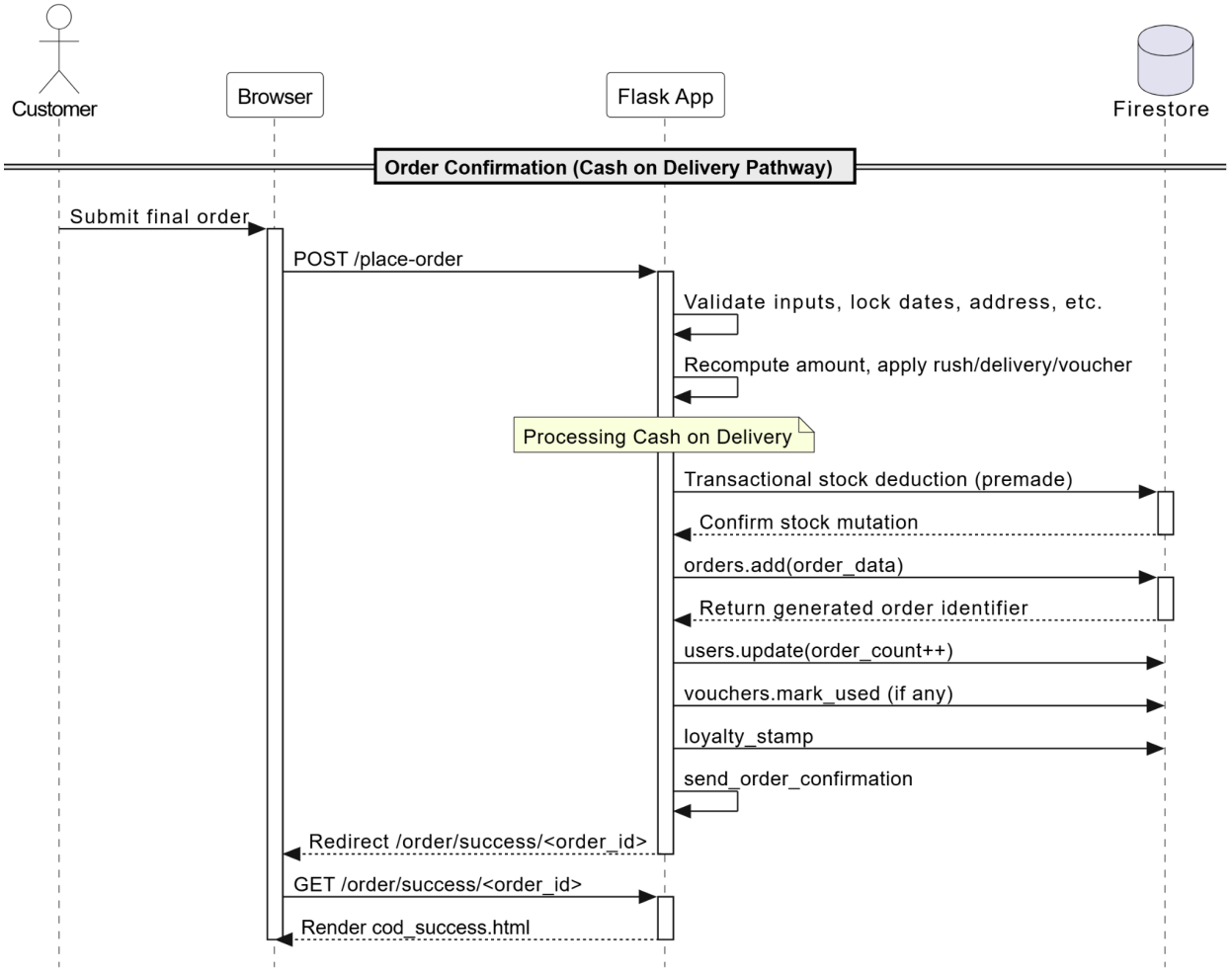




3.8.3 End-to-End Customer Checkout & Cash on Delivery Processing

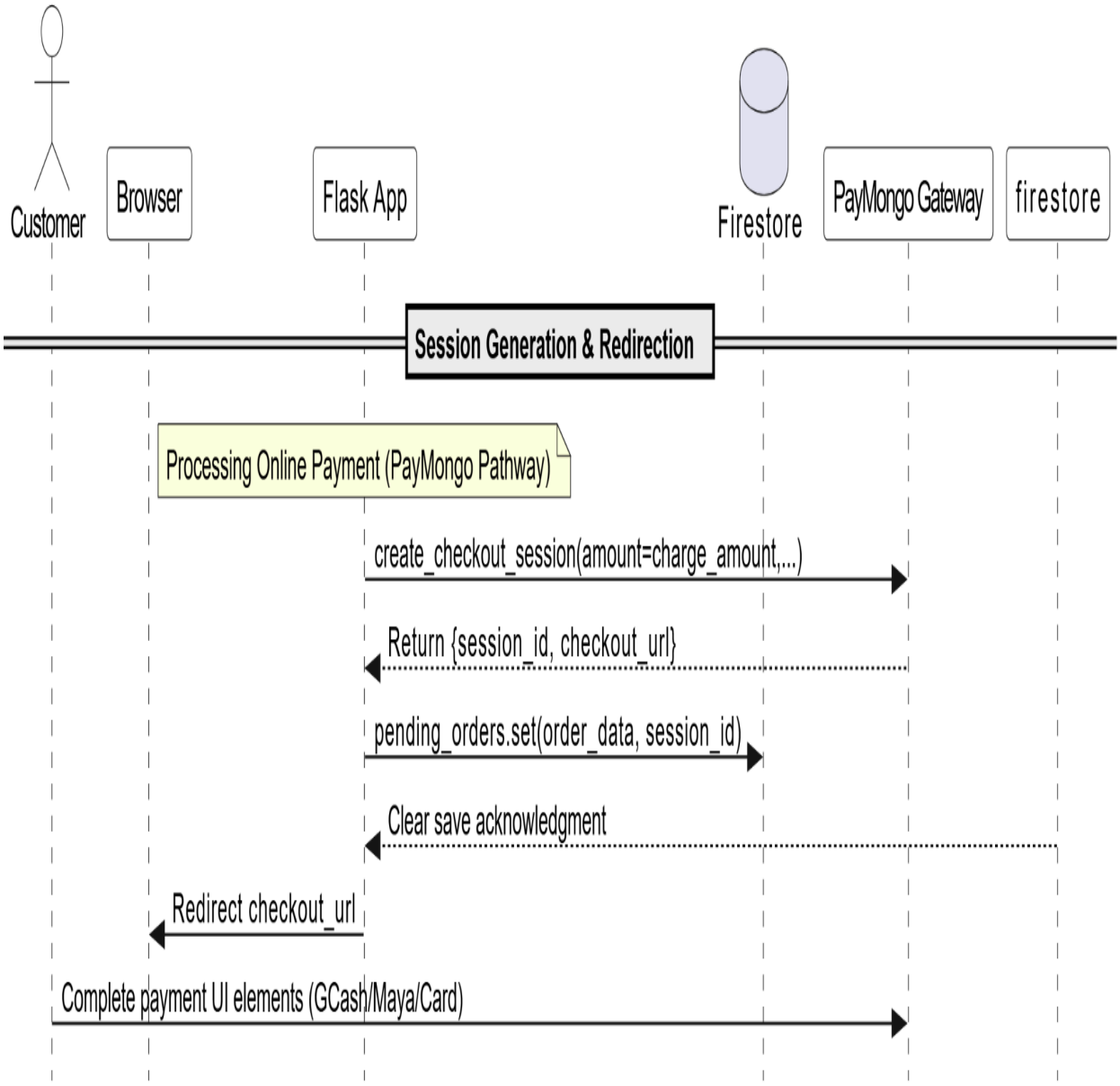
This diagram maps out the initial transactional logic branch within the platform checkout ecosystem. It presents the unified data sequence executing when a customer triggers their shopping basket handoff, validating stock variables and running price validation matrices internally before committing permanent database changes for Cash on Delivery (COD) transactions.

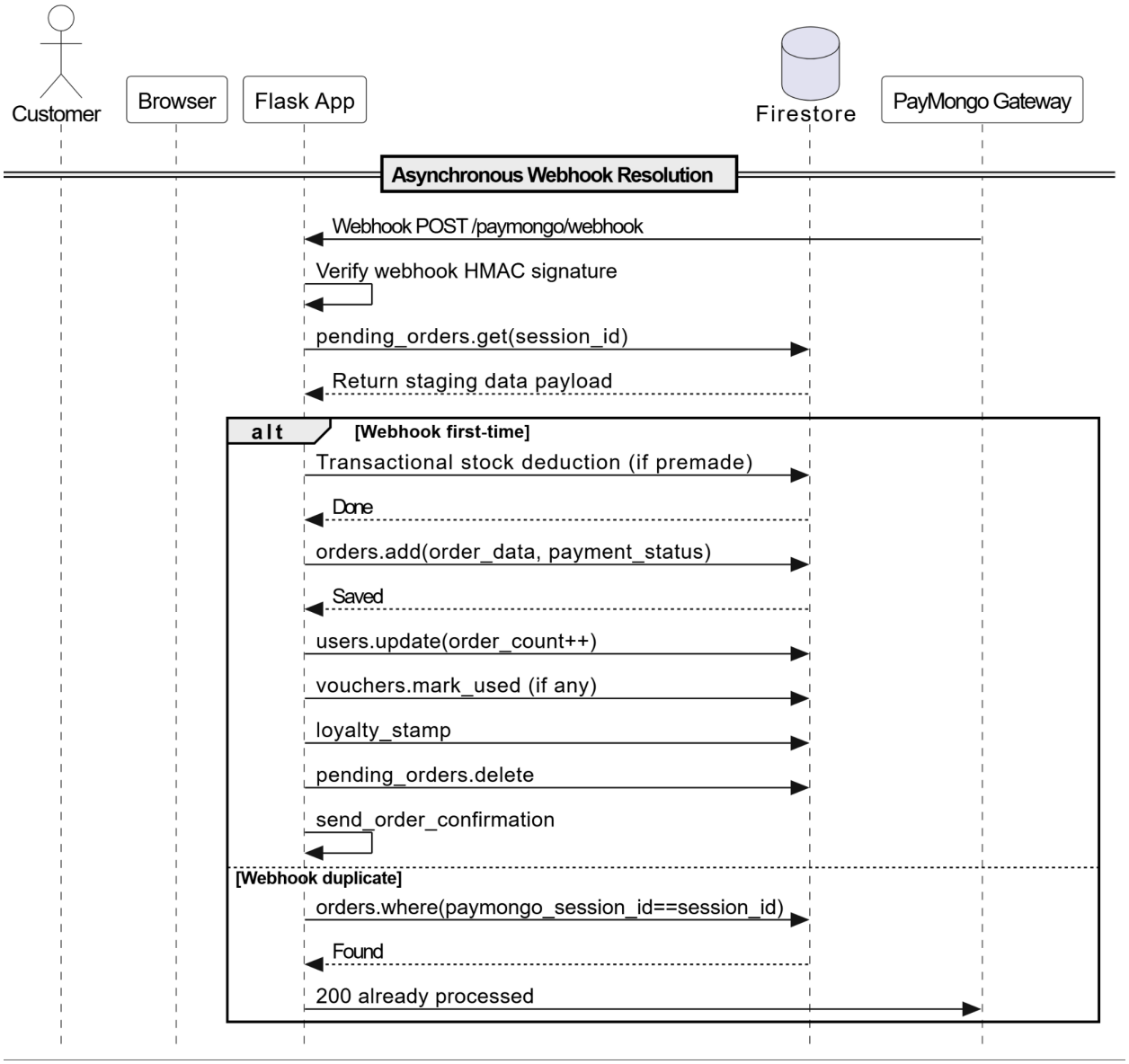

Customer
Browser
Flask App
Firestore
Cart Checkout Initialization
Select cakes / configure custom cake
Click Checkout
POST /order or /order/cake
Read cakes, custom_cake_price, vouchers
Return catalog and voucher documents
Validate stock/prices
Render checkout.html (order summary, vouchers)

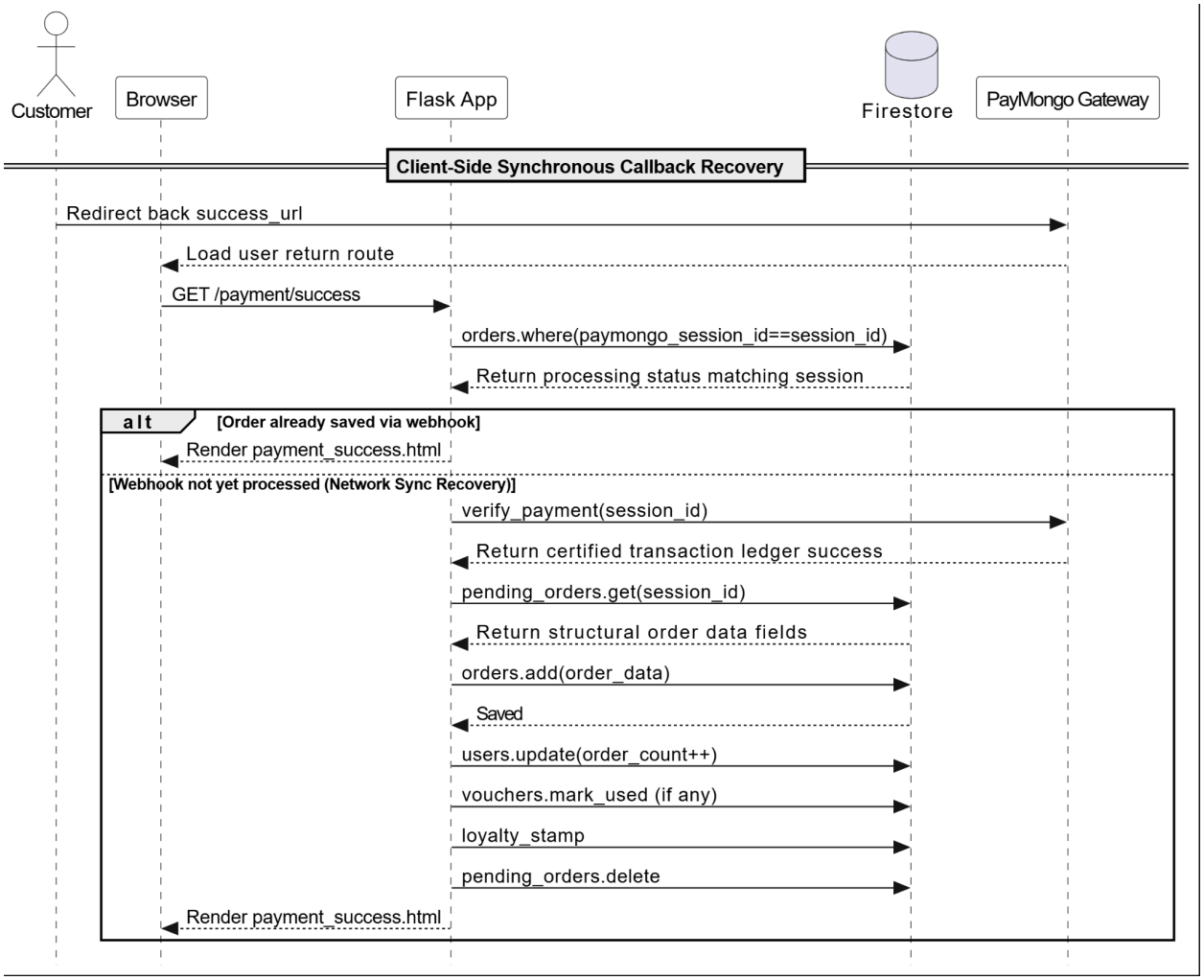

Customer
Browser
Flask App
Firestore
Order Confirmation (Cash on Delivery Pathway)
Submit final order
POST /place-order
Validate inputs, lock dates, address, etc.
Recompute amount, apply rush/delivery/voucher
Processing Cash on Delivery
Transactional stock deduction (premade)
Confirm stock mutation
orders.add(order_data)
Return generated order identifier
users.update(order_count++)
vouchers.mark_used (if any)
loyalty_stamp
send_order_confirmation
Redirect /order/success/<order_id>
GET /order/success/<order_id>
Render cod_success.html

3.8.4 Online Payment Gateway Lifecycle & Webhook Processing

This diagram breaks down the system synchronization logic executed for online settlement lanes using the external PayMongo API infrastructure. It details the complex asynchronous states running simultaneously between client browser interfaces, signature-verified payment confirmation webhooks, and programmatic recovery methods used to verify order consistency and preserve database integrity under unstable network conditions.


Customer
Browser
Flask App
Firestore
PayMongo Gateway
firestore
Session Generation & Redirection
Processing Online Payment (PayMongo Pathway)
create_checkout_session(amount=charge_amount,...)
Return {session_id, checkout_url}
pending_orders.set(order_data, session_id)
Clear save acknowledgment
Redirect checkout_url
Complete payment UI elements (GCash/Maya/Card)


Customer
Browser
Flask App
Firestore
PayMongo Gateway
Asynchronous Webhook Resolution
Webhook POST /paymongo/webhook
Verify webhook HMAC signature
pending_orders.get(session_id)
Return staging data payload
alt
[Webhook first-time]
Transactional stock deduction (if premade)
Done
orders.add(order_data, payment_status)
Saved
users.update(order_count++)
vouchers.mark_used (if any)
loyalty_stamp
pending_orders.delete
send_order_confirmation
[Webhook duplicate]
orders.where(paymongo_session_id==session_id)
Found
200 already processed



3.9 NoSQL Firestore Database Architecture and Data Dictionary

Because the system is built on a modern serverless infrastructure, it utilizes **Google Firestore**, a cloud-hosted, document-oriented NoSQL database, rather than a traditional Relational Database Management System (RDBMS). Firestore abandons fixed tables, strict row boundaries, and foreign key constraints in favor of a hierarchical model composed of **Collections** and **Documents**. Documents contain specific key-value pairs, nested maps, arrays, and primitives. This schema-less flexibility allows variable-heavy attributes—such as dynamic cake

configurations, variable delivery link metadata, and tracking parameters—to be nested cleanly inside a single database reference node.

The following section provides the complete technical data dictionary for the core administrative security logging collection deployed within the cakeshop-2faf4 project instance.

3.9.1 Root Collection: admin_logs

A. Subsystem Context and Purpose

The admin_logs collection forms the core security auditing subsystem of the platform. In a cloud-hosted environment, tracking the operations of highly privileged users is critical for maintaining data integrity. This collection automatically intercepts and archives every backend state mutation executed through the administrative control panel.

Whenever an administrator accepts an order, locks a date on the calendar, adjusts product prices, or voids a transaction, the backend framework instantly dispatches an un-indexed write command to this collection. By logging personnel details, client network parameters, action metrics, and target resource markers, the system ensures complete operational transparency, simplifies debugging during state conflicts, and satisfies standard security compliance audits for digital retail platforms.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Unique 20-character, cryptographically secure alphanumeric hash.
- **Generation Engine:** Automatically generated client-side by the Google Cloud Firestore SDK upon document insertion (e.g., 020N4vY8orjqc4mVavin).

- **Storage Pattern:** Flat, append-only transactional document log node. Documents within this collection are never modified or deleted by normal system routines to preserve audit integrity.
- **Indexing Strategy:** Single-field automatic indexes are applied to the timestamp and category fields to ensure high-speed sorting when rendering history timelines on the admin panel.

C. Field-Level Structural Rules

Table 3.4: admin_logs Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
action	String	No	"Changed order status to 'Accepted'"	A descriptive string detailing the precise database mutation or business logic

				operation executed by the administrator.
admin_name	String	No	"Justin Solamillo"	The explicit full display name extracted from the administrator's account profile to identify the individual responsible for the operation.
category	String	No	"order"	A rigid categorization string token used by the system to filter logs into

				specific operational modules. Accepted tokens are constrained to: "order", "inventory", "calendar", or "expense".
ip_address	String	No	"127.0.0.1"	The client internet protocol address captured at request execution time, utilized to track geographic access origins

				and detect suspicious remote sessions.
target	String	No	"Order #Y51X4gBTGXJjx1nrGrd6 — Justin Solamillo"	A specialized compound reference string that explicitly identifies the specific external collection document instance modified by this event.

timestamp	Timestamp	No	May 17, 2026 at 8:41:26 PM UTC+8	A server-side generated nanosecond clock marker recording the exact moment the write was committed to the Firestore cluster.

D. Operational Flow and Security Considerations

Data entries into admin_logs are protected by strict **Firestore Security Rules**. The database configuration enforces that normal CUSTOMER or DELIVERY_RIDER roles are completely restricted from reading, creating, or modifying documents within this path.

Only accounts possessing a validated account_role == "ADMIN" property inside their Firebase Authentication custom claims tokens can append logs. Furthermore, the application framework relies on automated serverless middleware to structure these inputs, preventing manual tampering or masking of audit identities.

3.9.2 Root Collection: cakes

A. Subsystem Context and Purpose

The cakes collection serves as the central digital inventory and menu repository for the storefront web application. In a cloud-hosted e-commerce ecosystem, dynamic data extraction is required to keep customer views synchronized with the bakery's real-time physical production capabilities. This collection holds structural documents for every active item available for public ordering—ranging from standardized everyday retail desserts to premium cakes requiring strict stock management tracking.

The Python Flask App repeatedly reads from this collection to populate the customer menu grid, verify item prices during checkout payload computations, and evaluate if an item has enough physical stock to satisfy incoming order arrays. By structuring these inventory records in a centralized collection node, the system ensures that changes made by administrators in the backend panel automatically update customer catalog menus across all active browser windows.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character cryptographically secure alphanumeric hash key.
- **Generation Engine:** Instantiated automatically by the Google Cloud Firestore SDK upon catalog creation (e.g., OVJABOZNjwwYHp7Vbzhv).
- **Storage Pattern:** Independent document blocks containing flat primitives and static absolute resource strings.
- **Indexing Strategy:** Composite indexes are defined for category and status to let users quickly filter products on the frontend storefront while skipping unavailable or

out-of-stock items.

C. Field-Level Structural Rules

Table 3.5: cakes Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
category	String	No	"Dessert"	The grouping tag used by frontend query loops to separate menu items into distinct user layout rows (e.g., "Dessert", "Custom", "Premium").

created_at	Timestamp	No	April 24, 2026 at 9:35:54 AM UTC+8	Records the exact chronological date and time the product was introduced into the digital catalog matrix.
description	String	No	"Pistachio Cheesecake"	A public-facing description text block used to details the specific flavor profile, ingredients, or allergens of the menu item.

image	String	No	"https://res.cloudinary.com/..."	An absolute URL pointing directly to an optimized web graphic asset stored within the project's external Cloudinary CDN bucket.
name	String	No	"Pistachio Cheesecake"	The master item string variable rendered as the prominent header text inside storefront cards and point-of-sale receipt slips.

price	Number (Integer)	No	250	The raw baseline unit cost of the item before shipping fees, rush service markups, or promotional coupon vouchers are applied.
quantity	Number (Integer)	No	8	The live physical inventory count. This value is decremented programmatically during automated checkouts to prevent

				over-purchasing.
status	Boolean	No	true	A boolean visibility control switch. Setting this to true exposes the item to the public web interface, while false hides it instantly without deleting its historical records.

D. Operational Flow and Security Considerations

Security rules configured for the cakes tree utilize an explicit read/write split based on authentication states. Public internet browsers are granted free read access (allow read: if true;) to fetch catalog names, pricing values, and image endpoints without needing an account session.

Conversely, creating, modifying, or deleting document properties inside this data tree is locked tight. It requires a validated Firebase Authentication signature identifying the user as an authorized employee (allow write: if request.auth.token.role == 'ADMIN';). This completely

blocks unauthorized API requests from tampering with prices or inflating product stock counts.

3.9.4 Root Collection: custom_cake_price

A. Subsystem Context and Purpose

The custom_cake_price collection acts as the dynamic configuration utility for managing the cost variables of custom cakes. Instead of hardcoding pricing thresholds within the backend server logic, this data path provides editable rules that the multi-step frontend configuration calculator references directly. This approach ensures that updates to standard tier pricing or specialized service fees apply instantly to incoming user design selections.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Alphanumeric descriptor matching specific pricing rule matrices.
- **Generation Engine:** Structured manually during database installation to provide reliable routing keys.
- **Storage Pattern:** Flat document entry maps containing numeric parameter lists.

C. Field-Level Structural Rules

Table 3.7: custom_cake_price Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
base_tier_1	Number (Integer)	No	1500	Minimum unit cost calculated for single-tier configuration projects before custom icing additions.
base_tier_2	Number (Integer)	No	2800	Baseline cost added automatically for multi-tiered structure layout choices.

fondant_modifier	Number (Integer)	No	500	Additional markup cost variable added for specialized icing designs.
rush_fee	Number (Integer)	No	350	Operational surcharge applied to calculations if selection dates sit within a 48-hour range.
update_marker	Timestamp	No	May 22, 2026 at 10:15:30 AM UTC+8	Standard calendar milestone tracking when adjustments were latest saved by system

				administrators.

D. Operational Flow and Security Considerations

Data assets mapped under the custom_cake_price collection enforce public evaluation permissions alongside secure administrative editing locks. Store clients are permitted standard fetch calls to parse active configuration elements into local pricing modules. Structural adjustments are completely restricted to administrative profile sessions (request.auth.token.role == 'ADMIN') to avoid external manipulation of financial rule matrices.

3.9.5 Root Collection: expenses

A. Subsystem Context and Purpose

The expenses collection functions as the primary financial ledger for tracking outbound operating costs, inventory replenishment outlays, and utility overhead expenses managed through the administration system. Whenever an administrator modifies ingredient volumes or purchases stock, the transactional system automatically saves a detailed record to this path.

By recording financial metrics—such as unit costs, precise timing logs, and item change logs—the application provides store management with a reliable audit trail for daily overhead tracking. This design isolates financial transactions from active inventory nodes, simplifying end-of-month accounting and budget calculations.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric unique identifier hash.
- **Generation Engine:** Generated automatically by the Cloud Firestore SDK at the moment of ledger document insertion (e.g., 02zMQALpTyyKH6sLa654).
- **Storage Pattern:** Flat root reference nodes acting as append-only ledger entries.

C. Field-Level Structural Rules

Table 3.8: expenses Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
cost	Number (Integer)	No	900	The total numeric monetary value of the financial expense transaction.

date	Timestamp	No	April 5, 2026 at 11:41:37 AM UTC+8	Explicit chronological tracking marker recording exactly when the expense was logged.
description	String	No	"Edited inventory: Flour(by kilo) - Qty: 10, Cost: ₱90.0"	Descriptive text detailing the breakdown, quantity, unit rates, and specific operational reason for the ledger entry.

D. Operational Flow and Security Considerations

Entries in the expenses collection are protected by backend validation constraints. Because these records contain sensitive financial data, public read access is entirely blocked.

Firestore Security Rules enforce that only authenticated accounts with a validated manager token (request.auth.token.role == 'ADMIN') can search, read, or append entries to this data path. To

preserve audit trail integrity, modification and deletion operations are completely disabled once a document is committed.

3.9.6 Root Collection: fcm_tokens

A. Subsystem Context and Purpose

The fcm_tokens collection acts as the target address registry for the platform's real-time alert systems. To send asynchronous push notifications—such as order status updates, newly submitted custom cake requests, or instant message warnings—the Firebase Cloud Messaging (FCM) engine requires valid hardware registration token paths. This collection maps active device strings to user identity roles, allowing the backend framework to group and push out automated event broadcasts immediately without scanning the entire central user table.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Fixed systemic alphanumeric strings designating specific account role classes.
- **Generation Engine:** Initialized intentionally on the console database configuration layer to act as predictable partition paths (e.g., admins).
- **Storage Pattern:** Single-document dictionaries containing multiple dynamic key-value pairs where keys map to randomized device handles and values store the cloud token string arrays.

C. Field-Level Structural Rules

Table 3.9: fcm_tokens Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
KPanLhrWCZ...	String	No	"cosGnr-X5iaJXpbK...KY4k5mV4VwqPSM63Poqg..."	A dynamic field key representing an active device instance identifier, mapping

				to its unique Firebase Cloud Messaging token string value.
RuNTPfmm9N...	String	No	"foR7xZWBwW56XoreOVGSdf:APA91b...jOuGw"	A dynamic field key storing the encrypted cloud message token value for an administrative

				device.
nlcXidMhGx...	String	No	"fxwhllytqLA1lldurbqqrj:APA91bFAJePc..RJ6AO1oYTyXZ..."	A dynamic field key mapping an additional unique connection instance to its live background push verification handle.

D. Operational Flow and Security Considerations

Because registration data inside fcm_tokens directly impacts device access vectors and client alert security, data mutation capabilities are strictly restricted. Client frontend applications can call programmatically bounded updates to register their own device vectors upon successful multi-factor authentication loops. Global retrieval paths or multi-node deletion routines are restricted entirely to verified administrator roles (request.auth.token.role == 'ADMIN') to prevent token hijacking or unauthorized alert blocking.

3.9.7 Root Collection: inventory

A. Subsystem Context and Purpose

The inventory collection serves as the raw material resource tracker for the baking application ecosystem. Rather than evaluating supplies manually, the production framework tracks asset availability—such as bulk raw ingredients, baking supplies, and finishing additions—programmatically through this path.

When administrators make structural stock revisions or purchase new assets, background mutations calculate the material costs and update the current volume counts. This automation allows the storefront to dynamically confirm material readiness before executing custom order checkout pipelines.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric unique identifier hash.

- **Generation Engine:** Provisioned automatically by the cloud database layer upon raw material registration entry initialization (e.g., 36ttDHUqirSMKIRhpymh).
- **Storage Pattern:** Flat root reference nodes tracking active material items.

C. Field-Level Structural Rules

Table 3.10: Inventory Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
category	String	No	"Ingredients"	Logical grouping tag classifying resource variants to optimize filtering across stockrooms (e.g., "Ingredients", "Packaging").

cost	Number (Integer)	No	350	The calculated unit procurement value or asset purchase expense recorded for the specific resource quantity.
created_at	Timestamp	No	April 6, 2026 at 10:49:48 PM UTC+8	Standard system clock marker archiving the absolute calendar creation date of the inventory line item.
item	String	No	"fondant"	Human-readable specific naming descriptive string used to identify the production asset or

				ingredient.
quantity	Number (Integer)	No	23	The real-time live stock volume unit count currently resting inside physical warehouse storage arrays.
updated_at	Timestamp	No	May 26, 2026 at 5:24:36 PM UTC+8	Tracking marker recording the precise moment an administrative transaction last modified this inventory record.

D. Operational Flow and Security Considerations

Data properties grouped within the inventory branch are isolated from public client mutation paths to maintain logistical accuracy. Guest or standard customer user categories can view material availability summaries when assembling custom baking designs. However, direct write modifications, balance adjustments, or asset removals are restricted by security rules to

verified administrator privileges (request.auth.token.role == 'ADMIN').

3.9.8 Root Collection: locked_dates

A. Subsystem Context and Purpose

The locked_dates collection serves as the core scheduling validator and constraint engine for the storefront reservation flow. To maintain kitchen delivery guarantees and prevent order over-saturation, the platform reads this path before rendering the calendar date-picker tool to customers during checkout. If an administrator manually sets a calendar block or if automated operations detect that a specific production date has met its maximum volume capacities, the target date is indexed here to instantly grey out that slot on the frontend interface.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Structured calendar date identifier strings formatted strictly as YYYY-MM-DD.
- **Generation Engine:** Provisioned explicitly based on the chosen blocked calendar date to allow high-speed direct document lookup queries (e.g., 2026-05-17).
- **Storage Pattern:** Flat systemic rule nodes storing operational block details.

C. Field-Level Structural Rules

Table 3.11: locked_dates Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
locked_at	Timestamp	No	May 17, 2026 at 11:32:27 PM UTC+8	Explicit chronological tracking marker recording exactly when the calendar restriction rule was committed.
locked_by	String	Yes	null	Stores the unique account profile handle or system automation ID that initialized the date block.

reason	String	No	"Fully Booked"	Descriptive status label displayed inside administrative calendars or tooltips to explain the scheduling block (e.g., "Fully Booked", "Holiday Closure").

D. Operational Flow and Security Considerations

Because scheduling data within locked_dates alters checkout rules and impacts incoming store revenues, the validation nodes rely on strict read/write permission divisions. Storefront buyers are granted open read parameters (allow read: if true;) to parse active block sets directly into the customer appointment calendar view. Conversely, creating new blocks or removing existing documents requires a verified administrator session verification claim (request.auth.token.role == 'ADMIN') to block unauthorized scheduling tampering.

3.9.9 Root Collection: notifications

A. Subsystem Context and Purpose

The notifications collection manages user-facing application alerts, acting as the historic log for systemic message events pushed to customer accounts. Whenever background payment hooks or administrative actions modify an order status, the server automatically appends a tracking block here. This allows client frontends to display reactive status notification lists, drop-down activity logs, and real-time alerts without constantly re-verifying active order detail matrices.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash identifier key.
- **Generation Engine:** Created automatically by the cloud framework routing rules upon notification document insertion (e.g., 0X69YpNEXxhavGcQrWM5).
- **Storage Pattern:** Flat, append-only history log nodes linked directly to user session targets.

C. Field-Level Structural Rules

Table 3.12: notifications Field-Level Schema Rules

Structur	Storage	Nullabl	Mock/Live Production Sample	Technical

al Field Key	Primitive	e?	Data	Constraints & Operational Purpose
created_at	Timestamp	No	May 19, 2026 at 1:29:40 AM UTC+8	Explicit chronological tracking marker storing the exact time an event log was recorded.
is_read	Boolean	No	true	UI flag tracking whether the user has viewed the alert, filtering dashboard notification badge counts.

message	String	No	"Your order #JPyR9H6T has been accepted"	Clear body string containing the actual text message displayed directly to the end user.
order_id	String	No	"JPyR9H6T1RnGjrqx1org"	Cross-reference value pointing directly back to the matching source document in the orders path.

title	String	No	"Order Accepted"	The bold headline text string displayed at the top of customer layout cards or modal alert bubbles.
type	String	No	"status_update"	Categorization token used by client routing logic to load custom layout icon wrappers (e.g., "status_update", "chat").

user_id	String	No	"KPanLhrWCZfXGDATn4AfH6Ipd1E3"	Target consumer identification string mapping the alert record directly to its owner profile.

D. Operational Flow and Security Considerations

Data entries in the notifications tree employ selective database routing rules to ensure privacy. Clients are granted read and update access solely for documents where the profile string matches their active session token (allow read, update: if request.auth.uid == resource.data.user_id;). This allows users to mark their own notifications as read while blocking third-party interception. Full record additions remain isolated to internal server processes and administrative account actions (request.auth.token.role == 'ADMIN').

3.9.10 Root Collection: orders

A. Subsystem Context and Purpose

The orders collection acts as the primary transaction ledger for the e-commerce framework, preserving completed checkout details, payment processing metrics, and delivery parameters. When a customer completes a checkout loop via the web application, the system converts the temporary cart session into a permanent tracking node inside this collection.

The administration backend reads this path continuously to manage fulfillment pipelines, update order dispatch stages, look up delivery tokens, and verify financial accounting updates. Isolating individual purchases into self-contained document nodes protects transactional histories from changes to the core product inventory.

B. Document ID Structure & Storage Mechanics

- **Document ID Type: 20-character secure alphanumeric hash key.**
- **Generation Engine: Generated automatically by the Cloud Firestore SDK upon verified checkout validation loops (e.g., 0JBipwJR1Uop5bHOqBcA).**
- **Storage Pattern: Document structures containing flat primitive fields, a nested metadata map (customer), and a dynamic item array (selected_items).**

C. Field-Level Structural Rules

Table 3.13: orders Root-Level Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
amount	Number (Integer)	No	250	Total numeric transaction value calculated for the purchase.
created_at	Timestamp	No	May 14, 2026 at 11:47:40 AM UTC+8	System timestamp tracking exactly when the order record was

				created.
custom_components	Map / Null	Yes	null	Contains configuration attributes if the record handles a customized project layout.
delivery_date	Timestamp	No	May 15, 2026 at 10:00:00 PM UTC+8	Fulfillment milestone specifying the scheduled receipt date.

delivery_token	String	No	"FJZ8dEV94oxGy7nKhz2wCQ"	Secure alphanumeric verification token used to confirm delivery drop-offs.
delivery_type	String	No	"Pickup"	Mode of logistics fulfillment (e.g., "Pickup", "Delivery").
discount_amount	Number (Integer)	No	0	Deductible balance subtracted if a valid coupon

				code was applied.
inspo_image	String / Null	Yes	null	Absolute reference link to an external image upload if the user provided design references.
item	String	No	"Pistachio Cheesecake (₱250)"	Summary text string summarizing the primary line items for rapid admin

				previews.
notes	String	Yes	""	Optional text area string storing specific instructions provided by the buyer.
order_type	String	No	"premade"	Categorization tag separating ready-made menu stock from custom designs (e.g., "premade", "custom").

payment_id	String / Null	Yes	null	External transaction ID reference mapped from the integrated payment gateway.
payment_method	String	No	"Cash on Delivery"	Verification string logging the chosen payment modality (e.g., "Cash on Delivery", "Online Payment").

payment_status	String	No	"Pending"	Core transaction workflow state monitoring collection tracking (e.g., "Pending", "Paid").
reviewed	Boolean	No	true	System toggle tracking whether the client completed a feedback submission for the transaction.

rush	Boolean	No	false	Priority state modifier tracking whether expedited fulfillment processing was requested.
status	String	No	"Completed"	Master administrative status tracking order completion stages (e.g., "Pending", "Accepted", "Completed").

user_id	String	No	"KPanLhrWCZfxGDATn4AfH6Ipd1E3"	Unique cross-reference ID mapping the transaction directly to a user account profile.
voucher_discount	Number (Integer)	No	10	Total percentage or value reduction applied by a voucher match.
voucher_id	String / Null	Yes	"n3Zl2lpz5gFCQdYQsGdH"	Unique identifier string

				tracking the specific voucher used during checkout.

Table 3.14: orders Nested customer Map Schema

This sub-map organizes contact and delivery details for the buyer inside the single document envelope.

Nested Field Key	**Storage Primitive**	**Nullable?**	**Mock/Live Sample Data**	**Operational Purpose**
address	**String**	**No**	"Pick Up at Shop"	**Physical shipping location string or delivery collection directive.**

age	**String**	**Yes**	""	**Optional age attribute provided during checkout.**
celebrant	**String**	**Yes**	""	**Optional descriptive context noting the event celebrant's identity.**
contact	**String**	**No**	"09123131232"	**Verified telephone or mobile number used for logistical notification routing.**

lat	**Number / Null**	**Yes**	null	**Geographical latitude coordinate pin used for delivery address lookup.**
lng	**Number / Null**	**Yes**	null	**Geographical longitude coordinate pin used for delivery address lookup.**
name	**String**	**No**	"Justin Solamillo"	**Full identity name string of the customer receiving the order fulfillment.**
occasion	**String**	**Yes**	""	**Optional event theme description string used for**

				baking customization context.

Table 3.15: orders Nested selected_items Array Schema

This array contains individual object dictionaries for each product included in the transaction.

Array Object Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
cake_id	**String**	**No**	"OVJABOZNjwwYHp7Vbzhv"	**Reference ID linking directly to the product source entry in the** cakes **collection.**

cake_name	**String**	**No**	"Pistachio Cheesecake"	**Product title rendered on order management summaries.**
category	**String**	**No**	"Dessert"	**Catalog grouping label used to help assemble production line tasks.**
image_url	**String**	**No**	"https://res.cloudinary.com/..."	**Absolute Cloudinary CDN resource URL used to display item previews within**

				receipts.
max_qty	**Number (Integer)**	**No**	8	**Snapshot tracking maximum inventory stock parameters when checkout completed.**
price	**Number (Integer)**	**No**	250	**Item unit cost calculated at the exact historical moment of transaction initialization .**

quantity	**Number (Integer)**	**No**	1	**Total item count purchased within the transaction line.**
subtotal	**Number (Integer)**	**No**	250	**Total subtotal cost matching product quantities against unit values.**

D. Operational Flow and Security Considerations

Because information within the orders tree governs financial records and client delivery metrics, access endpoints are strictly isolated using security rules. Customers are granted read privileges exclusive to documents matching their active identity signature (allow read: if request.auth.uid == resource.data.user_id;). Creation tasks are programmatically checked against server timeline rules to make sure requested dates are not flagged as locked. Update permissions for tracking modifiers are blocked entirely unless authenticated with high-level administrative

credentials (request.auth.token.role == 'ADMIN').

3.9.11 Root Collection: pending_orders

A. Subsystem Context and Purpose

The pending_orders collection manages active checkout sessions and unverified order configurations. It acts as a temporary holding area where order metadata is staged while awaiting external payment provider confirmation or formal checkout completion.

When a user initiates a checkout flow, the frontend app writes the pending selection details here. Once payment is confirmed by the system's integration hooks, backend processes migrate the document into the core orders collection and clear the pending slot. This design keeps incomplete checkout attempts from cluttering the main transaction logs.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash key.
- **Generation Engine:** Provisioned automatically by the cloud database layer upon checkout initialization (e.g., 0p8Lh8DIdfSbeA4U9wZq).
- **Storage Pattern:** Structurally identical to the orders collection schema, utilizing flat primitive attributes along with nested customer maps and selected_items object arrays.

C. Field-Level Structural Rules

Table 3.16: pending_orders Root-Level Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
amount	Number (Integer)	No	500	The calculated balance value due for the unverified checkout transaction.
created_at	Timestamp	No	May 19, 2026 at 4:32:00 PM UTC+8	Log tracking the exact time the pending checkout

				instance was first saved.
custom_components	Map / Null	Yes	null	Contains active customizer attributes if tracking a personalized design project.
delivery_date	Timestamp	No	May 22, 2026 at 12:00:00 AM UTC+8	Staged fulfillment date picked by the customer during checkout.

delivery_token	String	No	"qg2sehks6KPanLhrWCZfXG"	Staged verification token string generated for subsequent drop-off authenticati on.
delivery_type	String	No	"Delivery"	Selected mode of logistics fulfillment (e.g., "Delivery", "Pickup").
discount_amount	Number (Integer)	No	0	Deductible balance reduction applied if

				an active coupon matches.
inspo_image	String / Null	Yes	null	Reference URL pointing to external layout image file assets provided by the buyer.
item	String	No	"Classic Chocolate Cake (₱500)"	High-level text summary summarizing line items for quick backend table

				rendering.
notes	String	Yes	"Deliver in the afternoon"	Text area string preserving specific delivery or layout directives from the user.
order_type	String	No	"premade"	Classification tag separating catalog items from custom designs (e.g., "premade", "custom").

payment_id	String / Null	Yes	"pay_xyz123456789"	Transaction path reference ID mapped from the external payment processor gateway.
payment_method	String	No	"Online Payment"	Logging string recording the selected payment pathway (e.g., "Online Payment", "Cash on Delivery").

payment_status	String	No	"Pending"	Workflow variable tracking state changes (e.g., "Pending", "Processing").
reviewed	Boolean	No	false	Default safety bit initialized to false, blocking review tasks before fulfillment.
rush	Boolean	No	false	Priority toggle tracking

				whether expedited processing surcharges apply.
status	String	No	"Pending"	Administrative status flag tracking staged checkout sessions.
user_id	String	No	"NnD912grZsOAqzVQlMuFEmFgJ1D3"	Core user profile hash mapping the transaction to a registered client.

voucher_discount	Number (Integer)	No	0	Numeric reduction value tracked when matching voucher entities.
voucher_id	String / Null	Yes	null	Unique identifier string tracking an applied voucher document.

Table 3.17: pending_orders Nested customer Map Schema

This sub-map organizes contact and fulfillment data fields for the buyer during checkout staging.

Nested Field Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
address	String	No	"123 Baker Street, Cebu City"	Physical shipping location string or delivery collection directive.
age	String	Yes	""	Optional age attribute provided during checkout.
celebrant	String	Yes	""	Optional context naming the event celebrant.
contact	String	No	"09171234567"	Telephone or mobile number used for logistical notification

				routing.
lat	Number / Null	Yes	10.3157	Geographical latitude coordinate pin used for delivery lookup.
lng	Number / Null	Yes	123.8854	Geographical longitude coordinate pin used for delivery lookup.
name	String	No	"Althea Marie Roa"	Full name string of the customer receiving the order.
occasion	String	Yes	""	Optional event theme description string used for

				baking customization context.

Table 3.18: pending_orders Nested selected_items Array Schema

This array contains individual object dictionaries for each product included in the staged transaction session.

Array Object Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
cake_id	String	No	"Z8Batr30nbAbX0OTrtpV"	Reference ID linking directly to the product source entry in the cakes collection.

cake_name	String	No	"Classic Chocolate Cake"	Product title rendered on order summaries.
category	String	No	"Dessert"	Catalog grouping label used to help assemble production line tasks.
image_url	String	No	"https://res.cloudinary.com/..."	Absolute Cloudinary CDN resource URL used to display item previews.

max_qty	Number (Integer)	No	5	Snapshot tracking available stock parameters during checkout.
price	Number (Integer)	No	500	Item unit cost calculated at the moment of checkout initiation.
quantity	Number (Integer)	No	1	Total item count purchased within the transaction line.

subtotal	Number (Integer)	No	500	Total subtotal cost matching product quantities against unit values.

D. Operational Flow and Security Considerations

Security controls for pending_orders restrict access vectors based on active user context constraints. Customers retain creation, read, and delete permissions for entries matching their identity signature (allow read, write: if request.auth.uid == resource.data.user_id;). This lets users clear stale cart data or cancel checkout attempts. Update actions on critical transaction values, payment fields, or status definitions are blocked unless authorized by administrative roles or verified server webhook routines.

3.9.12 Root Collection: reviews

A. Subsystem Context and Purpose

The reviews collection serves as the public feedback and rating repository for the storefront web application. It stores user-submitted testimonials, text commentaries, and satisfaction scores linked to verified transactions.

The client application reads from this data tree to render community feedback sections under individual item display cards and compute product scoring distributions. This provides direct social proofing to prospective buyers while allowing business supervisors to monitor overall customer satisfaction indicators from the control panel.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash key.
- **Generation Engine:** Generated automatically by the Cloud Firestore SDK when a user commits a new feedback entry (e.g., 5WnAnrG3I7T6XbVfPdQs).
- **Storage Pattern:** Flat document structure tracking independent customer feedback events.

C. Field-Level Structural Rules

Table 3.19: reviews Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose

cake_id	String	No	"OVJABOZNjwwYHp7Vbzhv"	Cross-reference identifier mapping the rating directly to a specific item inside the cakes catalog.
cake_name	String	No	"Pistachio Cheesecake"	Descriptive product label preserved within the entry envelope to optimize rendering lists.

comment	String	Yes	"Super sarap! Highly recommended"	Text area value preserving the written open-ended feedback commentary provided by the customer.
created_at	Timestamp	No	May 19, 2026 at 5:10:24 PM UTC+8	Explicit chronological tracking marker storing the exact time the rating was committed to the

				platform.
`customer_name`	String	No	`"Justin Solamillo"`	Text display name mapping the feedback to a public reviewer persona.
`order_id`	String	No	`"JPyR9H6T1RnGjrqx1org"`	Unique reference code matching the transaction record to confirm the submission is from a

				verified buyer.
rating	Number (Integer)	No	5	Numeric score constraint bounding user satisfaction indicators (typically evaluated on a strict scale of 1 to 5).
user_id	String	No	"KPanLhrWCZfxGDATn4AfH6Ipd1E3"	Target account token key associating the record entry back

				to its originating user profile.

D. Operational Flow and Security Considerations

Security rules within the reviews tree use a balanced permission matrix to protect data integrity. Public browsers are allowed read visibility (allow read: if true;) to seamlessly stream dynamic review text and calculate item ratings.

However, creation tokens are restricted to authenticated users whose active credential matches the document's author descriptor (allow create: if request.auth.uid == request.resource.data.user_id;). This submission process is programmatically verified against the target transaction node to ensure a matching order exists and has not yet been reviewed. Modifying or deleting existing entries is disabled for public clients, restricting those actions to system administrators (request.auth.token.role == 'ADMIN') to prevent feedback tampering.

3.9.13 Root Collection: users

A. Subsystem Context and Purpose

The users collection serves as the central identity registry and profile repository for all registered accounts within the e-commerce application platform. When a customer creates an account via the frontend portal, the system instantiates a corresponding document within this

data path.

The platform reads from this collection to customize client-side greeting interfaces, pre-populate shipping and contact forms during checkout operations, verify access authorization roles, and maintain stable communication channels via logged email headers. By maintaining distinct user metadata fields in an isolated document tree, the database architecture securely decouples sensitive identity properties from dynamic order histories and public review records.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** Fixed alphanumeric unique user identification hash string (UID).
- **Generation Engine:** Provided directly by the Firebase Authentication service upon successful registration to guarantee exact matching configurations across authentication modules (e.g., 0bYQ3E67fDPlmCrsTz6X).
- **Storage Pattern:** Flat root reference nodes archiving individual user metadata attributes.

C. Field-Level Structural Rules

Table 3.20: users Field-Level Schema Rules

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational

				Purpose
address	String	Yes	"123 Baker Street, Cebu City"	The default physical residence or shipping destination string stored to simplify checkout loops.
contact	String	Yes	"09171234567"	Primary phone or mobile number utilized for shipping coordination and dispatch tracking updates.
created_at	Timestamp	No	April 15, 2026 at 2:14:05 PM UTC+8	Chronological system log capturing the

				exact date and time the user account was first registered.
email	String	No	"user.test@gmail.com"	Authenticated electronic mail path used for system sign-ins, billing confirmations, and data lookups.
name	String	No	"Jane Doe"	Full legal or display identity string rendered across public profile spaces and order invoice documents.

role	String	No	"CUSTOMER"	System access control string variable assigning interface privileges across the platform (e.g., "CUSTOMER", "ADMIN").

D. Operational Flow and Security Considerations

Information stored inside the users tree governs account roles and sensitive customer data. Global data collection queries are blocked by default to prevent credential scraping. Under active Firestore Security Rules, logged-in clients can read or modify data properties only if their active session token matches the document ID (allow read, update: if request.auth.uid == userId;).

Crucially, critical privilege fields like role are completely locked against public client mutations. Escalating a user profile to administrative status requires a direct database update through the system console or an authorized backend script using a verified administrator token (request.auth.token.role == 'ADMIN').

3.9.14 Root Collection: walkin_orders

A. Subsystem Context and Purpose

The walkin_orders collection serves as the offline sales ledger, tracking over-the-counter purchases made physically at the bakery storefront. By isolating physical storefront retail transactions into a dedicated data tree, the system keeps regular digital cart checkouts separate from point-of-sale operations.

The administration panel reads from this path to aggregate overall operational revenue and update cross-channel material balances. This design ensures that walk-in stock deductions immediately sync with the centralized inventory levels, preventing online buyers from reserving raw materials that have already been consumed by physical storefront sales.

B. Document ID Structure & Storage Mechanics

- **Document ID Type:** 20-character secure alphanumeric hash key.
- **Generation Engine:** Instantiated automatically by the Cloud Firestore SDK upon point-of-sale receipt confirmation (e.g., 1E7bZfQDoXPlmCrsTz7W).
- **Storage Pattern:** Structured documents composed of flat primitive attributes and an embedded item array (selected_items).

C. Field-Level Structural Rules

Table 3.21: walkin_orders Root-Level Field Schema

Structural Field Key	Storage Primitive	Nullable?	Mock/Live Production Sample Data	Technical Constraints & Operational Purpose
amount	Number (Integer)	No	500	The total gross monetary value collected over the counter for the physical transaction.
created_at	Timestamp	No	May 25, 2026 at 3:12:18 PM UTC+8	Explicit chronological tracking marker storing the exact time the sale was logged at the register.

customer_name	String	No	"Walk-in Customer"	Standardized default identity placeholder used to denote unregistered physical shop buyers.
discount_amount	Number (Integer)	No	50	Deductible cash value subtracted directly at the point of sale for manual promotional overrides.
item	String	No	"Classic Chocolate Cake (₱500)"	Concise summary line used by administrative dashboard

				viewports for rapid invoice indexing.
payment_method	String	No	"Cash"	Verification indicator recording physical payment modalities (e.g., "Cash", "E-Wallet").
payment_status	String	No	"Paid"	Financial workflow state. For walk-in sales, this defaults immediately to "Paid".

status	String	No	"Completed"	Core tracking variable monitoring fulfillment execution (e.g., "Completed", "Cancelled").
voucher_discount	Number (Integer)	No	10	Total structural percentage or value reduction applied via physical token scanning.
voucher_id	String / Null	Yes	null	Reference identity string used if an active discount document was redeemed.

Table 3.22: walkin_orders Nested selected_items Array Schema

This array stores individual object maps for every item bundled within the physical point-of-sale transaction.

Array Object Key	Storage Primitive	Nullable?	Mock/Live Sample Data	Operational Purpose
cake_id	String	No	"Z8Batr30nbAbX0OTrtpV"	Core reference identifier matching the product source node within the main cakes collection.
cake_name	String	No	"Classic Chocolate Cake"	Descriptive retail item label rendered across administrative

				sales reports.
category	String	No	"Dessert"	Product sorting classification used to balance daily bakery production summaries.
image_url	String	No	"https://res.cloudinary.com/..."	Resource asset link used to display item thumbnail graphics within the administrative history logs.

max_qty	Number (Integer)	No	12	Snapshot variable capturing the maximum available warehouse stock at the time of purchase.
price	Number (Integer)	No	500	Baseline unit retail cost of the product at the precise moment of physical checkout.
quantity	Number (Integer)	No	1	Total physical unit count bought by the customer

				during the checkout event.
subtotal	Number (Integer)	No	500	Line item total cost calculated by multiplying the product unit value by the purchase quantity.

D. Operational Flow and Security Considerations

Because entries in the walkin_orders path directly represent realized store revenue and alter core inventory balances, their access rights are tightly restricted. Public web clients are completely blocked from reading or querying this path.

Under the system's Firestore Security Rules, write mutations and historical search queries are granted exclusively to authenticated administrative accounts (request.auth.token.role == 'ADMIN'). This security layer prevents external users from tampering with storefront revenue ledgers or falsifying physical sales records.

3.9.15 Section Summary

In summary, the design of the cloud database architecture relies on a fully decoupled, document-oriented structure across 14 distinct root collections. By isolating core operational vectors—such as public menus, digital checkouts, real-time messaging pipelines, and physical point-of-sale logs—into self-contained schema matrices, the system minimizes data redundancy while maximizing transaction security.

Furthermore, the implementation of explicit role-based constraints within the database security rules prevents unauthorized data exposure and preserves financial ledger integrity. This robust data foundation directly supports the high-speed synchronization, automated resource deduction, and dynamic calculation engines required by the integrated bakery management platform.

Note to the instructors handling this subject

CHED BSIT ALIGNMENT STATEMENT

This capstone project complies with CHED BSIT standards by demonstrating competencies in System development, database management, systems analysis and design, and emerging technologies such as machine learning. The project follows outcomes-based education principles and integrates ethical considerations, user safety, and data privacy.